

Lookup Arguments over Rings and Applications to Batch-Verification of RAM Programs

Jonathan Bootle
IBM Research Zürich
jbt@zurich.ibm.com

Julia Guskind
Boston University
guskinju@bu.edu

Sikhar Patranabis
IBM Research India
sikhar.patranabis@ibm.com

Katerina Sotiraki
Yale University
katerina.sotiraki@yale.edu

March 6, 2026

Abstract

Lookup arguments are a key technique in SNARKs for reducing the cost of operations which are “arithmetization-unfriendly”, such as range checks and bitwise comparisons. The idea is to encode the valid outputs of the operation in a publicly known table, and then prove that every element of the SNARK witness belongs to the table. Existing constructions for lookup arguments, however, are designed for working over fields, making them incompatible with recent post-quantum lattice-based schemes that operate over rings.

In this work we formalize lookup arguments over rings for tables containing arbitrary ring elements. We bring attention to systematic issues that arise when translating techniques from fields to rings by showing several known lookup arguments are susceptible to attacks. We then extend two central polynomial IOPs, Plookup and LogUp, over the ring $\mathcal{R} = \mathbb{Z}_q[X]/(X^d + 1)$, and show how to compile them with polynomial commitments based on lattice assumptions to get succinct lattice-based lookup arguments. We additionally show how to apply ring lookups to obtain succinct arguments for batch-verification of RAM updates where the RAM entries are arbitrary ring elements.

Keywords: lookup arguments, PIOPs, ring arithmetic, lattices, batch-verifiable RAM

Contents

1	Introduction	4
1.1	Contributions	5
1.2	Related Work	7
1.3	Organization	9
2	Technical Overview	10
2.1	Common Challenges to building lookups over rings	10
2.2	Ring-Plookup	10
2.3	Ring-LogUp	12
2.4	Succinct Lattice-Based Lookup Arguments	13
2.5	Application to Batch-Verification of RAM Updates	14
3	Preliminaries	16
3.1	Notation	16
3.2	Rings	16
3.3	Lookup Arguments	17
4	Barriers to Directly Instantiating Lookup Arguments over Rings	18
5	PIOPs for Lookups over Rings	23
5.1	Ring-Plookup	23
5.2	Ring-LogUp	26
6	PIOP for Batch-Verification of RAM Updates over Rings	34
6.1	RAMs are Memory Consistent with Update	36
6.2	Multilinear PIOP for Almost Identical RAM states	40
6.3	Putting Everything Together	43
A	Additional Preliminaries	45
A.1	Vector operations	45
A.2	Holographic Polynomial IOPs	45
A.3	Sum-check Protocol over Rings	46
B	Sub-protocols	48
B.1	Scalar Products	48
B.2	Hadamard Product	49
B.3	Cyclic-shift test	49
B.4	Entry-product protocol	51
B.5	Integer Check	52
B.6	Binary Check	53
C	Plookup Vector Relation Proof	56
D	Proof of Lemma 5.6	57

E	Ring-LogUp For Tables with Special Entries	58
F	Proof of Lemma 5.12	60
	Acknowledgements	63
	References	63

1 Introduction

The goal of verifiable computation is to guarantee integrity by ensuring an untrusted party performs a task correctly. A succinct non-interactive argument of knowledge, also known as a SNARK [Bit+14], is a key tool for verifiable computation in which a prover convinces a verifier that a statement is true by sending a short proof in a single round of communication.

A recent line of work has explored *lookup arguments*, which are a central technique to mitigate the cost of SNARK-unfriendly operations (i.e., those that are not efficiently represented as an arithmetic circuit). The idea is to encode all valid outputs of the operation in a publicly known table and then prove that the SNARK witness is contained in the table. Indeed, even the simple task of performing bitwise comparison of two field elements typically requires the binary representation of the elements, which incurs a logarithmic overhead. Other examples of SNARK-unfriendly operations that can be handled by lookup arguments include hash function evaluation and proving a value is within a finite range.

Since they were first introduced by Arya [BCGJM18] and Plookup [GW20], lookup arguments have grown in popularity (see [HACF25] for a recent survey) and have found several applications including zero-knowledge virtual machines [STW23; AST23], verifiable computation on encrypted data [CCCFGSV25], verifiable RAM computation [DGPPS24], non-arithmetic operations in deep learning for zero-knowledge LLMs [SLZ24], and “a la carte” SNARKs where the prover cost grows with the size of the circuit [DXNT23; CGGSS23]. There are several flavors of lookup arguments which are optimized for different settings, including, but not limited to, indexed lookups [STW23] (which fixes the order of the lookup query), matrix-lookups [CFFLL24; CGGSS23; DXNT23] (which looks up a tuple instead of a single value), and structured-table lookups [STW23] (in which the lookup table can be decomposed into several smaller lookup tables).

Today, the most popular, general-purpose SNARKs are designed to handle computations that can be naturally represented as an arithmetic circuit over a large prime order finite field [Gro16; GWC19; Mar19]. However, these SNARKs usually rely on discrete-log or pairing-based assumptions which are insecure against a quantum adversary. As a result, there has been a recent surge in the design of *post-quantum* proof systems. One of the most promising post-quantum paradigms is that which relies on polynomial lattice-rings. These lattice-based proof systems [BBCPGL18; ENS20; BLNS20; ACK21; LNP22; BS22; BCS23; NS24; BC24; BC25] offer a competitively small proof size, fast verifier, and low memory costs. Even beyond lattice-based proof systems, computation over rings is prevalent in more general settings such as proving operations done on homomorphically encrypted data [PLFT25; CCCFGSV25] and computations that involve fixed or floating point arithmetic [GNSV21].

Despite the independent progress on both lookup arguments and verifiable computation over rings, their overlap, namely *lookup arguments over rings*, is scant. In fact, majority of prior lookup argument constructions are designed for finite fields (or depend on a polynomial commitment scheme such as KZG [KZG10] which must be instantiated over fields) to ensure compatibility with general purpose SNARKs. A naive attempt to “directly translate” prior works from fields to rings often fails due to subtleties in the algebraic underpinnings of the lookup relations, as their security proofs often rely on field-specific properties such as unique factorization and multiplicative inverses.

Dually motivated by (1) the growing interest in verifiable computation over rings and (2) a systematic oversight in the literature which poses barriers to extending lookup arguments from fields to rings, we ask:

Can we build a post-quantum lookup argument that directly supports arbitrary ring elements?

In this work, we answer in the affirmative and further formalize the study of lookup arguments over rings.

1.1 Contributions

In this work, we formalize the study of lookup arguments over rings. We start by identifying challenges that arise when extending prior works from fields to rings. Then, we show how to resolve these issues by introducing two new indexed lookup arguments over rings, which we call Ring-Plookup and Ring-LogUp. We first describe polynomial IOPs for the lookup relations, and then discuss the process of compiling our IOPs to succinct arguments using lattice-based polynomial commitments over rings. Finally, as an application of our ring lookups, we give a new protocol for batch verification of RAM programs over rings.

A more detailed summary of our contributions is as follows.

1. **Challenges with extending prior works to rings.** In this work, we identify two major obstacles in grand-product arguments which prevent prior works on lookup arguments over *field* elements from being directly extended to *arbitrary ring* elements. We take inspiration from prior works [BLMG25; CMM19] which give attacks on shuffle arguments (which are a special case of lookups). We generalize these attack techniques to lookup arguments and show that trivially restating the Plookup [GW20] and Lasso [CCCFGSV25; STW23] relations over *rings* is vulnerable to an attack that swaps entries in the CRT components from which the ring splits. Additionally, we identify a second type of attack due to the presence of zero-divisors in the ring and show how it applies when trivially extending the LogUp proof technique [Hab22] over rings.
2. **New lookup arguments over rings.** We give two new protocols for indexed lookup arguments and prove security directly over rings. Specifically, we carefully modify the prior works Plookup [GW20] and LogUp [Hab22], both of whose lookup relations were originally stated over fields. We call our new lookup arguments Ring-Plookup and Ring-LogUp, respectively. Our results require careful algebraic techniques to handle the aforementioned challenges that arise when using grand-products over polynomial rings instead of fields, such as lack of both unique factorization and multiplicative inverses. Along the way, we formalize a toolkit of several PIOPs over rings for vector relations, offline memory-checking, binary-valued vector tests, and integer-valued vector tests. We also discuss how to use lattice-based proof systems and polynomial commitments [Ngu24] to compile our polynomial oracle proofs to a succinct, non-interactive argument.

Below we present informal theorem statements for the efficiency of our Ring-Plookup and Ring-LogUp lookup argument PIOPs. Throughout this work, we consider the ideal lattice $\mathcal{R} := \mathbb{Z}_q[X]/(X^d + 1)$ for prime q and power-of-two d , which ensures compatibility with mainstream lattice-based proof systems. Note that in a PIOP, “proof length” refers to the total length of all oracles that get sent during the interaction.

Theorem 1.1. *(Informal Ring-Plookup PIOP (Lemma 5.6)) Construction 5.5 is a PIOP for a lookup argument of size M into a table of size N that satisfies the indexed relation R_{ILU} over lattice-rings $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ and has the following efficiency parameters*

- Soundness error: $O(\frac{N+M}{2^d} + \frac{d+\log M}{q})$
- Round complexity: $O(\log(N+M))$
- Proof length: $O(N + \text{poly}(d)M)\mathcal{R}$
- Prover cost: $O(N + \text{poly}(d)M)\mathcal{R}$
- Verifier cost: $O(\log(N+M) + d \log M)\mathcal{R}$
- Query complexity: $O(d)$

where 2^d is the size of the set of all ring elements with binary coefficients and q is the size of the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

Theorem 1.2. (Informal Ring-LogUp PIOP (Lemma 5.12)) Construction 5.11 is a PIOP for a lookup argument of size M into a table of size N that satisfies the index relation R_{LU} over lattice-rings $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ of the form $\mathcal{R} \cong \mathcal{R}_1 \times \mathcal{R}_2$ and has the following efficiency parameters

- Soundness error: $O(\frac{M+N}{2^d} + \frac{d+\log M+\log N}{q})$
- Round complexity: $O(\log M + \log N)$
- Proof length: $O(N + \text{poly}(d)M)\mathcal{R}$
- Prover cost: $O(N + \text{poly}(d)M)\mathcal{R}$
- Verifier cost: $O(\log N + d \log M)\mathcal{R}$
- Query complexity: $O(d)$

where 2^d is the size of the set of all ring elements with binary coefficients and q is the size of the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

- Batch verification of RAM programs over rings.** As an application of our ring lookup arguments, we design a new protocol for batch-verification of RAM updates over rings. We follow the high-level blueprint from prior work over fields [DGPPS24] and use careful techniques to extend the building blocks to rings. In particular, we introduce new multilinear PIOPs over rings for offline memory-checking and for testing that two vectors are equal everywhere *except* for a subset of known indices. The efficiency of our RAM protocol is as follows.

Theorem 1.3. (Informal Batch Verification of RAM (Theorem 6.13)) Theorem 6.13 is a PIOP for batch-verification of updates over a RAM with entries from a lattice-ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ that has the following efficiency properties for a RAM of size N and M read/write operations:

- Soundness error: $O(\frac{M+N}{2^d} + \frac{d+\log M+\log N}{q})$
- Round complexity: $O(\log N + \log M)$
- Proof length: $O(N + \text{poly}(d)M)\mathcal{R}$
- Prover cost: $O(N + \text{poly}(d)M)\mathcal{R}$
- Verifier cost: $O(\log N + d \log M)\mathcal{R}$
- Query complexity: $O(d)$

where 2^d is the size of the set of all ring elements with binary coefficients and q is the size of the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

1.2 Related Work

We provide a summary on prior techniques for constructing lookup arguments. Most practically efficient lookup arguments for general lookup tables use one of two main underlying techniques: grand-product or log-derivatives. We briefly explain below that these techniques cannot directly be instantiated over rings, thus requiring new techniques for lookups with arbitrary ring elements.

1. **Grand product arguments.** Early works on lookup arguments, including Plookup [GW20], Arya [BCGJM18], and Halo2 [BGH20], use grand-product claims. At a high level, a grand product argument proves a valid lookup by showing that a randomized *product* of the values from both the lookup query vector and the table vector are equal. This is translated to a polynomial identity test by “embedding” the lookup table and query values as roots of linear factors of univariate polynomials. However, as observed by prior works on permutation arguments [BLMG25; CMM19], grand-product claims must be carefully handled over rings because rings are not unique factorization domains.
2. **Logarithmic-derivatives.** The logarithmic-derivatives approach was first introduced by LogUp [Hab22] and later used by several other lookup arguments including Flookup [GK22], cq [EFG22], LogUp-GKR [PH23], cq+, zkqcq+, and cq++ [CFFLL24], Locq [ZSG24], μ -seek [CFG25], LogSpartan [GPS25], Logallookup [TNLKN25], Logup* [Sou25], and tlookup [SLZ24].

The log-derivatives technique remains a popular approach in the lookup argument literature for several reasons. First, the PIOP from [Hab22] improves upon Plookup in the batched lookup setting, where the validity of multiple lookup queries is proved simultaneously. This is because the LogUp prover need only send a constant number of oracles by combining all lookup instances into a single vector. However in Plookup the prover must create an auxiliary vector and send an oracle for each instance in the batch. Additionally, the log-derivatives technique achieves an asymptotically faster prover compared to other approaches (see [ZSG24, Table 1]) when table preprocessing is allowed.

The idea of LogUp is to express the lookup claim as a *summation* of rational terms, where each denominator contains a value from the lookup query or a value from the lookup table. To prove the log-derivatives claim, [Hab22] uses a special grand-product statement where soundness relies on all field elements having multiplicative inverses. In the interactive lookup protocol, the prover computes the individual rational terms within the summation, treats them as polynomial evaluations, and then reduces the lookup claim to polynomial identity tests via sum-check. As a result, this approach does not immediately extend to rings because multiplicative inverses are not guaranteed.

Lookups for Structured Tables. Lasso [STW23] is a recent lookup argument that supports tables with special structure. Subsequent works on structured lookups include Jolt [AST23], TaSSLE [Dor24], Shout [ST25], and the Lasso over rings protocol from [CCCFGSV25]. According to Lasso, a structured table satisfies two properties, decomposability and MLE-structure. Decomposability means that the table can be expressed as a tensor product of smaller tables, so that a single lookup into the full table corresponds to several lookups into the small tables. MLE-structure means that the low-degree (not necessarily multilinear) extension polynomial of each small table can be evaluated in logarithmic time. These protocols are based on proving a special form of matrix-vector

multiplication based on the observation that if $\mathbf{a} \subseteq \mathbf{b}$, then for each $\mathbf{a}_i$ there is a one-hot vector $\mathbf{v}_i$ such that $\mathbf{v}_i^T \mathbf{b} = \mathbf{a}_i$. Then, it suffices for the prover to show that there exists a binary matrix $V \in \{0, 1\}^{M \times N}$ (whose rows are one-hot vectors) such that $V\mathbf{b} = \mathbf{a}$. By taking advantage of the underlying table structure, the prover cost depends on the size of their lookup query and the size of a sub-table, instead of the size of the full table. While this line of work offers competitive performance by taking advantage of the sparse matrix-vector constraints, it does not support arbitrary, unstructured tables.

Lookups with Sublinear Prover. Early works on lookup arguments have a prover cost that depends on the size of the table itself. However, by preprocessing the table in (quasi)-linear time, subsequent works have achieved an online prover whose cost is sublinear in the table length (i.e., depends on the length of the lookup query). Sublinear prover lookups have been achieved in both the log-derivatives and matrix-vector product setting. This line of work includes Baloo [ZGKMR22], Caulk [ZBKMNS22], Caulk+ [PK22], Flookup [GK22], cq [EFG22], cq+ [CFLL24], Locq [ZSG24], and μ -seek [CFG25]. All protocols use the univariate KZG polynomial commitment scheme [KZG10] for table preprocessing and (except for μ -seek [CFG25]) also use KZG for the lookup protocol itself. However, the KZG commitment scheme currently does not have a lattice-based analogue and therefore it is unclear how to leverage quotient polynomials over lattices for faster lookups. Indeed, [ACLM22] presented a promising lattice-based vector commitment scheme that mimics the algebraic techniques from pairing-based schemes. However, the underlying assumption, k -R-ISIS, was later morally broken by [WW24] leading to doubts about the scheme’s knowledge soundness.

Separately, recent work [CCCFFG25] give a sublinear prover lookup argument that, at a high-level, combines Kedlaya-Umans polynomial preprocessing [KU08; KU11] with Generalized-Lasso [STW23] and only makes black-box use of an underlying vector commitment scheme. While Kedlaya-Umans technique is known to work for extension rings of the form $\mathbb{Z}_q[x]/(E(x))$ for any monic polynomial $E(x)$, we do not explore this approach due to its extreme practical inefficiency.

Duplex [HYOK25] and Notus [XHTP24] are recent works that achieve a sublinear prover and deviate from prior approaches by building a lookup argument using RSA accumulators over an RSA group. However, this approach is inherently not post-quantum secure.

Finally, we remark that Baloo [ZGKMR22] uses the matrix-vector approach which exploits sparsity to succinctly describe a binary matrix $V \in \{0, 1\}^{M \times N}$. It is not immediately clear how to efficiently handle these sparse matrices over lattice rings for lookup arguments, and we leave an exploration of this approach to future work.

Proof Systems over Rings. LatticeFold [BC24] and LatticeFold+ [BC25] are recent works that give lattice-based folding schemes and range proofs over rings to prove a vector of ring elements has small norm. The LatticeFold+ range proof can be extended to a lookup, but it has limited expressivity since it only supports integral-valued ring elements (i.e., constant term polynomials).

Zinc [GWHD25] introduces a hash-based SNARK to prove statements over the integers. One of the contributions is a framework for proving NP statements over $\mathbb{Q}$ wherein the witness must contain entries in $\mathbb{Z}$.

Finally, recent work on verifiable FHE [CCCFGSV25] extends Lasso from fields to rings. We identify a bug in the lookup argument theorem in the first version of this work. This issue has been fixed in a subsequent version by instead proving the lookup argument over fields directly and using a table which consists of small integers that are units in the full ring.

1.3 Organization

The rest of the paper is organized as follows. In Section 2 we provide a technical overview of our results. In Section 3 we present notation and preliminaries. In Section 4 we show attacks on trivial extensions of lookups over rings and on the first version of the lookup proposed in [CCCFGSV25]. In Section 5 we present two new PIOPs for lookup arguments over rings, Ring-Plookup (Section 5.1) and Ring-LogUp (Section 5.2). In Section 6 we describe how to apply these lookup arguments to achieve lattice-based arguments for batch-verification of RAM updates.

2 Technical Overview

Lookup arguments. A lookup argument is commonly expressed as a set membership check. The goal is to prove that $\mathbf{a} \subseteq \mathbf{b}$, where the lookup table is a vector $\mathbf{b} \in \Sigma^N$ with length N and the lookup query is a vector $\mathbf{a} \in \Sigma^M$ with length M for some alphabet Σ . This is equivalent to proving that for every index $i \in [M]$ in the lookup query, there exists some index $j \in [N]$ in the table such that $\mathbf{a}_i = \mathbf{b}_j$. Throughout this work, we allow the lookup query $\mathbf{a}$ to contain repeated entries but we assume that the entries in the public table $\mathbf{b}$ are unique. We do not consider zero-knowledge, though our results can be made zero-knowledge using standard proof composition techniques.

Nearly all prior works consider a lookup argument for entries in some finite field $\mathbb{F}$. In this work we consider lookups for unstructured tables containing arbitrary ring elements from the polynomial ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ for prime q and power-of-two d . This ring is a standard choice for lattice-based cryptography.

2.1 Common Challenges to building lookups over rings

Our goal is to construct lookup arguments that support tables containing arbitrary ring elements from $\mathcal{R}$. We bring awareness to common challenges that arise when translating lookup techniques over field to that over rings.

A straightforward approach to build a ring lookup is to simply take prior work over fields and directly run it over rings using, e.g., sum-check for rings [BCS21] and lattice-based polynomial commitments [CMNW24; NS24; HSS24; NOZ26]. However, in this work we systematically demonstrate that this approach is susceptible to attacks, since the security of many lookup arguments crucially relies on field-specific properties, including unique factorization and multiplicative inverses. In particular, we identify two common attacks.

1. **CRT-component swapping attack.** We generalize the attack technique identified in prior literature on permutation arguments over rings [BLMG25; CMM19]. Specifically, we can view a ring lookup argument over the CRT basis and show that each CRT slot may be a valid lookup instance, while their combination yields an invalid lookup over the full ring. We show that Plookup [GW20] and Lasso over rings presented in the first version of [CCCFGSV25] are susceptible to this attack.
2. **Zero-divisor attack.** Fields are integral domains (i.e., the product of non-zero elements is non-zero) which is a useful property to rely on when proving soundness of grand-products. We show that the presence of zero-divisors in a ring allows a cheating prover to construct a malicious grand-product polynomial that always satisfies a zero-test, thus breaking soundness. We use this to demonstrate that the LogUp proof technique [Hab22] is unsound over rings.

In this work we avoid these common pitfalls and instead provide techniques to safely extend popular lookup arguments from fields to rings.

2.2 Ring-Plookup

Original Plookup over fields. Plookup [GW20] is one of the first lookup arguments and uses a grand-product argument to prove that the evaluations of a committed polynomial over a finite field $\mathbb{F}$ are contained in a publicly known table of values.

We recall the original Plookup relation to show that $\mathbf{a} \subseteq \mathbf{b}$. Let $\mathbf{s} := \text{merge}_{\mathbf{b}}(\mathbf{a}) \in \mathbb{F}^{N+M}$ be the list given by $\mathbf{a}$ concatenated with $\mathbf{b}$, sorted by $\mathbf{b}$, i.e., the different values in $\mathbf{s}$ appear in the same order as they do in $\mathbf{b}$. The key observation of Plookup is that if $\mathbf{a} \subseteq \mathbf{b}$, the non-zero elements of $\{(\mathbf{s}_i + \beta \mathbf{s}_{i+1})\}_{i \in [N+M-1]}$ and $\{(\mathbf{b}_j + \beta \mathbf{b}_{j+1})\}_{j \in [N-1]}$ are the same, where $\beta \xleftarrow{\$} \mathbb{F}$. Choosing a random $\gamma \xleftarrow{\$} \mathbb{F}$, these lists of tuples are embedded into two separate bivariate polynomials

$$F(\beta, \gamma) := (1 + \beta)^M \prod_{i \in [M]} (\gamma + \mathbf{a}_i) \prod_{j \in [N-1]} (\gamma(1 + \beta) + \mathbf{b}_j + \beta \mathbf{b}_{j+1}) \quad (1)$$

and

$$G(\beta, \gamma) := \prod_{i \in [N+M-1]} (\gamma(1 + \beta) + \mathbf{s}_i + \beta \mathbf{s}_{i+1}). \quad (2)$$

Claim 3.1 of [GW20] proves that, over a finite field $\mathbb{F}$, $F \equiv G$ as bivariate polynomials if and only if $\mathbf{a} \subseteq \mathbf{b}$ and $\mathbf{s}$ is the vector obtained by concatenating $\mathbf{a}$ to $\mathbf{b}$ and sorting according to $\mathbf{b}$. At a high-level, the proof relies on unique factorization to show that every factor in F corresponds to a factor in G , which means for all $i \in [M]$, there is a $j \in [N]$ such that $\mathbf{a}_i = \mathbf{b}_j$ over the field. This polynomial relation is used to build an interactive argument: evaluating the bivariates F and G at the random point (β, γ) and checking that $\frac{F(\beta, \gamma)}{G(\beta, \gamma)} = 1$ reduces the lookup claim to a polynomial identity test.

This work: Ring-Plookup. In Section 4, we show that the original Plookup relation is unsound when instantiated over rings by giving an attack in which $\mathbf{a}$ is an invalid lookup into $\mathbf{b}$ yet the grand-product relation is still satisfied over the ring. Our attack takes inspiration from [BLMG25] by swapping the values within one CRT component to produce an invalid lookup query. Since the order of the lookup argument is not fixed across different components, this swapping is not “detected” by the Plookup relation and allows an invalid lookup query to still be valid within each component.

In Section 5.1, we show how to safely extend the original Plookup to the ring $\mathcal{R}$, avoiding the aforementioned attack. We first generalize the Plookup bivariate polynomial relation to rings using the technique introduced in [BLMG25]; we extend F and G to polynomials of three variables where the coefficient of the new variable is carefully chosen from a *challenge space* to guarantee that the two polynomials are equal if and only if $\mathbf{a} \subseteq \mathbf{b}$ over the ring. Informally, a challenge space is a subset of the ring such that the pairwise difference between elements is invertible. The coefficient on this new variable also enforces that the order of the lookup is consistent in each CRT component (i.e., it induces an *indexing* map on the lookup). Our new indexed Ring-Plookup relation is as follows

Lemma 2.1 (Ring-Plookup relation, informal). *Let $\mathcal{R}$ be a suitable ring. The following equality holds*

$$\begin{aligned} & \prod_{k=1}^{M+N-1} (\mathbf{s}_k + \mathbf{s}_{k+1} \cdot x_1 + \boldsymbol{\sigma}_k \cdot x_2 - x_3) = \\ & \prod_{i=1}^M (\mathbf{a}_i + \mathbf{a}_i \cdot x_1 + \mathbf{c}_i \cdot x_2 - x_3) \cdot \prod_{j=1}^{N-1} (\mathbf{b}_j + \mathbf{b}_{j+1} \cdot x_1 + g(j) \cdot x_2 - x_3) . \end{aligned} \quad (3)$$

over $\mathcal{R}[x_1, x_2, x_3]$, if and only if

1. $\mathbf{a} \in \mathcal{R}^M$ is a valid indexed lookup into table $\mathbf{b} \in \mathcal{R}^N$ with index vector $\mathbf{c} \in \mathcal{C}^M$;
2. $\mathbf{s} \in \mathcal{R}^{N+M}$ is the vector $(\mathbf{a}, \mathbf{b})$ sorted by $\mathbf{b}$;
3. $\mathbf{c} \in \mathcal{C}^M$ and $\boldsymbol{\sigma} \in \mathcal{C}^{M+N}$ such that $\sigma_{i+j-1} = \mathbf{c}_i = g(j)$ if and only if $\mathbf{a}_i = \mathbf{b}_j$,

where $g : [N] \rightarrow \mathcal{C}$ is a publicly known injective map and $\mathcal{C} \subseteq \mathcal{R}$ is a challenge space.

Finally, we present Ring-Plookup (Construction 5.5), a PIOP for the Ring-Plookup polynomial relation over rings. Our PIOP avoids performing a rational test on $\frac{F(\beta, \gamma)}{G(\beta, \gamma)}$ and instead proves the trivariate polynomial equality directly extending the sub-protocols and techniques from [BCG20] to rings.

2.3 Ring-LogUp

Original LogUp over fields. The logarithmic-derivatives approach was first introduced by LogUp [Hab22]. The key observation in LogUp is that a grand product statement can be transformed into a summation of rational terms by first taking its logarithm and then taking the first derivative. Informally, the LogUp relation (Lemma 5, [Hab22]) says that $\mathbf{a} \subseteq \mathbf{b}$ if and only if there exists a vector $\mathbf{m} \in \mathbb{Z}^N$, counting the multiplicity of each element of $\mathbf{b}$ as it appears in the look up query $\mathbf{a}$, such that

$$\sum_{i \in [N]} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j}. \quad (4)$$

The backwards direction of this bijective claim (i.e., if Equation (4) holds, then $\mathbf{a} \subseteq \mathbf{b}$) is proven in [Hab22] by rewriting Equation (4) as a summation over the entire field and clearing the denominator,

$$\sum_{v \in \mathbb{F}} (\mathbf{m}_{\mathbf{a}}[v] - \mathbf{m}_{\mathbf{b}}[v]) \prod_{z \in \mathbb{F}, z \neq v} (x - z) = 0 \quad (5)$$

where $\mathbf{m}_{\mathbf{a}}[v]$ counts the number of times v appears in $\mathbf{a}$ and $\mathbf{m}_{\mathbf{b}}[v] = \mathbf{m}[v]$ for $v \in \mathbf{b}$, otherwise $\mathbf{m}_{\mathbf{b}}[v] = 0$. At a high level, the argument is that if Equation (5) holds, it must be because $\mathbf{m}_{\mathbf{b}}[v] = \mathbf{m}_{\mathbf{a}}[v]$ for all $v \in \mathbb{F}$ since the product of non-zero elements in $\prod_{z \in \mathbb{F}, z \neq v} (x - z)$ is non-zero. So, by definition of $\mathbf{m}_{\mathbf{a}}$ and $\mathbf{m}_{\mathbf{b}}$, $\mathbf{a}$ is a valid lookup into $\mathbf{b}$.

The LogUp relation is used to build a PIOP in which the prover constructs an auxiliary polynomial by evaluating the individual rational terms in the summation from Equation (4) at a random field element. However, these values may not be computable over rings due to the lack of multiplicative inverses. Beyond this, we note that in [Hab22] the rational terms are viewed in the field of fractions. However, since rings are not integral domains it is possible that the denominator of a rational term is a zero-divisor and therefore the fraction is not well-defined.

This work: Ring-LogUp. In Section 4, we show that LogUp is unsound over rings by giving an attack that exploits the presence of zero-divisors in a ring. The idea is that a cheating prover can construct a polynomial that, when evaluated at any point in the ring, will always contain a product of zero-divisors. This polynomial will always satisfy a zero-test, even when the lookup query is invalid.

In Section 5.2 we fix this issue by introducing Ring-LogUp, a new lookup argument that safely extends LogUp to suitable lattice-rings. Similarly to the Plookup case, we achieve Ring-LogUp by

introducing a new indeterminate variable, i.e., a “tag”, in the denominator of the rational terms whose coefficient is carefully chosen from a special subset $\mathcal{C}$ of the ring. This also yields an *indexed* lookup relation, as the special coefficient will induce a map over the full ring between each entry in the lookup query and the corresponding index in the table. We observe that the proof technique from LogUp (1) relies on fields being integral domains and (2) involves rewriting Lemma 5.7 as a summation over the entire ring, at which point there are not enough special “tags” to embed in the denominator since $\mathcal{C} \subset \mathcal{R}$. We overcome these issues by proving our lookup relation using a different technique. Our new Ring-LogUp relation is as follows.

Lemma 2.2 (Ring-LogUp relation, informal). *Let $\mathcal{R} = \mathcal{R}_1 \times \mathcal{R}_2$ be a suitable ring. The following equality holds*

$$\sum_{i \in [M]} \frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 - x_2} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)x_1 - x_2} \quad (6)$$

if and only if

1. $\mathbf{a} \in \mathcal{R}^M$ is a valid indexed lookup into table $\mathbf{b} \in \mathcal{R}^N$ with index map $\mathbf{c} \in \mathcal{C}^M$;
2. there exists a vector $\mathbf{m} \in \mathbb{Z}_q^N$ that counts the multiplicity of each element in $\mathbf{a}$ as it appears in $\mathbf{b}$;
3. $\mathbf{c}_i = g(j)$ if and only if $\mathbf{a}_i = \mathbf{b}_j$,

where $g : [|\mathcal{C}|] \rightarrow \mathcal{C}$ is a publicly known bijection and $\mathcal{C} \subset \mathcal{R}$ is a challenge space.

Finally, we present a Ring-LogUp PIOP (Construction 5.11) for our new relation over rings that uses the standard sum-check approach from prior works. By working over a ring that decomposes into exactly two fields, we argue that the denominator of each rational term computed in the PIOP is invertible with high probability since it is a uniformly random ring element. We also ensure that each rational term in Equation (4) is well-defined by viewing it in the total ring of fractions, a generalization of the total field of fractions to non-integral domains.

2.4 Succinct Lattice-Based Lookup Arguments

We describe how the polynomial IOP protocols in our work can be used to construct succinct lattice-based lookup arguments.

Compiling PIOPs to Succinct Arguments. Following works such as [CHMMVW20], it is well-known that polynomial IOPs can be used to construct succinct arguments by compiling them using a polynomial commitment scheme. A polynomial commitment scheme works like a standard commitment scheme whose message space consists of polynomials over a suitable domain, with the addition of an *evaluation protocol*. The evaluation protocol is an interactive argument that allows a prover to convince a verifier that the polynomial inside a particular commitment satisfies a given input/output relation. To obtain a succinct argument, each oracle message sent by the prover as part of the polynomial IOP is replaced with a commitment to that message, and the answer to every polynomial evaluation query made by the verifier during the polynomial IOP is supplied by the prover. Thus, to compile the polynomial IOP protocols in our work to succinct lookup arguments, we require a succinct argument for polynomial evaluation as defined in the following relation.

Definition 2.3 (Evaluation relation, informal). *The **evaluation** relation $R_{\text{Eval}}(pp)$ is the set of tuples $(\mathbb{x}, \mathbb{w}) = ((\mathcal{R}, N, C, y, z), p, \rho)$ where $\mathcal{R}$ is a ring, $N \in \mathbb{N}$ is a degree bound, C is a commitment, ρ lies in the randomness space of the commitment scheme, and $y, z \in \mathcal{R}$ are respectively an evaluation point and an evaluation, C is a commitment to p using randomness ρ and $p(y) = z$.*

Example Instantiation with Lattice-Based Polynomial Commitments. We explain how to instantiate the above framework using the Greyhound polynomial commitment scheme [Ngu24]. Greyhound provides a lattice-based argument system for the following relation which has proof size $O(\log n)$ and verification time $O(\sqrt{n})$:

Definition 2.4. *Let R_{b_0, b_1} be defined as the set of tuples*

$$(\mathbb{x}, \mathbb{w}) = ((\mathcal{R}, A, B, D, \mathbf{a}, \mathbf{b}, \mathbf{u}, y, b_0, b_1, n, m, r), ((\mathbf{s}_i)_{i \in [r]}, (\mathbf{t}_i)_{i \in [r]}))$$

where $\mathcal{R} := \mathbb{Z}_p[x]/(x^d + 1)$ for prime p and power-of-two d , $A \in \mathcal{R}^{n \times m}$, $B \in \mathcal{R}^{n \times nr \log_{b_1}(p)}$, $D \in \mathcal{R}^{n \times r \log_{b_1}(p)}$, $\mathbf{a} \in \mathcal{R}^m$, $\mathbf{b} \in \mathcal{R}^r$, $\mathbf{u} \in \mathcal{R}^n$, $y \in \mathcal{R}$, $\mathbf{s}_i \in \mathcal{R}^m$ and $\mathbf{t}_i \in \mathcal{R}^{n \log_{b_1}(p)}$, satisfying

- $\forall i \in [r], A\mathbf{s}_i = G_{b_1, n}\mathbf{t}_i$, where $G_{b_1, n}$ is a gadget matrix containing powers of b_1 which takes the base b_1 decompositions of n integers and produces the n integers;
- $B(\mathbf{t}_1 || \dots || \mathbf{t}_r) = \mathbf{u}$;
- $\mathbf{a}^T[\mathbf{s}_1 | \dots | \mathbf{s}_r]\mathbf{b} = y$;
- $\forall i \in [r], |\mathbf{s}_i|_\infty \leq b_0/2$ and $\forall i \in [r], |\mathbf{t}_i|_\infty \leq b_1/2$.

Here, the matrices A and B are used to produce a commitment $\mathbf{u}$ to witness vectors $\mathbf{s}_i$.

Suppose that we would like to instantiate our polynomial IOPs and the message space from Definition 2.3 for polynomials over a ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^d + 1)$. In this case, a first simple instantiation would be to set b_0 in Definition 2.4 equal to q , and set $\mathbf{s}_1, \dots, \mathbf{s}_r$ to be the coefficients of a multilinear polynomial, arranged into a matrix S , and $\mathbf{t}_i$ and commitment $\mathbf{u}$ as specified, computed over a larger modulus $p \gg q$.

Our polynomial IOPs involve polynomial evaluation queries to multilinear extensions. Given a vector $\mathbf{s} = (s_0, \dots, s_{rm-1}) \in \mathcal{R}_q^{rm}$, and assuming for simplicity that rm is a power of 2, an evaluation y of its multilinear extension polynomial $\tilde{s}(X_1, \dots, X_{\log(rm)})$ at $x_1, \dots, x_{\log(rm)} \in \mathcal{R}_q$ can be written as $\langle \mathbf{s}, \bigotimes_{j=1}^{\log(rm)} (1 - x_j, x_j) \rangle$. Parsing $\mathbf{s}$ as a matrix $S \in \mathcal{R}_q^{r \times m}$, we can write $y = \mathbf{a}^T S \mathbf{b}$ where $\mathbf{a} := \bigotimes_{j=1}^{\log(r)} (1 - x_j, x_j)$ and $\mathbf{b} := \bigotimes_{j=\log(r)+1}^{\log(rm)} (1 - x_j, x_j)$. Thus, Greyhound is able to verify that evaluations of committed multilinear polynomials were computed correctly.

For improved efficiency, and to allow a smaller choice of p for Greyhound, it makes sense to decompose $S \in \mathcal{R}_q^{r \times m}$ into smaller integers using bit decomposition or another small base before committing to it. Replacing each column of S with bit decompositions of its entries, stacked vertically, this would produce a matrix $S' \in \mathcal{R}_2^{r \lceil \log(q) \rceil \times m}$. Greyhound can still be used to verify polynomial evaluations in this case by setting $\mathbf{b} = (1, 2, \dots, 2^{\lceil \log q \rceil - 1}) \otimes \bigotimes_{j=\log(r)+1}^{\log(rm)} (1 - x_j, x_j)$.

2.5 Application to Batch-Verification of RAM Updates

In Section 6 we describe an application of our lattice-based lookup arguments to realizing a new PIOP for batch-verification of RAM programs, where a prover convinces a verifier that the RAM

has been accessed by a sequence of read/write operations in a correct and consistent manner. It is standard to view a RAM V as a mapping between unique addresses and values. In particular, V is a vector of (address, value) pairs where addresses are integers and values are arbitrary ring elements. We allow two operations, reads and writes, to be performed on a RAM. A sequence of operations is a vector o where each operation is described by a tuple of three elements (operation, address, value). At a high-level, a batch-verification of RAM protocol seeks to prove that an initial RAM V and final RAM V' are consistent with respect to a known sequence of operations o .

To achieve batch-verification of RAM, we broadly follow the lookup argument-based blueprint for realizing batch-verification of RAM from [DGPPS24], which consists of three steps.

1. **Indexed lookup.** The prover isolates W , the portion of RAM that was accessed by the operations in o and proves that the values in W are consistent with the initial RAM V . This can be achieved using an indexed lookup argument, where the lookup query is the isolated portion W and the table is the full RAM V . The prover repeats this procedure on the isolated portion W' for the final RAM V' after applying the operations in o to V .
2. **Memory consistency.** The prover uses offline-memory checking [BEGKN91] to show that the isolated portions of initial and final RAM, W and W' (resp.), are consistent with respect to the operations in o .
3. **Almost identical RAM states.** The prover shows the initial RAM V and final RAM V' still have identical values at every address outside of the locations that were accessed by o . In [DGPPS24], this is done with a univariate PIOP.

Extending prior work to rings. The original protocol of [DGPPS24] works over finite fields of large prime order, and does not readily extend to RAMs containing arbitrary ring elements, making it incompatible with lattice-based proof systems. In particular, in the first step above, [DGPPS24] use a log-derivative style indexed lookup argument over finite fields. However, in this work we demonstrate that the original log-derivatives claim does not immediately extend to rings due to the presence of zero-divisors (see Section 5.2). To resolve this, we use our new Ring-LogUp lookup protocol which can handle RAM tables with arbitrary ring elements.

In both the second and third step, [DGPPS24] give a univariate PIOP which uses univariate polynomial division operations over fields. To extend the second step to rings, we introduce a new multilinear PIOP for offline-memory checking (see Section 6.1) over rings by adapting the techniques from [BEGKN91]. Our protocol avoids polynomial division by instead using Hadamard products (see Appendix B.2) and lookup arguments (see Section 5) over rings.

To address the third step, we give a new multilinear PIOP over rings for showing that two vectors are equal everywhere outside of a given set of indices. We describe the intuition of the protocol here with vectors. Let $\mathbf{u}$ be an indicator vector such that $\mathbf{u}[i] = 1$ if address i was accessed during the RAM operations in o and $\mathbf{u}[i] = 0$ otherwise. Then we observe that V and V' are equal everywhere *outside* of the indices in $\mathbf{u}$ if and only if $(1 - \mathbf{u}[i])(V[i] - V'[i]) = 0$ for all addresses i in the RAM. Our sum-check style PIOP for this relation is formally presented in Construction 6.9.

Putting these sub-protocols together yields the first PIOP for batch-verification of updates over RAMs where all entries are arbitrary ring elements, while being fully compatible with state-of-the-art lattice-based arguments and commitment schemes, e.g. Greyhound [NS24].

3 Preliminaries

3.1 Notation

For an integer $N \in \mathbb{N}$ we let $[N] := \{1, \dots, N\}$. We use boldface $\mathbf{a}$ to denote a vector. To obtain the vector entry at index i , we use subscript $\mathbf{a}_i$. For a vector that represents a key-value map, we use bracket notation $\mathbf{a}[\mathbf{b}_i]$ to denote the value stored in $\mathbf{a}$ at the entry indexed by key $\mathbf{b}_i$. We use $\mathbf{1}^N$ to denote the vector of all 1s with length N . We use $\text{int}(\cdot)$ to denote the map that sends a binary string to its integer representation. We use $\text{bin}(\cdot)$ to denote the map that sends an integer to a string representing its binary decomposition. For an integer q we use $\mathbb{Z}_q$ to mean the integers modulo q and we use $\mathbb{Z}_q^\times$ to denote the integers modulo q that are relatively prime to q .

3.2 Rings

We work over the polynomial ring $\mathcal{R} := \mathbb{Z}_q[X]/(X^d + 1)$ for power-of-two d and prime q .

Definition 3.1 (Integral domain). *An **integral domain (ID)** is a non-zero commutative ring in which for all a, b in the ring, $ab = 0 \implies a = 0$ or $b = 0$.*

Definition 3.2 (Unique factorization domain). *A **unique factorization domain (UFD)** is an integral domain in which every non-unit element can be written as a product of irreducibles. Furthermore, this decomposition is unique up to ordering and units.*

Definition 3.3 (Localization). *The **localization** of a commutative ring $\mathcal{R}$ by a set $\mathcal{S} \subset \mathcal{R}$ is a new ring denoted by $\mathcal{S}^{-1}\mathcal{R}$ consisting of fractions $\frac{r}{s}$ where $r \in \mathcal{R}$ and $s \in \mathcal{S}$. Furthermore, $\mathcal{S}$ is closed under multiplication and contains 1.*

Definition 3.4 (Total ring of fractions). *The **total ring of fractions** of a ring $\mathcal{R}$ is a new ring $Q(\mathcal{R}) := \mathcal{S}^{-1}\mathcal{R}$ obtained by localization where $\mathcal{S}$ is the multiplicatively closed set that contains all non-zero-divisors in $\mathcal{R}$.*

Definition 3.5 (Sampling space). *A **sampling space $\mathcal{C}$** in a ring $\mathcal{R}$ is a subset of $\mathcal{R}$ such that for every pair of distinct $u, v \in \mathcal{C}$ the difference $u - v$ is invertible in $\mathcal{R}$.*

In this work, we consider sampling spaces of exponentially large size.

Every ring element $y \in \mathcal{R}$ can be written as a polynomial of degree less than d whose coefficients are in $\mathbb{Z}_q$.

$$y \leftrightarrow y_0 + y_1X + \dots + y_{d-1}X^{d-1}$$

The ℓ_2 and ℓ_∞ norms of a ring element $y \in \mathcal{R}$ are taken over the vector of coefficients $(y_0, \dots, y_{d-1}) \in \mathbb{Z}_q^d$. We define the ℓ_2 and ℓ_∞ of a vector of ring elements $\mathbf{y} \in \mathcal{R}^N$ as $\|\mathbf{y}\|_2 := \sqrt{\|\mathbf{y}_1\|^2 + \dots + \|\mathbf{y}_N\|^2}$ and $\|\mathbf{y}\|_\infty := \max_{i \in [N]} \{\|\mathbf{y}_i\|_\infty\}$.

We recall a generalization of the Schwartz-Zippel Lemma [Sch80; Zip79] over rings.

Lemma 3.6 (Schwartz-Zippel for rings). *Let $\mathcal{R}$ be a ring and $f \in \mathcal{R}[x_1, \dots, x_n]$ be a non-zero polynomial of total degree D . If $\mathcal{C}$ is a sampling space for $\mathcal{R}$ then $\Pr_{\mathbf{r} \leftarrow \mathcal{C}^n} [f(\mathbf{r}) = 0] \leq \frac{D}{|\mathcal{C}|}$.*

Lemma 3.7 (Chinese Remainder Theorem for rings). *Let $p(x) \in \mathbb{Z}_q[x]$ be a polynomial that decomposes into irreducible factors $p_1(x), \dots, p_t(x) \in \mathbb{Z}_q[x]$ for some $t \in \mathbb{N}$. Then*

$$\mathbb{Z}_q[x]/(p(x)) \cong \mathbb{Z}_q[x]/(p_1(x)) \times \dots \times \mathbb{Z}_q[x]/(p_t(x))$$

where each $\mathbb{Z}_q[x]/(p_i(x))$ is a field. The isomorphism is given by the map $y \mapsto (y \bmod p_1(x), \dots, y \bmod p_t(x))$ for $y \in \mathbb{Z}_q[x]/(p(x))$.

Lemma 3.8 (Corollary 1.2, [LS17]). *Let $d \geq k > 1$ be powers-of-two. Let $q \equiv 2k + 1 \pmod{4k}$ be a prime. Then the polynomial $x^d + 1$ factors over $\mathbb{Z}_q[x]$ as $x^d + 1 = \prod_{i=1}^k (x^{d/k} - r_i) \bmod q$ where each term $(x^{d/k} - r_i)$ is irreducible and r_i are distinct. Additionally, if $0 < \|y\|_\infty < \frac{1}{\sqrt{k}} \cdot q^{1/k}$ or $0 < \|y\|_2 < q^{1/k}$, then y has a multiplicative inverse in $\mathbb{Z}_q[x]/(x^d + 1)$.*

3.3 Lookup Arguments

Lookup arguments allow a prover to convince a verifier that every entry in a witness vector $\mathbf{a} \in \mathcal{R}^M$ is contained in a public table $\mathbf{b} \in \mathcal{R}^N$, which we will denote using the short-hand notation $\mathbf{a} \subseteq \mathbf{b}$.

Definition 3.9. *The lookup relation R_{LU} is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = ((g(I), g([N])), (\mathcal{R}, g, M, N), (\mathbf{a}, \mathbf{b}))$$

where g is a publicly known map that is injective over the domain $[N]$, $|I| = M$, $\mathbf{a} \in \mathcal{R}^M$, $\mathbf{b} \in \mathcal{R}^N$, and $\{(\mathbf{a}_i, g(I_i))\}_{i \in [M]} \subseteq \{(\mathbf{b}_j, g(j))\}_{j \in [N]}$.

Remark 3.10. We remark that the lookup indexed relation is traditionally defined over the indices of the entries in the lookup query and lookup table. In particular, I is a vector such that $I_i = j$ if and only if $\mathbf{a}_i = \mathbf{b}_j$. We generalize this in Definition 3.9 above by instead considering a publicly known injective map g that sends the indices in I to another domain. Then the indexed relation R_{LU} is defined over the new set of “lifted” indices $g(I) = \{g(I_1), \dots, g(I_M)\}$. If g is the identity map then we are in the traditional setting. In later sections, we denote $g(I)$ by an arbitrary list of elements $\mathbf{c} \in \mathcal{R}^M$ that has a one-to-one correspondence with the index list I .

In Definition 3.9, the vector I is part of the public index $\mathbf{i}$. To give lookup protocols where the indices of $\mathbf{a}$ contained in $\mathbf{b}$ are not publicly fixed, I can be included in the witness.

Definition 3.11. *The indexed lookup relation R_{ILU} is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (g([N]), (\mathcal{R}, g, M, N), (\mathbf{a}, \mathbf{b}, g(I)))$$

where g is a publicly known map that is injective over the domain $[N]$, $|I| = M$, $\mathbf{a} \in \mathcal{R}^M$, $\mathbf{b} \in \mathcal{R}^N$, and $\{(\mathbf{a}_i, g(I_i))\}_{i \in [M]} \subseteq \{(\mathbf{b}_j, g(j))\}_{j \in [N]}$.

Remark 3.12. Later, we give IOPs for R_{ILU} in which the index vector I is part of the witness and the verifier has oracle access to it. Since the index vector $g(I)$ is not publicly fixed, an IOP for R_{ILU} should check that the vector $g(I) \in \mathbf{w}$ is well-formed with respect to the publicly known map g (i.e., the prover correctly “lifted” the index vector I to the new domain $g(\cdot)$).

4 Barriers to Directly Instantiating Lookup Arguments over Rings

In this section we show that lookup arguments over fields require careful treatment when being “translated” directly to rings. In particular, we give attacks that show that the lookup relations from Plookup [GW20], LogUp [Hab22], and the Lasso-style protocol from the first version of [CCCFGSV25] are insecure when instantiated directly over rings¹. We give specific examples to clearly illustrate our attacks, but our techniques are nonetheless general and can be extended to other grand-product relations.

At a high-level, our attacks bring awareness to bugs in grand-product statements over rings that have been systematically overlooked. Grand-products are a core building block of lookup arguments. In general, a grand-product shows that two lists are identical up to a permutation by showing that the randomized *product* of the entries in both lists are equal. This can be cast as a PIOP by creating two polynomials whose roots are the entries of each list, respectively, and then evaluating the polynomials at a random point. The intuition for security comes from Schwartz-Zippel and fields being unique factorization domains; if the evaluations are equal then the polynomials must share the same roots which means the lists are related by a permutation.

However, prior work on verifiable mixnets [BLMG25] pointed out that grand-products require careful treatment over rings. We observe that these techniques can be generalized to lookup arguments which rely on grand-products. The idea is that if the “order” of the entries in the lookup query is not fixed within each CRT component that the ring splits into, then a cheating prover can produce a lookup query that is valid when projected onto each CRT component, yet invalid over the full ring. We demonstrate that Plookup and Lasso are susceptible to this CRT-swapping attack when their grand-product arguments are instantiated/stated directly over rings. In addition to the CRT-swapping attack, we identify a second bug that arises when the lookup table contains zero-divisors. In particular, a cheating prover can construct a polynomial that always passes a zero-test even though the lookup query is invalid. We show that this attack can be used on LogUp [Hab22] over rings.

Lasso. Recent work [CCCFGSV25] directly extends Lasso [STW23] from fields to rings. To the best of our knowledge, this is the first work that gives an explicit construction for ring lookups for structured tables. The lookup argument is used for a range proof (i.e., the table consists of small integers) within a verifiable FHE scheme to prove that each component in the CRT decomposition of a vector of ring elements is bounded by the correct CRT modulus.

At a high level, the table structure comes from the observation that each $\mathbb{Z}_q$ -coefficient of a ring element in the coefficient basis can be decomposed into k digits of some powers of smaller base β . Then a ring element can be written as a summation over all monomial coefficients, wherein each coefficient is itself a summation of digits from the base- β decomposition. Using this representation of the ring element, a range-proof for each coefficient in the monomial basis can be re-expressed as several range proofs on the digits from the base- β decomposition of each coefficient in the monomial basis.

We identify a bug in the use of Lemma 4.6 within the proof of Theorem 4.8 from the first version of [CCCFGSV25]. In particular, Lemma 4.6 claims that the grand-product from Spartan [Set20] directly extends from fields to rings. In the proof of Theorem 4.8, this Lemma is invoked and it is claimed that soundness follows from generalized Schwartz-Zippel over rings. However, we show

¹We note that this issue was fixed in a subsequent version of [CCCFGSV25] when the values of the lookup table are restricted to have a special form.

that these claims are not generally true by giving an attack on the grand-product statement when instantiated directly over rings.

Before giving our attack on the first version of [CCCFGSV25], we start by recalling necessary background on the Lasso lookup argument over fields [STW23]. Lasso essentially boils down to several “nested” lookup arguments into the smaller sub-tables (which arise from decomposing the full table). To handle each nested lookup argument into the sub-tables, Lasso uses offline memory-checking from Spartan [Set20], which in turn reduces to a grand-product².

In the first version of Lemma 4.6 from [CCCFGSV25], it is claimed that the Spartan grand-product immediately holds over rings when the size of the lookup query $\mathbf{a}$ is smaller than the ring modulus q . In particular, $\mathbf{a} \subseteq \mathbf{b}$ is a valid lookup if and only if there exist polynomials $\text{read_ts} \in \mathcal{R}[x_1, \dots, x_M]$ and $\text{final_cts} \in \mathcal{R}[x_1, \dots, x_\beta]$ such that

$$\overline{\text{WS}}_t \cdot \overline{\text{WS}}_h = \overline{\text{RS}} \cdot \overline{\text{S}}, \quad (7)$$

where

$$\begin{aligned} \overline{\text{WS}}_t &= \prod_{j \in \{0,1\}^\beta} (\mathbf{b}(j) - x_2) & \overline{\text{WS}}_h &= \prod_{i \in \{0,1\}^M} (\mathbf{a}(i) + x_1(\text{read_ts}(i) + 1) - x_2) \\ \overline{\text{RS}} &= \prod_{i \in \{0,1\}^M} (\mathbf{a}(i) + x_1 \text{read_ts}(i) - x_2) & \overline{\text{S}} &= \prod_{j \in \{0,1\}^\beta} (\mathbf{b}(j) + x_1 \text{final_cts}(j) - x_2) \end{aligned}$$

where β is the size of the sub-table and M is the size of the lookup query. At a high level, read_ts and final_cts are the multilinear extension polynomials of integer-valued vectors from Spartan’s offline-memory checking procedure. These vectors count the number of times each entry in the lookup query $\mathbf{a}$ appears in the table $\mathbf{b}$; they do not contain any information about the *location* at which each lookup query appears in the table. [CCCFGSV25] uses Lemma 4.6 within the proof of Theorem 4.8, where it is claimed that soundness of Equation (7) evaluated at a random point immediately follows from the generalized Schwartz-Zippel lemma.

We present an attack on the above formulation of Lemma 4.6 by showing that the assumption that $M < q$ is not sufficient for grand-products over rings. While this issue has been subsequently fixed for the specific use case, we believe that our attack still emphasizes the important subtleties that arise when building lookups over rings. The idea is that we can construct a lookup query which is *invalid* over the full ring, yet the grand product claim in Equation (7) still holds, thus breaking soundness and violating Lemma 4.6 and Theorem 4.8 of [CCCFGSV25].

For illustrative purposes, let $\mathcal{R}_{15} = \mathcal{R}_3 \times \mathcal{R}_5$ and lookup table $\mathbf{b} = \{1, 2, 3\}$. First, consider a valid lookup $\mathbf{a} = \{1, 2\}$ with $\text{final_cts} = \{1, 1, 0\}$ and $\text{read_ts} = \{0, 0\}$. This clearly satisfies the grand product relation over the full ring, and therefore over each CRT component, since $\overline{\text{WS}}_t \cdot \overline{\text{WS}}_h = \overline{\text{RS}} \cdot \overline{\text{S}}$ where

$$\begin{aligned} \overline{\text{WS}}_t &= (1 - x_2)(2 - x_2)(3 - x_2) & \overline{\text{WS}}_h &= (1 + x_1 - x_2)(2 + x_1 - x_2) \\ \overline{\text{RS}} &= (1 - x_2)(2 - x_2) & \overline{\text{S}} &= (1 + x_1 - x_2)(2 + x_1 - x_2)(3 - x_2). \end{aligned}$$

In the CRT basis, we have $1 \rightarrow (1, 1)$, $2 \rightarrow (2, 2)$, $3 \rightarrow (0, 3)$. For our attack we consider swapping the first CRT components of the lookup query to produce the invalid lookup $\mathbf{a}' := \{7, 11\} \subsetneq \mathbf{b}$ which

²Note that subsequent works HyperPianist [LZLHQZ25] and Tassle [Dor24] optimize Lasso by replacing the nested grand-product with a log-derivatives “grand summation” instead. However, we show that log derivatives is also not directly instantiable over rings which means this optimized version of Lasso is still insecure.

is $\{(1, 2), (2, 1)\}$ in the CRT basis. We keep `final.cts` and `read.ts` the same as before. By swapping CRT components, a malicious prover can break soundness and prove invalid grand-products, since the claim $\overline{WS}_t \cdot \overline{WS}_h = \overline{RS} \cdot \overline{S}$ still holds true over each CRT component, and hence it holds over the ring, as shown below.

$$\begin{array}{ll}
\mathbb{Z}_3 : & \mathbb{Z}_5 : \\
\overline{WS}_t = (1 - x_2)(2 - x_2)(-x_2) & \overline{WS}_t = (1 - x_2)(2 - x_2)(3 - x_2) \\
\overline{WS}_h = (1 + x_1 - x_2)(2 + x_1 - x_2) & \overline{WS}_h = (2 + x_1 - x_2)(1 + x_1 - x_2) \\
\overline{S} = (1 + x_1 - x_2)(2 + x_1 - x_2)(-x_2) & \overline{S} = (1 + x_1 - x_2)(2 + x_1 - x_2)(3 - x_2) \\
\overline{RS} = (1 - x_2)(2 - x_2) & \overline{RS} = (2 - x_2)(1 - x_2)
\end{array}$$

This concludes our attack.

Remark 4.1. We remark that in [CCCFGSV25], the lookup table consists of integer units $\{0, \dots, \beta\}$ where β is smaller than the smallest CRT prime modulus, and therefore the CRT components of the table entries are always equal. This property is not true in general for lookups with arbitrary ring elements.

Plookup. We demonstrate that trivially extending Plookup [GW20] from fields to rings is vulnerable to the same kind of attack as the Lasso grand-product. Once again, this is because the original Plookup relation does not impose any restrictions on the order of the lookup argument. This means that a lookup can be invalid over the full ring, but still valid within each individual CRT component according to a *different* lookup.

As before, we consider $\mathbb{Z}_{15} = \mathbb{Z}_3 \times \mathbb{Z}_5$ with invalid lookup $\mathbf{a}' = \{7, 11\} \rightarrow \{(1, 2), (2, 1)\}$ and table $\mathbf{b} = \{1, 2, 3\}$. In Plookup, the prover must create a vector $\mathbf{w}$ which is purportedly the vector given by $\mathbf{a}$ concatenated with $\mathbf{b}$, sorted by $\mathbf{b}$. Let a malicious prover set $\mathbf{w} = \{1, 1, 2, 2, 3\}$.

Then if we analyze the Plookup polynomials F and G in the CRT basis over $(\mathbb{Z}_3, \mathbb{Z}_5)$, we have

$$\begin{array}{ll}
F(\beta, \gamma) = (\gamma(1 + \beta) + (1, 2) + \beta(1, 2)) & G(\beta, \gamma) = (\gamma(1 + \beta) + (1, 1) + \beta(1, 1)) \\
\cdot (\gamma(1 + \beta) + (2, 1) + \beta(2, 1)) & \cdot (\gamma(1 + \beta) + (1, 1) + \beta(2, 2)) \\
\cdot (\gamma(1 + \beta) + (1, 1) + \beta(2, 2)) & \cdot (\gamma(1 + \beta) + (2, 2) + \beta(2, 2)) \\
\cdot (\gamma(1 + \beta) + (2, 2) + \beta(3, 3)) & \cdot (\gamma(1 + \beta) + (2, 2) + \beta(3, 3))
\end{array}$$

It follows that $F \equiv G$ in each CRT component by inspection, and hence the equality also holds over the ring, which concludes the attack on Plookup.

LogUp. Unlike Plookup and Lasso, the LogUp approach [Hab22] represents a lookup argument algebraically as a “grand-summation” instead of a grand-product.

$$\sum_{i \in [M]} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j}$$

Clearing the denominators produces the following equivalent polynomial relation:

$$P(x) = \left(\prod_{j \in [N]} (x - \mathbf{b}_j) \cdot \sum_{i \in [M]} \prod_{\ell \neq i, \ell \in [M]} (x - \mathbf{a}_\ell) \right) - \left(\prod_{i \in [M]} (x - \mathbf{a}_i) \cdot \sum_{j \in [N]} \mathbf{m}[\mathbf{b}_j] \cdot \prod_{k \neq j, k \in [N]} (x - \mathbf{b}_k) \right) = 0 \quad (8)$$

There exists a vector $\mathbf{m}$ such that the polynomial identity $P(x) = 0$ holds if and only if $\mathbf{a} \subseteq \mathbf{b}$ is a valid lookup.

We show that LogUp is vulnerable to three styles of attacks when instantiated directly over rings. First, Equation (8) does not immediately extend from fields to rings because it is vulnerable to the same CRT-swapping attack as in Plookup and Lasso. In addition to this, we identify another attack on the proof technique from [Hab22] that exploits the presence of zero-divisors in the ring. To demonstrate our attack, we begin by using the technique from [Hab22], in which the lookup relation is re-expressed as an equivalent summation over all field elements

$$\underbrace{\sum_{i \in [M]} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j}}_{\text{Original LogUp relation}} \iff \underbrace{\sum_{v \in \mathbb{F}} (\mathbf{m}_{\mathbf{a}}[v] - \mathbf{m}_{\mathbf{b}}[v]) \prod_{z \in \mathbb{F}, z \neq v} (x - z)}_{\text{Equivalent relation, as a summation over entire field}} = 0 \quad (9)$$

where $\mathbf{m}_{\mathbf{a}}[v]$ counts the number of times the value v appears in $\mathbf{a}$ and $\mathbf{m}_{\mathbf{b}}[v] = \mathbf{m}[v]$ if $v \in \mathbf{b}$ and $\mathbf{m}_{\mathbf{b}}[v] = 0$ otherwise. By Lemmas 4 and 5 in [Hab22], a lookup argument is valid if and only if $\mathbf{m}_{\mathbf{a}}[v] = \mathbf{m}_{\mathbf{b}}[v]$ for all $v \in \mathbb{F}$.

We now show that this claim does not hold over rings with zero divisors by giving a simple example for $\mathbb{Z}_6$. In particular, we break soundness by showing that an invalid lookup claim can still satisfy the right-hand side of Equation (9). Let the table be $\mathbf{b} := \{1, 2, 3, 4\}$ and consider an invalid lookup $\mathbf{a} := \{5\}$. Clearly $\mathbf{a} \subsetneq \mathbf{b}$ yet we will show that the right-hand side of Equation (9) is still satisfied. Notice that when $v = 5$ we have $\mathbf{m}_{\mathbf{b}}[v] - \mathbf{m}_{\mathbf{a}}[v] \neq 0$ since 5 is an invalid lookup value (i.e., $5 \in \mathbf{a}$ but $5 \notin \mathbf{b}$). Over fields, one can argue that if $\mathbf{m}_{\mathbf{b}}[v] - \mathbf{m}_{\mathbf{a}}[v] \neq 0$ then Equation (9) is not satisfied, as expected. However, this argument cannot be made over rings since there are zero-divisors and a product of non-zero terms can go to zero. In particular, if we rewrite Equation (9) over $\mathbb{Z}_6$, then the right-hand side expression, denoted $f(x)$, is identically 0 as a polynomial.

$$f(x) := \sum_{v \in \mathbb{Z}_6} (\mathbf{m}_{\mathbf{a}}[v] - \mathbf{m}_{\mathbf{b}}[v]) \prod_{z \in \mathbb{Z}_6, z \neq v} (x - z) = 0$$

This is because for any fixed $x, v \in \mathbb{Z}_6$, the product $\prod_{z \in \mathbb{Z}_6, z \neq v} (x - z)$ goes to zero since it will always contain at least two zero-divisors whose product is zero. We do note, however, that if one were to instantiate this argument over a suitable subset of the ring in which the pairwise difference between distinct elements was a non-zero-divisor, then the proof would go through as it does over fields. We refer to Appendix E for more details on this restricted setting. This concludes our attack.

While the above example is an attack on the proof technique for LogUp, we finally show a third attack that breaks soundness of the LogUp relation itself (irrespective of the proof technique).

We start with the equivalent polynomial relation

$$P(x) = \underbrace{\left(\prod_{j \in [N]} (x - b_j) \cdot \sum_{i \in [M]} \prod_{\ell \neq i, \ell \in [M]} (x - a_\ell) \right)}_{*} - \underbrace{\left(\prod_{i \in [M]} (x - a_i) \cdot \sum_{j \in [N]} m[b_j] \cdot \prod_{k \neq j, k \in [N]} (x - b_k) \right)}_{**}.$$

There exists a vector $\mathbf{m}$ such that the polynomial identity $P(x) = 0$ should hold if and only if $\mathbf{a} \subseteq \mathbf{b}$ is a valid lookup. However, we use a concrete example over $\mathbb{Z}_{12}$ to demonstrate that this claim is unsound over rings.

Let the table be $b = \{1, 2, 3, 4, 5, 6\}$. Pick any invalid lookup $a \subsetneq b$. For any fixed $x \in \mathbb{Z}_{12}$, define the set $S = \{(x-1), (x-2), (x-3), (x-4), (x-5), (x-6)\} \bmod 12$, which contains exactly six distinct, contiguous elements in $\mathbb{Z}_{12}$. Let S_k denote the k^{th} element in the set S , for $k \in \{1, \dots, |S|\}$. We claim $\prod_{k \neq j, k \in [|S|]} S_k \equiv 0 \bmod 12$ for any fixed $j \in [|S|]$. That is, the product of all but one of the elements in S will always be 0 mod 12. This follows from inspection; S will always contain (a) the element 0 or (b) two elements whose combined factors are $2 \cdot 2 \cdot 3 = 12$. This means that for any $x, j \in \mathbb{Z}_{12}$, $\prod_{j \in [N]} (x - \mathbf{b}_j) = 0 \bmod 12$ and $\prod_{k \neq j, j \in [N]} (x - \mathbf{b}_j) = 0 \bmod 12$. We conclude that both (*) and (**) are 0 mod 12 which means $P(x) = 0$ over $\mathbb{Z}_{12}$ for any invalid lookup, thus breaking soundness. Though this attack is a toy example for $\mathbb{Z}_{12}$, it nonetheless highlights the issues that zero-divisors pose. If using LogUp directly over rings, one must be careful to ensure that the ring is not susceptible to this style of attack.

5 PIOPs for Lookups over Rings

In this section, we present constructions for lookup PIOPs that support arbitrary tables with ring elements. We extend two well-studied lookup arguments, Plookup [GW20] and LogUp [Hab22], both of which were originally designed over finite fields. In Section 5.1 we present Ring-Plookup and in Section 5.2 we present Ring-LogUp.

5.1 Ring-Plookup

In this section, we first recall the original algebraic Plookup relation and show how to extend the relation to rings in Lemma 5.4. We then present Ring-Plookup, a PIOP for our new indexed Plookup relation that generalizes the results from [BCG20] to rings. Ring-Plookup uses several sub-protocols from [BCG20] instantiated over rings and we refer to Appendix B for details.

5.1.1 Extending Plookup relation to rings

We begin by restating the non-indexed Plookup relation, where `shift` and `merge` are given in Definition A.1 and Definition A.2, respectively.

Lemma 5.1 (Claim 3.1, Plookup [GW20]). *Let $\mathbb{F}$ be a prime order field. Let $\mathbf{a} \in \mathbb{F}^M$ and $\mathbf{b} \in \mathbb{F}^N$ be vectors of field elements. Then $((\perp, \perp), (\mathbb{F}, N, N), (\mathbf{a}, \mathbf{b})) \in R_{\text{LU}}$ if and only if there exists $\mathbf{w} \in \mathbb{F}^{N+M}$ such that*

$$\prod_{j=1}^{M+N} (x_1 + \mathbf{w}_j + \text{shift}(\mathbf{w})_j x_2) = \prod_{j=1}^M (x_1 + (1 + x_2) \mathbf{a}_j) \prod_{j=1}^N (x_1 + \mathbf{b}_j + \text{shift}(\mathbf{b})_j x_2). \quad (10)$$

In the case that $\mathbf{a} \subseteq \mathbf{b}$, we may take any $\mathbf{w} \in \text{merge}_{\mathbf{b}}(\mathbf{a})$ to satisfy Equation (10).

Remark 5.2. The `shift()` operation arises from the fact that if the pairwise differences between entries in sorted lists are the same, then the lists contain the same entries. We refer to Appendix C for a standalone proof of this claim extended over rings.

Plookup does not immediately extend to rings for two main reasons. First, the grand product relation in Equation (10) assumes the underlying algebraic structure is a unique factorization domain (UFD). Second, the Plookup protocol has the prover compute a polynomial that aggregates a product of rational terms. Specifically, the prover will use Equation (10) and the verifier's random challenges $\beta, \gamma \in \mathbb{F}$ to define bivariate polynomials $F(\beta, \gamma)$ and $G(\beta, \gamma)$ from Equation (1) and Equation (2). Then to prove Equation (10) (i.e., $\frac{F(\beta, \gamma)}{G(\beta, \gamma)} = 1$), the prover will aggregate partial evaluations of $\frac{F(\beta, \gamma)}{G(\beta, \gamma)}$. However, the denominator of these aggregated values is not guaranteed to be invertible over a ring, which means this protocol is not well-defined for our setting.

To ameliorate the first issue regarding the UFD, we take inspiration from the shuffle argument in [BLMG25], which fixes the grand product relation by adding another variable whose coefficient comes from a low-norm sampling space in the ring. We fix the second issue regarding multiplicative inverses by presenting a new PIOP (Construction 5.5) for *indexed* Plookup-style lookup arguments over rings. Our protocol leverages several sub-protocols of [BCG20] extended over rings (see Appendix B).

We begin by restating the following helpful lemma from [BLMG25].

Lemma 5.3 (Shuffle relation [BLMG25]). *Let $\mathcal{R} = \mathcal{R}_1 \times \cdots \times \mathcal{R}_t$ be a product of integral domains $\mathcal{R}_i$. Let $\mathcal{C} \subseteq \mathcal{R}$ be a sampling set. Let $N \in \mathbb{N}$ and consider an injective map $g : [N] \rightarrow \mathcal{C}$. Let $\mathbf{a}, \mathbf{b} \in \mathcal{R}^N$. If*

$$\prod_{i=1}^N (\mathbf{a}_i + \mathbf{c}_i \cdot x_1 - x_2) = \prod_{i=1}^N (\mathbf{b}_i + g(i) \cdot x_1 - x_2) \quad (11)$$

over $\mathcal{R}[x_1, x_2]$, where $\mathbf{c}_i \in \mathcal{C}$ for all $i \in [N]$, then the entries of $\mathbf{a}$ are a permutation of the entries of $\mathbf{b}$.

Using the technique of Lemma 5.3, we obtain the extension of Lemma 5.1 for arbitrary ring elements.

Lemma 5.4 (Ring-Plookup relation). *Let $\mathcal{R} = \mathcal{R}_1 \times \cdots \times \mathcal{R}_t$ be a product of unique factorization domains $\mathcal{R}_i$. Let $N \in \mathbb{N}$, $\mathcal{C} \subseteq \mathcal{R}$ be a sampling set, and $g : [N] \rightarrow \mathcal{C}$ be an injective map. Let $\mathbf{a} \in \mathcal{R}^M$, $\mathbf{b} \in \mathcal{R}^N$, and $\mathbf{c} \in \mathcal{C}^M$. Then $(g([N]), (\mathcal{R}, g, M, N), (\mathbf{a}, \mathbf{b}, \mathbf{c})) \in R_{\text{ILU}}$ if and only if there exists $\mathbf{w} \in \mathcal{R}^{M+N}$ and $\boldsymbol{\sigma} \in \mathcal{C}^{M+N}$ such that*

$$\begin{aligned} & \prod_{k=1}^{M+N} (\mathbf{w}_k + \text{shift}(\mathbf{w})_k \cdot x_1 + \boldsymbol{\sigma}_k \cdot x_2 - x_3) = \\ & \prod_{i=1}^M (\mathbf{a}_i + \mathbf{a}_i \cdot x_1 + \mathbf{c}_i \cdot x_2 - x_3) \cdot \prod_{j=1}^N (\mathbf{b}_j + \text{shift}(\mathbf{b})_j \cdot x_1 + g(j) \cdot x_2 - x_3) . \end{aligned} \quad (12)$$

over $\mathcal{R}[x_1, x_2, x_3]$. In the case that $(g([N]), (\mathcal{R}, M, N), (\mathbf{a}, \mathbf{b}, \mathbf{c})) \in R_{\text{ILU}}$, we may take any $\mathbf{w} \in \text{merge}_{\mathbf{b}}(\mathbf{a})$ and $\mathbf{c}$ to be the canonical index vector such that $\mathbf{c}_i = g(j)$ if and only if $\mathbf{a}_i = \mathbf{b}_j$ to satisfy Equation (12).

Proof. First, we prove the result when $t = 1$, then we prove that this implies the general case. When $t = 1$, $\mathcal{R}$ is a unique factorization domain, which implies that $\mathcal{R}[x_1, x_2, x_3]$ is also a unique factorization domain.

Let $p(x_1, x_2, x_3)$ be the polynomial on the left hand side of Equation (12). Since p has $(-1)^{M+N}$ as the coefficient of x_3^{M+N} , it is non-zero. Moreover, we will show by contradiction that p cannot be a unit of $\mathcal{R}[x_1, x_2, x_3]$. Assume that there was a polynomial q such that $pq = 1$ in $\mathcal{R}[x_1, x_2, x_3]$ and let $ax_1^b x_2^c x_3^d$ be the term of q with the maximum degree of x_3 . Multiplied by $(-x_3)^{M+N}$ would lead to pq having a term $(-1)^{M+N} ax_1^b x_2^c x_3^{d+M+N}$. This term does not cancel out since d and $M+N$ are the maximum degree of x_3 of q and p . However, since $pq = 1$, this term must have coefficient zero. Since $\mathcal{R}$ is a unique factorization domain, this implies that $a = 0$, which is a contradiction. Similarly, the right side of Equation (12) is a non-zero, non-unit polynomial in $\mathcal{R}[x_1, x_2, x_3]$.

Both the left and right sides of Equation (12) are non-zero, non-unit elements. Each element in the product is a non-zero linear polynomial, and hence irreducible. Hence, as $\mathcal{R}[x_1, x_2, x_3]$ is a unique factorization domain, there is a one-to-one correspondence between the factors on both the left and right hand sides of Equation (12), up to multiplication by units. Since the coefficient of x_3 in each linear factor is -1 , the correspondence is actually exact.

Comparing coefficients of the linear factors implies that

$$\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\}_{k \in [M+N]} = \{\mathbf{a}_i, \mathbf{a}_i\}_{i \in [M]} \cup \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}_{j \in [N]} ,$$

and hence Lemma C.1 implies that $\mathbf{a} \subseteq \mathbf{b}$. Moreover, since each $\mathbf{a}_i$ is part of the same factor as $\mathbf{c}_i$ and $\mathbf{b}_j$ is part of the same factor as $g(j)$, we have that $\{(\mathbf{a}_i, \mathbf{c}_i)\}_{i \in [M]} \subseteq \{(\mathbf{b}_j, g(j))\}_{j \in [N]}$.

The result for the case $t = 1$ implies the lookup relation over each subring separately. In the general case of $t > 1$, note that no two elements $g(i)$ and $g(j)$ with $i \neq j$ can be equal after being projected into a subring $\mathcal{R}_k$. This is because $g(i) \neq g(j)$ in $\mathcal{R}$ by injectivity of g , and if their components over $\mathcal{R}_k$ were equal, the difference $g(i) - g(j)$ would not be invertible, which contradicts the fact that $\mathcal{C}$ is a sampling set. Hence, the indices for the lookup relation in each subring are the same, and we have $\{(\mathbf{a}_i, \mathbf{c}_i)\}_{i \in [M]} \subseteq \{(\mathbf{b}_j, g(j))\}_{j \in [N]}$ over $\mathcal{R}$. $\square$

5.1.2 Ring-Plookup PIOP

We present a PIOP using the Ring-Plookup relation in Lemma 5.4.

Construction 5.5 (Ring-Plookup PIOP). We construct a holographic PIOP for the indexed relation R_{ILLU} . Consider the ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ such that $\mathcal{R} = \mathcal{R}_1 \times \dots \times \mathcal{R}_t$ is a product of unique factorization domains. Let $\mathcal{C} \subset \mathcal{R}$ be an exponentially-sized challenge space consisting of all ring elements with binary coefficients. Let $g : [N] \rightarrow \mathcal{C}$ be a publicly known map that is injective over $[N]$.

Given as input an index $\mathbf{i} = g([N])$, the indexer $\mathbf{I}$ outputs the oracle message $\Pi_0 := g([N])$. The prover $\mathbf{P}$ takes as input the index $\mathbf{i}$, instance $\mathbf{x} = (\mathcal{R}, M, N)$, and witness $\mathbf{w} = (\mathbf{a}, \mathbf{b}, \mathbf{c})$; the verifier $\mathbf{V}$ has query access to the index $\mathbf{i}$ and the witness $\mathbf{w}$ and takes as input the instance $\mathbf{x}$.

- The prover $\mathbf{P}$ constructs the vectors $\mathbf{w}, \boldsymbol{\sigma} \in \mathcal{R}^{N+M}$ that contain the same entries as $(\mathbf{a}, \mathbf{b})$ and $(\mathbf{c}, g([N]))$ with entries $\mathbf{b}_i, \boldsymbol{\sigma}_i$ ordered by the index $i \in [N]$ that contributes to those entries. In more detail, $\mathbf{P}$ does the following:
 - Scan $(\mathbf{c}, g([N])) \in \mathcal{R}^{N+M}$ to deduce the frequency vector $f \in \mathbb{N}^N$ where f_i is the number of occurrences of $\mathbf{c}_i$ in $(\mathbf{c}, g([N]))$.
 - Allocate the zero vectors $\mathbf{w}, \boldsymbol{\sigma} := 0^{N+M}$ and then perform linear scans on $\mathbf{b}, g([N])$, inserting f_i copies of $\mathbf{b}_j$ such that $\mathbf{c}_i = g(j)$ into $\mathbf{w}$ and $g(j)$ into $\boldsymbol{\sigma}$ for each $i \in [N]$.

Then $\mathbf{P}$ sets $\mathbf{b}_\circ := \text{shift}(\mathbf{b})$ and $\mathbf{w}_\circ := \text{shift}(\mathbf{w})$, and sends the oracle message $(\mathbf{b}_\circ, \mathbf{w}, \mathbf{w}_\circ) \in \mathcal{R}^N \times \mathcal{R}^{M+N} \times \mathcal{R}^{M+N}$.

- The verifier $\mathbf{V}$ sends random challenges $\alpha, \gamma, \beta \in \mathcal{C}$ to $\mathbf{P}$.
- The prover $\mathbf{P}$ computes the following vectors:

$$\mathbf{a}^* := \mathbf{a} + \alpha \cdot \mathbf{a} + \gamma \cdot \mathbf{c} - \beta \cdot \mathbf{1}^M, \quad (13)$$

$$\mathbf{b}^* := \mathbf{b} + \alpha \cdot \mathbf{b}_\circ + \gamma \cdot g([N]) - \beta \cdot \mathbf{1}^N, \quad (14)$$

$$\mathbf{w}^* := \mathbf{w} + \alpha \cdot \mathbf{w}_\circ + \gamma \cdot \boldsymbol{\sigma} - \beta \cdot \mathbf{1}^{M+N}. \quad (15)$$

Next, $\mathbf{P}$ computes the entry products $\chi_{\mathbf{a}^*} := \text{prod}(\mathbf{a}^*)$, $\chi_{\mathbf{b}^*} := \text{prod}(\mathbf{b}^*)$, and $\chi_{\mathbf{w}^*} := \text{prod}(\mathbf{w}^*)$. Then $\mathbf{P}$ sends the oracle message $(\mathbf{a}^*, \mathbf{b}^*, \mathbf{w}^*) \in \mathcal{R}^{2(N+M)}$ and non-oracle message $(\chi_{\mathbf{a}^*}, \chi_{\mathbf{b}^*}, \chi_{\mathbf{w}^*}) \in \mathcal{R}^3$ to $\mathbf{V}$.

- The prover $\mathbf{P}$ and verifier $\mathbf{V}$ engage in several sub-protocols.
 - An entry-product protocol (Construction B.12) with $\mathbf{x} = (\mathcal{R}, M, \chi_{\mathbf{a}^*})$ and $\mathbf{w} = \mathbf{a}^*$ to show that $\chi_{\mathbf{a}^*} = \text{prod}(\mathbf{a}^*)$.

- An entry-product protocol with $\mathbf{x} = (\mathcal{R}, N, \chi_{\mathbf{b}^*})$ and $\mathbf{w} = \mathbf{b}^*$ to show that $\chi_{\mathbf{b}^*} = \text{prod}(\mathbf{b}^*)$.
- An entry-product protocol with $\mathbf{x} = (\mathcal{R}, N + M, \chi_{\mathbf{w}^*})$ and $\mathbf{w} = \mathbf{w}^*$ to show that $\chi_{\mathbf{w}^*} = \text{prod}(\mathbf{w}^*)$.
- The verifier $\mathbf{V}$ checks that $\chi_{\mathbf{w}^*} = \chi_{\mathbf{a}^*} \cdot \chi_{\mathbf{b}^*}$. Next, $\mathbf{V}$ samples random evaluation points $\eta_1 \in \mathcal{R}^{\log M}$, $\eta_2 \in \mathcal{R}^{\log N}$, $\eta_3 \in \mathcal{R}^{\log(M+N)}$ and performs the checks below, by using polynomial evaluation queries to obtain each term on the left-hand side of each equation and by directly computing the right-hand side using factorizations such as $\widehat{\mathbf{1}}^N(\eta) = \prod_{i=1}^{\log(M+N)} (1 + \eta[i])$.

$$\widehat{\mathbf{a}}^*(\eta_1) = \widehat{\mathbf{a}}(\eta_1) + \alpha \cdot \widehat{\mathbf{a}}(\eta_1) + \gamma \cdot \widehat{\mathbf{c}}(\eta_1) - \beta \cdot \widehat{\mathbf{1}}^M(\eta_1) \quad \text{which checks Eq. (13),}$$

$$\widehat{\mathbf{b}}^*(\eta_2) = \widehat{\mathbf{b}}(\eta_2) + \alpha \cdot \widehat{\mathbf{b}}_{\circ}(\eta_2) + \gamma \cdot \widehat{g}([N])(\eta_2) - \beta \cdot \widehat{\mathbf{1}}^N(\eta_2) \quad \text{which checks Eq. (14),}$$

$$\widehat{\mathbf{w}}^*(\eta_3) = \widehat{\mathbf{w}}(\eta_3) + \alpha \cdot \widehat{\mathbf{w}}_{\circ}(\eta_3) + \gamma \cdot \widehat{\boldsymbol{\sigma}}(\eta_3) - \beta \cdot \widehat{\mathbf{1}}^{M+N}(\eta_3) \quad \text{which checks Eq. (15).}$$

- $\mathbf{V}$ checks that $\mathbf{b}_{\circ} = \text{shift}(\mathbf{b})$ and $\mathbf{w}_{\circ} = \text{shift}(\mathbf{w})$, using Construction B.9.
- Finally, $\mathbf{V}$ checks that $\mathbf{c}$ is well-formed with respect to $\mathcal{C}$ by running a binary-check sub-protocol (Construction B.19).

We refer to Appendix D for a proof of the following lemma.

Lemma 5.6. *For every ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ with suitable challenge space $\mathcal{C}$ consisting of all ring elements with binary coefficients, and positive integers $M, N < \text{char}(\mathcal{R})$, Construction 5.5 is a holographic PIOP for the indexed relation R_{ILU} that supports instances over $\mathcal{R}$ with the following efficiency parameters*

- *Soundness error:* $O(\frac{N+M}{|\mathcal{C}|} + \frac{d+\log M}{q})$
- *Round complexity:* $O(\log(N + M))$
- *Proof length:* $O(N + \text{poly}(d)M)\mathcal{R}$
- *Prover cost:* $O(N + \text{poly}(d)M)\mathcal{R}$
- *Verifier cost:* $O(\log(N + M) + d \log M)\mathcal{R}$
- *Query complexity:* $O(d)$

where $|\mathcal{C}| = 2^d$ and q is the size of the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

5.2 Ring-LogUp

The original LogUp lookup protocol is written for multivariate polynomials, supports non-indexed lookups, and can be extended to support a batch of multiple lookup queries (often called a multi-column lookup) into the same table ³. Subsequent works (1) improve efficiency [PH23; Sou25; TNLKN25], (2) extend to indexed-lookup relations [LZLHQZ25; GPS25], and (3) achieve prover

³For simplicity, we give our protocols in the single-instance setting where the batch size is one, though our results can be extended using similar techniques from [Hab22].

time that only depends on the lookup query size via preprocessing the table in quasi-linear time [GK22; EFG22; CFFLL24; ZSG24; CFG25].

In this section, we present Ring-LogUp to extend the log-derivatives style lookup argument to arbitrary ring elements. We first recall the original log-derivatives relation and then extend it to rings by giving a new lookup relation (Lemma 5.8). We give a direct proof that is different than the proof technique from prior work. We then present Ring-LogUp (Construction 5.11), a PIOP for the relation in Lemma 5.8. The Ring-LogUp PIOP supports lookups for arbitrary ring elements with query and table length at most the size of the challenge space (and $\text{char}(\mathcal{R})$ to prevent wraparounds). We leave it as an open task to extend the log-derivatives argument to lookup tables and queries with length larger than $|\mathcal{C}|$.

5.2.1 Extending LogUp relation to rings

We begin by recalling the original log-derivatives lookup relation from Lemma 5 of [Hab22].

Lemma 5.7 (Lemma 5, LogUp [Hab22]). *Let $\mathbb{F}$ be a field with characteristic $p > N$. Let $\mathbf{a} \in \mathbb{F}^N$ and $\mathbf{b} \in \mathbb{F}^N$ be vectors of field elements. Then we have the following: $((\perp, \perp), (\mathbb{F}, N, N), (\mathbf{a}, \mathbf{b})) \in R_{\text{LU}}$ if and only if there exists a vector $\mathbf{m} \in \mathbb{F}^N$ such that*

$$\sum_{i \in N} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in N} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j}$$

in the field of rational functions $\mathbb{F}(x)$.

Recall that in Section 2, it was shown that this claim is unsound over rings due to the presence of zero-divisors. However, if one considers instead the restricted setting with tables $\mathbf{b}$ and queries $\mathbf{a}$ whose entries belong to a sampling space $\mathcal{C} \subseteq \mathcal{R}$, then the proof technique for Lemma 5.7 (see also the proof of Lemma 4.1 in [Dor24]) can be directly extended. We refer to Appendix E for more details.

To extend Lemma 5.7 to rings, we leverage a technique from the shuffle argument in [BLMG25] that involves “tagging” each ring element with a short norm element from the challenge space. By “tagging”, we mean that instead of the rational term $\frac{1}{x - \mathbf{a}_i}$, we use $\frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 - x_2}$ where $\mathbf{c}_i \in \mathcal{C}$ is a unique tag to be associated with entry $\mathbf{a}_i$. To give brief intuition behind tagging, it is helpful to look at just the denominator and consider the following

$$\mathbf{a}_i + \mathbf{c}_i x_1 - x_2 = \mathbf{b}_j + \mathbf{c}_j x_1 - x_2 \implies (\mathbf{a}_i - \mathbf{b}_j) + (\mathbf{c}_i - \mathbf{c}_j) x_1 = 0$$

At a high level, since $\mathbf{c}_i, \mathbf{c}_j \in \mathcal{C}$, we know $\mathbf{c}_i = \mathbf{c}_j$. Then by a careful argument using Chinese Remainder Theorem for rings, one can also show $\mathbf{a}_i = \mathbf{b}_j$, which is precisely the goal of a lookup argument. We refer to the proof of Lemma 5.8 for details.

We show that this modification yields a new *indexed* lookup relation that supports arbitrary ring elements. However, a slight technicality still remains: the “tagged” bivariate polynomial in the denominator of the rational terms may not be well-defined in the bivariate ring $\mathcal{R}[x_1, x_2]$, since $\mathcal{R}$ is not an integral domain. Note that this was not an issue in prior work over fields [Hab22] because each rational term was well-defined in a field of fractions. To fix this, we view each rational term in the total ring of fractions $Q(\mathcal{R}(x_1, x_2))$ and then show that each denominator is a non-zero-divisor in $\mathcal{R}(x_1, x_2)$.

To prove Lemma 5.8, we first observe that the proof technique from prior works [Hab22; Dor24] (rewriting as a summation over all ring elements, shown in Equation (5)) does not readily extend to our new bivariate rational expression. The reason is because there are not enough unique “tags” to form a bijective map with all ring elements. Put differently, since $\mathcal{C} \subset \mathcal{R}$ there are too few unique choices for $\mathbf{c}_i$ to embed as a coefficient of x_1 in the denominator. We instead prove Lemma 5.8 directly using induction and avoid the technical issue that arises from the proof technique in prior works.

Lemma 5.8 (Ring-LogUp relation). *Let $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ be a quotient ring for power-of-two d and prime q such that $\mathcal{R} \cong \mathcal{R}_1 \times \mathcal{R}_2$ is a direct product of integral domains. Let $\mathcal{C} \subset \mathcal{R}$ be a sampling space. Let $\mathbf{a} \in \mathcal{R}^M$, $\mathbf{b} \in \mathcal{R}^N$, and $\mathbf{c} \in \mathcal{C}^M$ such that $\max\{M, N\} < |\mathcal{C}|$. Let $g : [|\mathcal{C}|] \rightarrow \mathcal{C}$ be a publicly known bijection. Then $(g([N]), (\mathcal{R}, M, N), (\mathbf{a}, \mathbf{b}, \mathbf{c})) \in R_{\text{ILU}}$ if and only if there exists a vector $\mathbf{m} \in \mathbb{Z}_q^N$ such that*

$$\sum_{i \in [M]} \frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 - x_2} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)x_1 - x_2} \quad (16)$$

over the total ring of fractions $Q(\mathcal{R}(x_1, x_2))$.

Remark 5.9. The vector $\mathbf{c}$ implicitly defines an index map $I : [M] \rightarrow [N]$ such that $\mathbf{c}_i = g(I_i)$ where $I_i = j$ means that $\mathbf{a}_i = \mathbf{b}_j$. Recall from Definition 3.9 that g is a bijection that maps an index to another domain. If g is the identity map, then the lookup argument is defined over the original indices in $[M]$ and $[N]$. In our formulation of Lemma 5.8 (which uses the log-derivatives expression to instantiate the indexed lookup relation R_{ILU}), we require g to be a bijection that sends an integer in $[|\mathcal{C}|]$ to an element in $\mathcal{C}$. For example, when $\mathcal{C}$ is the set of ring elements with binary coefficients, then $g(i)$ is the binary decomposition of i . This choice for g is intentional, as the proof for Lemma 5.8 requires the pairwise differences of values $g(i), g(j)$ for $i \neq j$ to be invertible, a property that is guaranteed over $\mathcal{C}$.

Proof. We start by showing that each fractional term in the summation is well-defined in the total ring of fractions $Q(\mathcal{R}(x_1, x_2))$ (i.e., the denominator is a non-zero-divisor in $\mathcal{R}(x_1, x_2)$). We know that $\mathbf{a}_i + \mathbf{c}_i x_1 - x_2 \neq 0$ is a non-zero-divisor in $\mathcal{R}(x_1, x_2)$ because the coefficient of the $-x_2$ term is -1 , which cannot be a zero-divisor in $\mathcal{R}$. We also have that the multiplicative closure of the set of polynomials of the form $\mathbf{a}_i + \mathbf{c}_i x_1 - x_2$ does not contain zero divisors, since the highest degree monomial in x_2 will always have coefficient 1 or -1 . The same holds for polynomials of the form $\mathbf{b}_i + g(i)x_1 - x_2$.

($\implies$) We show that if $\mathbf{a}$ is a valid lookup query into table $\mathbf{b}$, then we can define a multiplicity vector $\mathbf{m}$ and tag vector $\mathbf{c}$ such that Equation (16) holds. In particular, $\mathbf{m}$ is a key-value map that can be indexed into using $\mathbf{m}[\cdot]$ notation. More formally, suppose that $(N, (\mathcal{R}, M, N), (\mathbf{a}, \mathbf{b}, \mathbf{c})) \in R_{\text{ILU}}$. By definition of R_{ILU} , we have for $i \in [M]$, $\mathbf{c}_i = g(j) \in [N]$ if and only if $\mathbf{a}_i = \mathbf{b}_j$. That is, $\mathbf{c}$ behaves exactly as the index map.

Define $\mathbf{m} \in \mathbb{Z}_q^N$ to be a vector of multiplicities that maps the element $\mathbf{b}_j$ to the number of times it appears in $\mathbf{a}$. That is, if $\mathbf{a}$ contains v instances of the value $\mathbf{b}_j$, then we set $\mathbf{m}[\mathbf{b}_j] = v$.

For every $i \in [M]$ there exists some $j \in [N]$ such that $\mathbf{a}_i = \mathbf{b}_j$. This means $\mathbf{c}_i = g(j)$. For $j \in [N]$, let $\mathbf{m}[\mathbf{b}_j]$ denote the number of times $\mathbf{b}_j$ appears in $\mathbf{a}$. Hence, by definition of $\mathbf{m}$ and $\mathbf{c}$,

the term $\frac{1}{\mathbf{b}_j + g(j)x_1 + x_2}$ appears exactly $\mathbf{m}[\mathbf{b}_j]$ times on the LHS of Equation (16) which implies that

$$\sum_{i \in [M]} \frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 - x_2} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)x_1 - x_2}.$$

($\Leftarrow$) We show that if the expression in Equation (16) holds, then $\mathbf{a}$ is a valid indexed lookup query into the table $\mathbf{b}$ with index map $\mathbf{c}$. We proceed with a proof by induction on the size of the lookup query M . Let $\text{LogDeriv}_{M,N}(\mathbf{a}, \mathbf{b}, \mathbf{m}, \mathbf{c})$ be short-hand notation for the claim

$$\sum_{i \in [M]} \frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 - x_2} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)x_1 - x_2} \quad (17)$$

Base case ($M = 1$). We consider a lookup query $\mathbf{a}_1 \in \mathcal{R}$ of size $M = 1$, which produces the claim

$$\text{LogDeriv}_{1,N}(\mathbf{a}, \mathbf{b}, \mathbf{m}, \mathbf{c}) \rightarrow \frac{1}{\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)x_1 - x_2} \quad (18)$$

We wish to prove that if this expression holds, then there must exist some index $k \in [N]$ such that

1. $\mathbf{c}_1 = g(k)$ (equivalently, we can implicitly define an index map I such that $g(I_1) = g(k) \implies k = I_1$ since g is a bijection);
2. $\mathbf{a}_1 = \mathbf{b}_k$;
3. $\mathbf{m}[\mathbf{b}_k] = 1$;
4. $\mathbf{m}[\mathbf{b}_j] = 0$ for all $j \neq k$.

To do so, we first introduce the following helpful bivariate polynomials.

$$\begin{aligned} P(x_1, x_2) &:= (\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2) \prod_{j \in [N]} (\mathbf{b}_j + g(j)x_1 - x_2) \\ P_{\mathbf{a}_1}(x_1, x_2) &:= \frac{P(x_1, x_2)}{\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2} = \prod_{j \in [N]} (\mathbf{b}_j + g(j)x_1 - x_2) \\ P_{\mathbf{b}_j}(x_1, x_2) &:= \frac{P(x_1, x_2)}{\mathbf{b}_j + g(j)x_1 - x_2} \end{aligned}$$

Multiplying Equation (18) by $P(x_1, x_2)$ gives us

$$\begin{aligned} \frac{P(x_1, x_2)}{\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2} &= \frac{P(x_1, x_2) \left(\sum_{j \in [N]} \mathbf{m}[\mathbf{b}_j] \prod_{l \in [N], l \neq j} (\mathbf{b}_l + g(l)x_1 - x_2) \right)}{\prod_{j \in [N]} (\mathbf{b}_j + g(j)x_1 - x_2)} \\ &\Rightarrow P_{\mathbf{a}_1}(x_1, x_2) = (\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2) \left(\sum_{j \in [N]} \mathbf{m}[\mathbf{b}_j] \prod_{l \in [N], l \neq j} (\mathbf{b}_l + g(l)x_1 - x_2) \right) \\ &\Rightarrow P_{\mathbf{a}_1}(x_1, x_2) = \sum_{j \in [N]} \mathbf{m}[\mathbf{b}_j] P_{\mathbf{b}_j}(x_1, x_2). \end{aligned} \quad (19)$$

Let $x_2 = \mathbf{b}_k + g(k)x_1$ for a fixed $k \in [N]$. The LHS of Equation (19) is zero since

$$P_{\mathbf{a}_1}(x_1, \mathbf{b}_k + g(k)x_1) = ((\mathbf{b}_k - \mathbf{b}_k) + (g(k) - g(k))x_1) \prod_{j \neq k} ((\mathbf{b}_j - \mathbf{b}_k) + (g(j) - g(k))x_1).$$

Similarly, on the RHS we have $P_{\mathbf{b}_j}(x_1, \mathbf{b}_k + g(k)x_1) = 0$ for all $j \neq k$ since the term $((\mathbf{b}_k - \mathbf{b}_k) + (g(k) - g(k))x_1) = 0$ appears in the product. Hence, for any $k \in [N]$

$$\begin{aligned} 0 &= \mathbf{m}[\mathbf{b}_k] P_{\mathbf{b}_k}(x_1, \mathbf{b}_k + g(k)x_1) \\ &= \mathbf{m}[\mathbf{b}_k] \left((\mathbf{a}_1 - \mathbf{b}_k) + (\mathbf{c}_1 - g(k))x_1 \right) \cdot \left(\prod_{j \in [N], j \neq k} ((\mathbf{b}_j - \mathbf{b}_k) + (g(j) - g(k))x_1) \right). \end{aligned} \quad (20)$$

We consider two cases.

1. **Case 1: $\mathbf{m}[\mathbf{b}_k] = 0$.** In this case, the entry $\mathbf{b}_k$ is not being looked up.
2. **Case 2: $\mathbf{m}[\mathbf{b}_k] \neq 0$.** In this case, the entry $\mathbf{b}_k$ is being looked up and we argue that $\mathbf{b}_k = \mathbf{a}_1$ and $\mathbf{m}[\mathbf{b}_k] = 1$.

From the claim $\text{LogDeriv}(\mathbf{m}, 1, N)$, we know that there is at least one k such that $\mathbf{m}[\mathbf{b}_k] \neq 0$, i.e., it leads to Case 2. Since if not, then Equation (18) implies that $\frac{1}{\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2} = 0$ which is a contradiction.

To show that $\mathbf{c}_1 = g(k)$ and $\mathbf{a}_1 = \mathbf{b}_k$ we recall that $\mathcal{R} \cong \mathcal{R}_1 \times \mathcal{R}_2$, where $\mathcal{R}_1$ and $\mathcal{R}_2$ are integral domains. We project the RHS of Equation (20) to $\mathcal{R}_1[x_1]$ and $\mathcal{R}_2[x_1]$. Since $\mathbf{m}[\mathbf{b}_k] \in \mathbb{Z}_q^\times$, it holds that $\mathbf{m}[\mathbf{b}_k]$ is still a unit in $\mathcal{R}_1[x_1]$ and $\mathcal{R}_2[x_1]$, and therefore it follows from Equation (20) that over the integral domains $\mathcal{R}_1[x_1]$ and $\mathcal{R}_2[x_1]$

$$\underbrace{(\mathbf{a}_1 - \mathbf{b}_k) + (\mathbf{c}_1 - g(k))x_1}_{(*)} = 0 \text{ or } \underbrace{\prod_{j \in [N], j \neq k} ((\mathbf{b}_j - \mathbf{b}_k) + (g(j) - g(k))x_1)}_{(**)} = 0$$

Since for all $j \neq k$, $g(j) - g(k)$ is a unit in $\mathcal{R}$, it holds that $\prod_{j \in [N], j \neq k} (g(j) - g(k)) \neq 0$ over $\mathcal{R}_1$ and $\mathcal{R}_2$. Thus $(**) \neq 0$ and we must have $(*) = 0$ in $\mathcal{R}_1[x_1]$ and $\mathcal{R}_2[x_1]$, which means that $\mathbf{a}_1 = \mathbf{b}_k$ over $\mathcal{R}_1$ and $\mathcal{R}_2$. By uniqueness of a CRT solution, this directly implies that $\mathbf{c}_1 = g(k)$ and $\mathbf{a}_1 = \mathbf{b}_k$ over $\mathcal{R}$. Using $\mathbf{c}$, we can implicitly define an index map I such that $g(I_1) = \mathbf{c}_1 = g(k)$, and by injectivity of g it holds that $I_1 = k$.

All that remains to be shown is that $\mathbf{m}[\mathbf{b}_k] = 1$ and $\mathbf{m}[\mathbf{b}_j] = 0$ for $j \in [N], j \neq k$. From above, we know that for each $\mathbf{b}_k$ such that $\mathbf{m}[\mathbf{b}_k] \neq 0$ it holds that $\mathbf{b}_k = \mathbf{a}_1$ and $\mathbf{c}_1 = g(k)$. Since each $\mathbf{b}_k \in \mathbf{b}$ is unique, the RHS of Equation (18) collapses to the single term $\frac{\mathbf{m}[\mathbf{b}_k]}{\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2}$. Since the LHS of Equation (18) is $\frac{1}{\mathbf{a}_1 + \mathbf{c}_1 x_1 - x_2}$, we must have $\mathbf{m}[\mathbf{b}_k] = 1$.

This concludes the base case.

Inductive assumption. We assume that $\text{LogDeriv}_{M-1, N}(\mathbf{a}, \mathbf{b}, \mathbf{m}, \mathbf{c})$ is true for some $M \geq 2$.

Inductive step. Consider a lookup query of size M , producing the claim

$$\text{LogDeriv}_{M, N}(\mathbf{a}, \mathbf{b}, \mathbf{m}, \mathbf{c}) \rightarrow \sum_{i \in [M]} \frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 - x_2} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)x_1 - x_2}. \quad (21)$$

A size M lookup query vector can be separated into a size one vector containing the entry $\mathbf{a}_{i^*}$ for $i^* \in [M]$ and a size $M - 1$ vector $\mathbf{a}' := \mathbf{a} \setminus \mathbf{a}_{i^*}$. We show that the claim $\text{LogDeriv}_{M,N}(\mathbf{a}, \mathbf{b}, \mathbf{m}, \mathbf{c})$ can be reduced into the size- $(M - 1)$ claim $\text{LogDeriv}_{M-1,N}(\mathbf{a}', \mathbf{b}, \mathbf{m}', \mathbf{c}')$ where $\mathbf{m}'[\mathbf{a}_{i^*}] = \mathbf{m}[\mathbf{a}_{i^*}] - 1$ and $\mathbf{m}'[\mathbf{b}_j] = \mathbf{m}[\mathbf{b}_j]$ for all j such that $g(j) \neq \mathbf{c}_{i^*}$, and $\mathbf{c}'_i = \mathbf{c}_i$ for all $i < i^*$ and $\mathbf{c}'_i = \mathbf{c}_{i+1}$ for all $i > i^*$ (i.e., $\mathbf{c}'$ is a vector which removes the i^* -th element).

It suffices to show that there exists some index $i^* \in [M]$ and $k \in [N]$ such that $g(k) = \mathbf{c}_{i^*}$, $\mathbf{a}_{i^*} = \mathbf{b}_k$, and $\mathbf{m}[\mathbf{b}_k] \neq 0$. Then, we can subtract $\frac{1}{\mathbf{a}_{i^*} + \mathbf{c}_{i^*}x_1 - x_2}$ from the LHS and $\frac{1}{\mathbf{b}_k + g(k)x_1 - x_2}$ from the RHS of Equation (21), and reduce to the claim $\text{LogDeriv}_{M-1,N}(\mathbf{a}', \mathbf{b}, \mathbf{m}', \mathbf{c}')$ which holds from the inductive hypothesis.

The proof technique is nearly identical to that of the base case. We use the bivariate polynomial $P(x_1, x_2) := \prod_{i \in [M]} (\mathbf{a}_i + \mathbf{c}_i x_1 - x_2) \prod_{j \in [N]} (\mathbf{b}_j + g(j)x_1 + x_2)$. Multiplying $P(x_1, x_2)$ into Equation (21) and setting $x_2 = b_k + g(k)x_1$ gives us

$$0 = \mathbf{m}[\mathbf{b}_k] \cdot \underbrace{\prod_{i \in [M]} (\mathbf{a}_i - \mathbf{b}_k) + (\mathbf{c}_i - g(k))x_1}_{(\dagger)} \cdot \underbrace{\prod_{j \in [N], j \neq k} ((\mathbf{b}_j - \mathbf{b}_k) + (g(j) - g(k))x_1)}_{(\dagger\dagger)}. \quad (22)$$

As before, we have two cases, $\mathbf{m}[\mathbf{b}_k] = 0$ or $\mathbf{m}[\mathbf{b}_k] \neq 0$. If $\mathbf{m}[\mathbf{b}_k] = 0$, then entry $\mathbf{b}_k$ is not being looked up. We argue that there must exist some $k \in [N]$ such that $\mathbf{m}[\mathbf{b}_k] \neq 0$ since if not, then Equation (21) implies $\sum_{i \in [M]} \frac{1}{\mathbf{a}_i + \mathbf{c}_i x_1 + x_2} = 0$ which is a contradiction. Unlike the base case, we allow $\mathbf{m}[\mathbf{b}_k] \geq 1$ because there may be repeated values in the M -size lookup.

Analogous to the base case, we project Equation (22) onto the CRT basis. Since for all $j \neq k$, $g(j) - g(k)$ is a unit in $\mathcal{R}$, it holds that $\prod_{j \in [N], j \neq k} (g(j) - g(k)) \neq 0$ over $\mathcal{R}_1$ and $\mathcal{R}_2$. which implies that $(\dagger) = 0$ over $\mathcal{R}_1[x_1]$ and $\mathcal{R}_2[x_1]$. Since $\mathbf{c}_i, g(k) \in \mathcal{C}$, we know $\mathbf{c}_i - g(k)$ is a unit for $\mathbf{c}_i \neq g(k)$. Hence there must exist some index $i^* \in [M]$ such that $(\mathbf{a}_{i^*} - \mathbf{b}_k) + (\mathbf{c}_{i^*} - g(k))x_1 = 0$ which means that $\mathbf{a}_{i^*} = \mathbf{b}_k$ and $\mathbf{c}_{i^*} = g(k)$ over $\mathcal{R}_1$ and $\mathcal{R}_2$. By the uniqueness of CRT decomposition, this directly implies that $\mathbf{a}_{i^*} = \mathbf{b}_k$ in $\mathcal{R}$. Thus, we can remove $\frac{1}{\mathbf{a}_{i^*} + \mathbf{c}_{i^*}x_1 - x_2}$ from the LHS and $\frac{1}{\mathbf{b}_k + g(k)x_1 - x_2}$ from the RHS of Equation (21). Let $\mathbf{m}'[\mathbf{b}_k] = \mathbf{m}[\mathbf{b}_k] - 1$ and $\mathbf{m}'[\mathbf{b}_j] = \mathbf{m}[\mathbf{b}_j]$ for all $j \neq k$. Let $\mathbf{c}' = \mathbf{c} \setminus \{\mathbf{c}_{i^*}\}$. The lemma follows from the inductive hypothesis. $\square$

Remark 5.10. We elaborate more on the requirement that $\mathcal{R} \cong \mathcal{R}_1 \times \mathcal{R}_2$ (i.e., we work over a ring that splits into two fields). From Lemma 3.8, we can obtain maximally large CRT basis fields by choosing parameters such that $x^d + 1$ splits into two irreducible factors in $\mathbb{Z}_q[x]$. This gives an expression of the form $x^n + 1 = (x^{d/2} - r_1)(x^{d/2} - r_2)$ where the fields $\mathbb{Z}_q[x]/(x^{d/2} - r_1)$ and $\mathbb{Z}_q[x]/(x^{d/2} - r_2)$ have size $q^{d/2}$. For large enough q , this is exponentially sized.

Recall that in Construction 5.11, we need to argue that for $\alpha, \beta \xleftarrow{\$} \mathcal{R}$, the fractional terms $\left\{ \frac{1}{\mathbf{a}_i + \mathbf{c}_i \alpha - \beta} \right\}_{i \in [M]}$, $\left\{ \frac{1}{\mathbf{b}_j + g(j) \alpha - \beta} \right\}_{j \in [N]}$ are invertible with high probability. Indeed, each element $\mathbf{a}_i + \mathbf{c}_i \alpha - \beta$ is both a uniformly random element in the full ring $\mathcal{R}$ and after projecting onto the CRT basis $\mathcal{R}_1$ and $\mathcal{R}_2$. For each CRT basis slot in $\mathcal{R}_1$ and $\mathcal{R}_2$, there is a $\Theta(1/q^{d/2})$ chance

that the element is zero. Since an element in $\mathcal{R}$ is invertible if and only if none of its CRT basis slots are zero, we conclude that the term $\mathbf{a}_i + \mathbf{c}_i\alpha - \beta$ is invertible over $\mathcal{R}$ with probability $\Theta((1 - \frac{1}{q^{d/2}})^2) = \Theta(1 - \frac{2}{q^{d/2}} + \frac{1}{q^d})$. For large enough q and d , this happens with overwhelming probability.

5.2.2 Ring-LogUp PIOP

We now give a PIOP for the lattice-based log-derivatives relation in Lemma 5.8.

Construction 5.11 (Ring-LogUp PIOP). We give a PIOP $(\mathbf{P}, \mathbf{V})$ for the relation R_{LLU} based on Lemma 5.8. We consider the ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ for power-of-two d and large prime q . Let $x^d + 1$ split into two irreducibles mod q such that $\mathcal{R} \cong \mathcal{R}_1 \times \mathcal{R}_2$ is the CRT basis representation. Let $\mathcal{C} \subset \mathcal{R}$ be an exponentially-sized challenge space consisting of all ring elements with binary coefficients.

Let $g : [|\mathcal{C}|] \rightarrow \mathcal{C}$ be a publicly known injective map that sends an integer to an element of the challenge space. The prover $\mathbf{P}$ receives index $i = g([N])$, instance $\mathbf{x} = (\mathcal{R}, M, N)$, and witness $\mathbf{w} = (a, b, \mathbf{c})$ where $g([N]) = \{g(1), \dots, g(N)\}$. We assume M and N are smaller than the size of the challenge space $\mathcal{C} \subset \mathcal{R}$. The verifier receives instance $\mathbf{x}$ and has query access to the index and witness given by oracles $\widehat{\mathbf{c}}, \widehat{g_{[N]}}, \widehat{\mathbf{a}}$, and $\widehat{\mathbf{b}}$ where $\widehat{g_{[N]}} := g([N])$.

- The prover $\mathbf{P}$ creates multiplicity vector $\mathbf{m} \in \mathbb{Z}_q^N$ that maps an element $\mathbf{b}_j \in \mathbf{b}$ to the number of times $\mathbf{b}_j$ appears in $\mathbf{a}$. The prover sends an oracle for $\mathbf{m}$, denoted $\widehat{\mathbf{m}} \in \mathcal{R}[x_1, \dots, x_{\log N}]$ to the verifier.
- The verifier $\mathbf{V}$ samples $\alpha, \beta \leftarrow \mathcal{C}$ and sends α, β to the prover.
- The prover $\mathbf{P}$ creates multivariate polynomials $A(x) \in \mathcal{R}[x_1, \dots, x_{\log M}]$ and $B(x) \in \mathcal{R}[x_1, \dots, x_{\log N}]$ such that

$$A(\text{bin}(i)) := \frac{1}{\mathbf{a}_i + \mathbf{c}_i\alpha - \beta}, \quad B(\text{bin}(j)) := \frac{\mathbf{m}[\mathbf{b}_j]}{\mathbf{b}_j + g(j)\alpha - \beta},$$

where $\text{bin}(i), \text{bin}(j)$ are the binary decomposition of integers i, j , respectively. The prover sends oracles $\widehat{A}(x), \widehat{B}(x)$ to the verifier.

- The prover $\mathbf{P}$ computes $v = \sum_{\mathbf{x} \in \{0,1\}^{\log M}} \widehat{A}(\mathbf{x}) = \sum_{\mathbf{x} \in \{0,1\}^{\log N}} \widehat{B}(\mathbf{x})$. The prover sends v to the verifier.
- The prover $\mathbf{P}$ and the verifier $\mathbf{V}$ run two sum-checks in parallel for the claims

$$v = \sum_{\mathbf{x} \in \{0,1\}^{\log M}} \widehat{A}(\mathbf{x}), \quad v = \sum_{\mathbf{x} \in \{0,1\}^{\log N}} \widehat{B}(\mathbf{x})$$

- The verifier $\mathbf{V}$ samples random $\gamma \xleftarrow{\$} \mathcal{C}^{\log M}$ and $\delta \xleftarrow{\$} \mathcal{C}^{\log N}$ and sends them to the prover $\mathbf{P}$.
- The prover and verifier engage in two parallel zero-check protocols for the following relations to check the well-formedness of $A(\mathbf{x})$ and $B(\mathbf{x})$. Let

$$\begin{aligned} \widehat{P}(\mathbf{x}) &= \widehat{A}(\mathbf{x})(\widehat{\mathbf{a}}(\mathbf{x}) + \widehat{\mathbf{c}}(\mathbf{x})\alpha - \beta) - 1 \\ \widehat{Q}(\mathbf{x}) &= \widehat{B}(\mathbf{x})(\widehat{\mathbf{b}}(\mathbf{x}) + \widehat{g_{[N]}}(\mathbf{x})\alpha - \beta) - \widehat{\mathbf{m}}(\mathbf{x}) \end{aligned}$$

such that $\hat{P}(\mathbf{x}) \in \mathcal{R}[x_1, \dots, x_{\log M}]$ and $\hat{Q}(\mathbf{x}) \in \mathcal{R}[x_1, \dots, x_{\log N}]$. The zero-check claims are

$$\sum_{\mathbf{x} \in \{0,1\}^{\log M}} \hat{P}(\mathbf{x}) \text{EQ}(\mathbf{x}, \gamma) = 0$$

$$\sum_{\mathbf{x} \in \{0,1\}^{\log N}} \hat{Q}(\mathbf{x}) \text{EQ}(\mathbf{x}, \delta) = 0$$

where in the final round, the verifier makes the queries to $\hat{P}, \hat{Q}$ using oracle queries to $\hat{A}, \hat{\mathbf{a}}, \hat{\mathbf{c}}, \hat{B}, \hat{\mathbf{b}}, \widehat{g_{[N]}}, \hat{\mathbf{m}}$.

- The verifier $\mathbf{V}$ checks that $\mathbf{m}$ contains integer entries by running an integer-check sub-protocol (Construction B.15). This requires sampling random $\mathbf{r} \xleftarrow{\$} \mathbb{Z}_q^{\log N}$ and making a single oracle query to $\hat{\mathbf{m}}$ to check if $\hat{\mathbf{m}}(\mathbf{r}) \in \mathbb{Z}_q$.
- The verifier $\mathbf{V}$ checks that $\mathbf{c}$ contains elements from the challenge space $\mathcal{C}$ by running a binary-check sub-protocol (see Construction B.19).

We refer to Appendix F for the proof of the following lemma.

Lemma 5.12. *For every suitable ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ of the form $\mathcal{R} \cong \mathcal{R}_1 \times \mathcal{R}_2$ with challenge space $\mathcal{C}$ consisting of all ring elements with binary coefficients, and positive integers $M, N < |\mathcal{C}|$, Construction 5.11 is a holographic PIOP for the indexed relation R_{ILU} that has the following efficiency parameters,*

- *Soundness error:* $O(\frac{M+N}{|\mathcal{C}|} + \frac{d+\log M+\log N}{q})$
- *Round complexity:* $O(\log M + \log N)$
- *Proof length:* $O(N + \text{poly}(d)M)\mathcal{R}$
- *Prover cost:* $O(N + \text{poly}(d)M)\mathcal{R}$
- *Verifier cost:* $O(\log N + d \log M)\mathcal{R}$
- *Query complexity:* $O(d)$

where $|\mathcal{C}| = 2^d$ and q is the size of the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

6 PIOP for Batch-Verification of RAM Updates over Rings

Arithmetic circuit-based representations are typically inefficient in applications of verifiable computation to scenarios involving the result of a program execution on memory/state. Examples of such applications include proving the correctness of query execution on a database, verifiable inference using a decision tree, or proving the validity of (a batch of) updates to a table of account balances. A general and useful abstraction for such applications is *verifiable random access memory* (RAM), where a prover convinces a verifier that the RAM has been accessed/updated by a sequence of read/write operations in a correct and consistent manner. We specifically focus on *batch-verification* of RAM, where verifiable update of the RAM state is required for (small) batches of updates. A prominent application of batch-verifiable RAM is *blockchain rollup* [RU19] that allows a blockchain to efficiently verify succinct proofs attesting to the correctness of expensive off-chain computation. We note here that lookup protocols already suffice for batch-verifiable read-only memory (ROM), i.e., when all operations are read operations. However, handling write operations introduces additional complexities, as noted in certain recent works [DGPPS24; OLB25].

Current solutions for batch-verification of RAM can be broadly classified into two categories: (i) solutions based on accumulators [OWWB20; CFHKKO22] realized using groups of unknown order (e.g., RSA, class-groups) [CL02; BBF19; OWWB20; CFHKKO22]⁴, and (ii) solutions [DGPPS24; OLB25] using pairing-based, logarithmic-derivative-style lookup arguments [Hab22; GK22; EFG22; CFFLL24], that typically work over finite fields with large prime order. The first approach is inherently incompatible with lattice-rings and the second approach does not readily extend from fields to rings (see Section 4). This leaves open the question of designing protocols for batch-verification of RAM that work over rings and are compatible with mainstream lattice-based proof systems [BBCPGL18; ENS20; BLNS20; ACK21; LNP22; BS22; BCS23; NS24; BC24; BC25].

In this section, we describe an application of our lookup arguments over rings to realize a new PIOP over rings for batch-verification of random access memory (RAM) updates. We broadly follow the blueprint for realizing batch-verification of RAM updates from [DGPPS24]. However, a key difference is that we consider RAMs where the entries are arbitrary ring elements, as opposed to elements of a finite field of large prime order as in [DGPPS24]. This requires us to additionally introduce new multilinear PIOPs over rings for (i) checking read/write consistency of RAM updates (Section 6.1), and (ii) checking that the initial and final RAM states before and after a batch of updates are identical in all memory locations that are “untouched” by the updates (Section 6.2). Putting these sub-protocols together with our lookup arguments over rings and compiling with a suitable polynomial commitment scheme yields the first succinct argument system for batch-verification of updates over RAMs where all entries are arbitrary ring elements.

Defining RAM. We begin by establishing a working definition of RAM that is used in the rest of the section. Informally, a RAM maps indices (addresses) to values, where we assume that values come from a ring $\mathcal{R}$ with a suitable challenge space, while indices come from a set $I \subset \mathbb{N}$. For simplicity of exposition, we limit our focus to RAMs where I is of the form $[m]$ for some integer $m \in \mathbb{N}$ (which may be different from size of the RAM M). Finally, for an index, there should be at most one value in the RAM, i.e., the association is unambiguous. In this paper, we consider RAMs where the values are elements of a ring $\mathcal{R}$ (associated with an appropriate challenge space).

⁴While accumulators can also be realized using Merkle-trees [Mer88], they are not batching-friendly; see discussion in [DGPPS24].

Definition 6.1 (Random Access Memory (RAM)). *Given integers $M, m \in \mathbb{N}$ and a ring $\mathcal{R}$, a RAM of size M over a set of indices $I = [m]$ is a tuple $V = (\mathbf{a}, \mathbf{v}) \in [m]^M \times \mathcal{R}^M$ such that $\forall i, j \in [M], v_i = v_j$ whenever $a_i = a_j$. We think of V as a table with vectors $\mathbf{a}$ and $\mathbf{v}$ denoting its columns. The set of all such RAMs will be denoted by $\mathcal{V}_{\mathcal{R}, m, M}$.*

For a RAM $V = (\mathbf{a}, \mathbf{v}) \in \mathcal{V}_{\mathcal{R}, m, M}$, we refer to tuples (a_i, v_i) , $i \in [M]$ as records of the RAM V . We use the key-value access notation $v = V[a]$ to mean that (a, v) is a record of V (note there can be multiple such records according to our definition). When we consider RAMs in $\mathcal{V}_{\mathcal{R}, m, M}$ for $m = M$ where the first column (of indices) consists of unique entries in $[M]$ (i.e., the address associated with value v_i is simply i), we denote such RAMs by $V \in \mathcal{R}^M$.

RAM Operations. For a RAM $V \in \mathcal{V}_{\mathcal{R}, m, M}$, a RAM operation is a three tuple (op, a, v) with $op \in \{0, 1\}$, $a \in [m]$ and $v \in \mathcal{R}$. We consider two kinds of RAM operations: (i) an operation with $op = 0$ is called a *read* operation which denotes reading a value v mapped to index a in the RAM. Such an operation is formally denoted as a *read* operation and is represented by the tuple $(0, a, \perp)$ (where $\perp$ is a special symbol used to denote an element not in the ring $\mathcal{R}$); (ii) an operation with $op = 1$ is called a *write* operation, which denotes associating a new value v' with index a in the RAM. Such an operation is formally denoted as a *write* operation and is represented by the tuple $(1, a, v')$.

Achieving Batch-Verification of RAM. We use vectors in $\mathcal{R}^M$ to denote “large” RAMs, where the index column is implicitly assumed to be the set of unique values $(1, \dots, M)$. Let $V, V' \in \mathcal{R}^M$ denote the initial and final RAM states. For $\mathcal{O} = \{0, 1\} \times [M] \times (\mathcal{R} \cup \{\perp\})$, let $o \in \mathcal{O}^k$ be a sequence of k RAM operations that *update* V to V' . More formally, if $V_0 := V$, and for all $i \in [k]$, V_i is obtained from V_{i-1} by updating according to o_i , then $V_k = V'$. To prove the transformation of V to V' via the operation sequence o , we proceed via the following sub-protocols:

- **Sub-protocol-1: Indexed Lookup.** We isolate the k -sized sub-RAMs $W = (\mathbf{a}, \mathbf{v})$ and $W' = (\mathbf{a}, \mathbf{v}')$ of the large RAMs V and V' consisting of rows corresponding to indices in $\mathbf{a}$. This requires proving $\mathbf{v} = V[\mathbf{a}]$ and $\mathbf{v}' = V'[\mathbf{a}]$. Observe that this can be proven using two invocations of our indexed ring-based lookup arguments (see Section 5.1 and Section 5.2), where the same set of lookup addresses $\mathbf{a}$ is reused across both instances.
- **Sub-protocol-2: Memory Consistency.** On the isolated sub-RAMs W and W' of size k , we use memory checking arguments to prove that sequence o correctly and consistently updates W to W' . The authors of [DGPPS24] presented a univariate PIOP to prove RAM consistency, which is not suitable for adaptation to the ring setting due to its extensive use of polynomial division operations. In Section 6.1, we propose a new multilinear PIOP for proving memory consistency of RAM updates over rings. Our protocol is based on known offline memory checking techniques from [BEGKN91] and can hence be used in conjunction with our ring-based lookup arguments.
- **Sub-protocol-3: Almost Identical RAM States.** Finally, we show that the RAMs V and V' are *almost* identical, i.e., they are identical outside the indices $\mathbf{v}$ in the sequence of operations. Again, [DGPPS24] realized this step via a univariate PIOP, which extensively uses polynomial division operations. In Section 6.2, we propose a new multilinear PIOP for proving almost identical RAM states over rings.

6.1 RAMs are Memory Consistent with Update

In this subsection, we present a multilinear PIOP for the memory-consistency relation described formally below. The high-level goal is to prove that two RAMs V and V' are related according to a given sequence of read and write operations.

Definition 6.2. *The memory-consistency relation R_{MC} consists of instances $\mathbb{x} = (\mathcal{R}, M, k)$ and witnesses $\mathbb{w} = (V, o, V')$ such that*

- $V, V' \in \mathcal{R}^M$;
- for $\mathcal{O} = \{0, 1\} \times [M] \times \mathcal{R}$, $o \in \mathcal{O}^k$ is a sequence containing read operations $(0, a, \perp)$ and write operations $(1, a, v')$; and
- if $V_0 := V$, and for all $i \in [k]$, V_i is obtained from V_{i-1} by updating according to o_i , then $V_k = V'$.

Our construction is based on techniques for offline memory checking from [BEGKN91], and we use these techniques to construct an offline memory checker suitable for our purposes.

Construction 6.3 (Offline Memory Checker). The $\text{memcheck}((\mathcal{R}, M, k), (V, o, V'))$ procedure performs the following steps:

1. Set $V_0 := (0, \dots, 0) \in \mathcal{R}^M$. Set $T_0 := (0, \dots, 0) \in \mathbb{N}^M$. Set $R, W = []$, i.e., to empty lists.
2. Consider an augmented operations list o^* in which all elements of V are written in address order, then operations in o are applied, and finally, all elements are read in address order.
3. For $i = 1, \dots, 2M + k$, perform one of the following to update V_{i-1}, T_{i-1} , R and W depending on whether $o_i^* \in \{0, 1\} \times [M] \times \mathcal{R}$ is a read or write operation.

$\text{write}(1, a, v')$:

- Read the value $v = V_{i-1}[a]$ and time $t = T_{i-1}[a]$.
- Check whether t is less than the current time $t' := i$.
- Append (v, a, t) to R .
- Write $v' \rightarrow V_i[a]$, $t' \rightarrow T_i[a]$.
- Append (v', a, t') to W .

$\text{read}(0, a, v')$:

- Read the value $v = V_{i-1}[a]$ and time $t = T_{i-1}[a]$.
- Check whether t is less than the current time $t' := i$.
- Append (v, a, t) to R .
- Write $v \rightarrow V_i[a]$, $t' \rightarrow T_i[a]$.

- Append (v, a, t') to W .

Output $R, W \in (\mathcal{R} \times [M] \times \mathbb{N})^{2M+k}$.

Next, we relate our offline memory checker to the memory-consistency relation, using a similar strategy to [BEGKN91, Lemma 1].

Lemma 6.4. *Let $\mathfrak{x} = (\mathcal{R}, M, k)$, and $\mathfrak{w} = (V, o, V') \in \mathcal{R}^M \times \mathcal{O}^k \times \mathcal{R}^M$. Parse o as three lists $o_1 \in \{0, 1\}^k$, $o_2 \in [M]^k$ and $o_3 \in \mathcal{R}^k$. Then, $(\mathfrak{x}, \mathfrak{w}) \in R_{\text{MC}}$ if and only if there exist $R, W \in (\mathcal{R} \times [M] \times \mathbb{N})^{2M+k}$ (parsed as lists V_R, A_R, T_R and V_W, A_W, T_W of values, addresses and timestamps) such that*

1. $V_R[0 : M] = 0^M$, $A_R[0 : M] = [M]$, $T_R[0 : M] = 0^M$;
2. $V_W[0 : M] = V$, $A_W[0 : M] = [M]$, $T_R[0 : M] = [M]$;
3. $V_W[M + k : 2M + k] = V'$, $A_W[M + k : 2M + k] = [M]$, $T_W[M + k : 2M + k] = M + k + [M]$;
4. $R[M : 2M + k]$ and $W[0 : M + k]$ contain the same records up to ordering;
5. The records in R and W all apply to the same addresses, and in particular, the addresses at timestamps from M to $M + k$ are equal to o_2 ;
6. The records in R and W use the same value when o_1^* contains a read operation;
7. The records in W use the same value as o_3 when o_1^* contains a write operation;
8. We have that $T_W = [2M + k]$; and
9. We have that all entries of $T_W - T_R$ are greater than 0.

Proof. If $(\mathfrak{x}, \mathfrak{w}) \in R_{\text{MC}}$, then the required $R, W \in (\mathcal{R} \times [M] \times \mathbb{N})^{2M+k}$ can be obtained by running **memcheck**. It is easy to see that all conditions with the exception of 4 are satisfied, by construction of **memcheck**.

For condition 4, note that with the exception of the M final read operations, every record (v, a, t) in W is also stored in R the next time a read or write operation is applied to address a . Since no address is accessed more than once during the M initial write operations, all records from $W[0 : M + k]$ appear in $R[M : 2M + k]$. Finally, all records from $W[0 : M + k]$ are unique because they have a unique timestamp, so comparing cardinalities shows that $W[0 : M + k]$ is equal to $R[M : 2M + k]$.

Conversely, suppose that there exist $R, W \in (\mathcal{R} \times [M] \times \mathbb{N})^{2M+k}$ satisfying conditions 1 to 9. Taking all of the conditions except for 4 together, we see that locally, R and W are consistent with an execution of **memcheck**, in the sense that the i -th records of R and W are consistent with a read or write operation corresponding to o . However, it remains to show that R and W are globally consistent. This is guaranteed by 4, because it shows that R and W contain the same records, except for the initial and final records, and if a record (v, a, t) in R appeared at an earlier index than the corresponding record in W , it would be impossible for the timestamp of (v, a, t) to increase. $\square$

Construction 6.5 (Memory Consistency with Updates). We construct a PIOP for the relation R_{MC} . The prover **P** takes as input the instance $\mathfrak{x} = (\mathcal{R}, M, k)$, and witness $\mathfrak{w} = (V, o, V')$; the verifier **V** has query access to the witness $\mathfrak{w}$ and takes as input the instance $\mathfrak{x}$.

1. The prover $\mathbf{P}$ computes $R, W := \text{memcheck}((\mathcal{R}, M, k), (V, o, V'))$. The prover computes lists $V_R, V_W \in \mathcal{R}^{2M+k}$, $A_R, A_W \in [M]^{2M+k}$ and $T_R, T_W \in \mathbb{N}^{2M+k}$ of values, addresses, and timestamps inside R and W . The prover sends $V_R[M : M+k] \in \mathcal{R}^k$, $V_R[M+k : 2M+k] \in \mathcal{R}^M$, $V_W[M : M+k] \in \mathcal{R}^k$, $T_R[M : M+k] \in \mathbb{N}^k$, $T_R[M+k : 2M+k] \in \mathbb{N}^k$ to the verifier.
2. The verifier samples $\alpha_1, \alpha_2, \alpha_3 \leftarrow \mathcal{C}$.
3. The prover and verifier engage in the following subprotocols. Here, the verifier simply computes the answers to queries on publicly known witness vectors for themselves, and to combinations of vectors by querying the individual vectors.

- An indexed lookup argument with index $\mathfrak{i} = (\{0, 1\})$, instance $\mathfrak{x} = (\mathcal{R}, k, 2)$ and witness $\mathfrak{w} = (o_1, \{0, 1\})$ to check whether o_1 contains values in $\{0, 1\}$.
- An indexed lookup argument with index $\mathfrak{i} = ([M])$, instance $\mathfrak{x} = (\mathcal{R}, k, M)$ and witness $\mathfrak{w} = (o_2, [M])$ to check whether o_2 contains values in $[M]$.
- A permutation argument with instance $\mathfrak{x} = (\mathcal{R}, M+k, \mathcal{C}, g)$ and witness containing the vectors $(\alpha_1 V_R + \alpha_2 A_R + \alpha_3 T_R)[M : 2M+k]$ and $(\alpha_1 V_W + \alpha_2 A_W + \alpha_3 T_W)[0 : M+k]$ to check Item 4, where $V_W[: M]$ is defined in terms of V , $A_R[M : M+k]$ and $A_W[M : M+k]$ are defined using o_2 , $A_R[M+k : 2M+k]$ and $A_W[: M]$ are defined using $[M]$ in accordance with Item 5 and Item 3, and T_W is defined using $[2M+k]$ in accordance with Item 8⁵.
- A Hadamard product argument (Construction B.5) with witness $\mathfrak{w}$ and instance $\mathfrak{x}$:

$$(\mathfrak{w}, \mathfrak{x}) = ((1^M || o_1 || 0^M, V_R - V || o_3 || V', V_W - V_R), (\mathcal{R}, k))$$

to check Item 6 and Item 7.

- An indexed lookup argument with index $\mathfrak{i} = ([k])$, instance $\mathfrak{x} = (\mathcal{R}, k, M)$ and witness $\mathfrak{w} = (o_2, [M])$.
- An indexed lookup argument with index $\mathfrak{i} = ([2M+k])$, instance $\mathfrak{x} = (\mathcal{R}, 2M+k, 2M+k)$ and witness $\mathfrak{w} = ([2M+k] - T_R, [2M+k], [2M+k] - T_R)$ to check Item 9, with $T_R[: M]$ defined by 0^M in accordance with Item 8.

Lemma 6.6. *For every $\mathcal{R} = \mathbb{Z}_q/(x^d + 1)$ as in Section 5 and positive integers $M, k < |\mathcal{C}|$, Construction 6.5 is a holographic PIOPP for the indexed relation R_{MC} that supports instances over $\mathcal{R}$ in which M and N are powers of 2, and has the following parameters:*

- *Soundness error:*

$$\frac{1}{|\mathcal{C}|} + 4 \cdot \text{Soundness}(\text{Lookup}, 2M+k) + \text{Soundness}(\text{Hadamard}, 2M+k)$$

- *Round complexity:*

$$O(1) + \max(\text{Rounds}(\text{Lookup}, 2M+k), \text{Rounds}(\text{Hadamard}, 2M+k))$$

⁵Note that a permutation argument checking that the two witness vectors are a permutation can easily be obtained from a lookup argument by setting the lookup and table vectors to be the two witness vectors, and in the case of the log-up based protocol, setting the multiplicity vector equal to the all-ones vector.

- *Proof length:*

$$O(M + k) + 4 \cdot \text{ProofLen}(\text{Lookup}, 2M + k) + \text{ProofLen}(\text{Hadamard}, 2M + k)$$

- *Prover cost:*

$$O(M + k) + 4 \cdot \text{Prover}(\text{Lookup}, 2M + k) + \text{Prover}(\text{Hadamard}, 2M + k)$$

- *Verifier cost:*

$$O(\text{Verifier}(\text{Lookup}, 2M + k) + \text{Verifier}(\text{Hadamard}, 2M + k))$$

- *Query complexity:*

$$O(\text{Queries}(\text{Lookup}, 2M + k) + \text{Queries}(\text{Hadamard}, 2M + k))$$

where $\mathcal{C}$ denotes the set of all ring elements with binary coefficients, and where e.g. $\text{Soundness}(\text{Lookup}, M)$ represents the soundness error of a lookup protocol with vector lengths at most M .

Proof sketch. First, note that the completeness and efficiency properties of the protocol follow directly from that of the corresponding subprotocols and Lemma 6.4.

For soundness, note that if the memory consistency relation is not satisfied, then some item in Lemma 6.4 is not true, and will be caught by one of the subprotocols. In particular, if Item 4 is not satisfied, then there is some record in $R[M : 2M + k]$ which does not appear in $W[0 : M + k]$, or vice versa. Then, except with probability at most $(2M + k)/|\mathcal{C}|$, the linear combination of the record elements using the verifier's challenges will satisfy the same property, and this will be caught by the permutation subprotocol. $\square$

Plugging into the above protocol: (i) any of our lattice lookup PIOPs, (ii) the multilinear sumcheck protocol over rings from Appendix A.3, and (iii) the Hadamard product argument from Appendix B.2, we immediately obtain the following corollary.

Corollary 6.7. *For every ring $\mathcal{R} = \mathbb{Z}_q/(x^d + 1)$ as in Section 5 and positive integers $M, k < |\mathcal{C}|$, there exists a holographic PIOP for the relation R_{MC} that has the following parameters:*

- *Soundness error:* $O(\frac{k+M}{|\mathcal{C}|} + \frac{d+\log k+\log M}{|\mathcal{S}|})$
- *Round complexity:* $O(\log M + \log k)$
- *Proof length:* $O(M + \text{poly}(d)k)$
- *Prover cost:* $O(M + \text{poly}(d)k)\mathcal{R}$
- *Verifier cost:* $O(\log M + d \log k)\mathcal{R}$
- *Query complexity:* $O(d)$

where $\mathcal{C}$ denotes the set of all ring elements with binary coefficients and $\mathcal{S}$ denotes the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

6.2 Multilinear PIOP for Almost Identical RAM states

In this subsection, we present a multilinear PIOP for the almost identical RAM states relation described formally below. The high-level goal is to prove that two RAMs V and V' have the same values everywhere *except* for a subset of addresses.

Definition 6.8 (Almost Identical RAM States). *The almost identical RAM states relation R_{AidRAM} is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, M, k), (V, V', \mathbf{a}))$$

where

- $V, V' \in \mathcal{R}^M$ for a positive integer $M \in \mathbb{N}$;
- $\mathbf{a} \in [M]^k$ for a positive integer $k \in \mathbb{N}$; and
- letting $\text{uniq}(\mathbf{a})$ denote a vector with entries containing the unique values in $\mathbf{a}$, $V[i] = V'[i]$ for all $i \in [M]$ such that $i \notin \text{uniq}(\mathbf{a})$.

Defining Some Multilinear Polynomials. Let $\widehat{V} : \mathcal{R}^{\log M} \rightarrow \mathcal{R}$, $\widehat{V}' : \mathcal{R}^{\log M} \rightarrow \mathcal{R}$, $\widehat{a} : \mathcal{R}^{\log k} \rightarrow \mathcal{R}$, and $\widehat{u} : \mathcal{R}^{\log M} \rightarrow \mathcal{R}$ be multilinear polynomials defined as follows:

- Let $V(\mathbf{x})$, $V'(\mathbf{x})$, and $a(\mathbf{x})$ be polynomials defined as:

$$V(\mathbf{x}) := \sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{EQ}(x, \mathbf{i}) \cdot V[\text{int}(i)]$$

$$V'(\mathbf{x}) := \sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{EQ}(x, \mathbf{i}) \cdot V'[\text{int}(i)]$$

$$a(\mathbf{x}) := \sum_{\mathbf{j} \in \{0,1\}^{\log k}} \text{EQ}(x, \mathbf{j}) \cdot a[\text{int}(j)]$$

We define the *witness* multilinear polynomials $\widehat{V}$, $\widehat{V}'$, and $\widehat{a}$ to be the multilinear extensions of $V(\mathbf{x})$, $V'(\mathbf{x})$, and $a(\mathbf{x})$, respectively, over the ring $\mathcal{R}$.

- Let $\mathbf{u} \in \mathcal{R}^M$ be a vector such that for each $i \in [M]$, $\mathbf{u}_i = 1$ if $i \in \text{uniq}(\mathbf{a})$ and $\mathbf{u}_i = 0$, otherwise. Let $\mathbf{u}(\mathbf{x})$ be a polynomial defined as:

$$\mathbf{u}(\mathbf{x}) := \sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{EQ}(x, \mathbf{i}) \cdot \left(\sum_{a_{\text{int}(i)} \in \text{uniq}(\mathbf{a})} \text{EQ}(x, \text{bin}(a_{\text{int}(i)})) \right)$$

We define the *auxiliary* multilinear polynomial $\widehat{\mathbf{u}}$ to be the multilinear extension of $\mathbf{u}(\mathbf{x})$ over the ring $\mathcal{R}$.

Construction 6.9 (Multilinear PIOP for Almost Identical RAMs). We construct a PIOP for the relation R_{AidRAM} . The prover $\mathbf{P}$ takes as input the instance $\mathbf{x} = (\mathcal{R}, M, k)$ and witness $\mathbf{w} = (V, V', \mathbf{a})$ as described above; the verifier $\mathbf{V}$ takes as input the instance $\mathbf{x}$ and has query access to the witness multilinear polynomials $(\widehat{V}, \widehat{V}', \widehat{a})$ and the auxiliary multilinear polynomial $\widehat{\mathbf{u}}$ as defined above.

1. $\mathbf{V}$ samples a challenge vector $\mathbf{r} \leftarrow \mathcal{C}^{\log M}$ and sends $(\mathbf{r}, \mathbf{s})$ to $\mathbf{P}$.
2. $\mathbf{P}$ and $\mathbf{V}$ engage in a multilinear sum-check PIOP to check that the following relation holds:

$$\sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{EQ}(\mathbf{r}, \mathbf{i}) \cdot (1 - \widehat{u}(\mathbf{i})) \cdot (\widehat{V}(\mathbf{i}) - \widehat{V}'(\mathbf{i})) = 0$$

3. $\mathbf{V}$ samples a binary challenge vector $\mathbf{s} \leftarrow \{0,1\}^{\log M}$ and sends $\mathbf{s}$ to $\mathbf{P}$.
4. $\mathbf{P}$ computes $u = \widetilde{u}(\mathbf{s})$, vector $\mathbf{v} = (\text{EQ}(\mathbf{s}, \text{bin}(a_i)))_{i \in [k]}$, polynomial $v(x)$ as

$$v(x) := \sum_{\mathbf{i} \in \{0,1\}^{\log k}} \text{EQ}(\mathbf{i}, x) \cdot \text{EQ}(\mathbf{s}, \text{bin}(a_{\text{int}(\mathbf{i})}))$$

and multilinear polynomial $\widetilde{v} : \mathcal{R}^{\log k} \rightarrow \mathcal{R}$ as the multilinear extension of $\mathbf{v}$. $\mathbf{P}$ sends across $(u, \widetilde{v})$ to $\mathbf{V}$. $\mathbf{V}$ makes an oracle call to $\widetilde{u}$ to verify that $u = \widetilde{u}(\mathbf{s})$.

5. $\mathbf{P}$ and $\mathbf{V}$ then engage in the following sub-protocols:

- A multilinear sumcheck protocol for the relation $u = \sum_{\mathbf{i} \in \{0,1\}^{\log k}} \widetilde{v}(\mathbf{i})$ (to check the well-formedness of $\widetilde{u}$ with respect to $\text{uniq}(\mathbf{a})$).
- A lookup argument for the indexed lookup relation R_{ILU} with respect to the set of tuples $(\mathbf{i}_{\text{ILU}}, \mathbf{x}_{\text{ILU}}, \mathbf{w}_{\text{ILU}})$ where

$$\mathbf{i}_{\text{ILU}} = g([M]), \quad \mathbf{x}_{\text{ILU}} = (\mathcal{R}, g, k, M), \quad \mathbf{w}_{\text{ILU}} = (\mathbf{v}, \mathbf{e}, g(\text{uniq}(\mathbf{a})))$$

where the vector $\mathbf{e} = (\text{EQ}(\mathbf{s}, \text{bin}(i)))_{i \in [M]}$, and g is a publicly known map.

Lemma 6.10. *For every ring $\mathcal{R} = \mathbb{Z}_q/(x^d + 1)$ as in Section 5 and positive integers $M, k < |\mathcal{C}|$, Construction 6.9 is a holographic PIOP for the indexed relation R_{AidRAM} that has the following parameters:*

- *Soundness error:* $O(\frac{\log(M+k)}{|\mathcal{C}|}) + \text{Soundness}(\text{Sumcheck}, M) + \text{Soundness}(\text{Lookup}, M, k) + \text{Soundness}(\text{Sumcheck}, k)$
- *Round complexity:* $O(1) + \text{Rounds}(\text{Sumcheck}, M) + \max(\text{Rounds}(\text{Lookup}, M, k), \text{Rounds}(\text{Sumcheck}, k))$
- *Proof length:* $O(M+k) + \text{ProofLen}(\text{Sumcheck}, M) + \text{ProofLen}(\text{Lookup}, M, k) + \text{ProofLen}(\text{Sumcheck}, k)$
- *Prover cost:* $O(M+k)\mathcal{R} + \text{Prover}(\text{Sumcheck}, M) + \text{Prover}(\text{Lookup}, M, k) + \text{Prover}(\text{Sumcheck}, k)$
- *Verifier cost:* $O(\log(M+k)) + \text{Verifier}(\text{Sumcheck}, M) + \text{Verifier}(\text{Lookup}, M, k) + \text{Verifier}(\text{Sumcheck}, k)$
- *Query complexity:* $O(1) + \text{Queries}(\text{Sumcheck}, M) + \text{Queries}(\text{Lookup}, M, k) + \text{Queries}(\text{Sumcheck}, k)$

where $\mathcal{C}$ denotes the set of all ring elements with binary coefficients, and where e.g. $\text{Soundness}(\text{Lookup}, M, k)$ represents the soundness error of an indexed lookup PIOP over the ring $\mathcal{R}$ where the lengths of the table and the lookup vector are at most M and k , respectively, while $\text{Soundness}(\text{Sumcheck}, k)$ represents the soundness error of a multilinear sumcheck PIOP over the ring $\mathcal{R}$ where the number of variables is at most k .

Proof. We begin by analyzing the efficiency properties.

Efficiency. Round complexity is easy to verify. The prover's cost and the proof length outside of the sumcheck and lookup sub-protocols are $O(M + k)$ as the prover evaluates $u = \tilde{u}(\mathbf{s})$ (which requires $O(M)\mathcal{R}$ work and incurs $O(M)$ proof length) and also computes $\tilde{v}$ (which requires $O(k)\mathcal{R}$ work and incurs $O(k)$ proof length). The verifier samples the challenge vectors of length $\log M$ and $\log k$, respectively, and makes a single oracle call to $\tilde{u}$, outside of the sumcheck and lookup sub-protocols. This concludes the analysis of efficiency.

Completeness. Consider an instance tuple $\mathbf{x} = (\mathcal{R}, M, k)$ and witness tuple $\mathbf{w} = (V, V', \mathbf{a})$ such that $(\mathbf{x}, \mathbf{w})$ satisfies the relation R_{AidRAM} . Then we must have

$$\sum_{i \in [M]} (1 - u_i) \cdot (V[i] - V'[i]) = 0$$

which ensures that for each $i \in [M]$, if $u[i] = 0$, then we must have $V[i] = V'[i]$, which is essentially what the first multilinear sum-check verifies. In addition, if $\tilde{u}$ is well-formed with respect to $\text{uniq}(\mathbf{a})$, we must have

$$\sum_{\mathbf{i} \in \{0,1\}^{\log k}} \tilde{v}(\mathbf{i}) = u$$

At a high level, a summation over all $v(\mathbf{i})$ essentially checks whether $s \in \text{uniq}(\mathbf{a})$, while $u = \tilde{u}(s)$ checks the same by construction. More formally, we have

$$\begin{aligned} \sum_{\mathbf{i} \in \{0,1\}^{\log k}} \tilde{v}(\mathbf{i}) &= \sum_{\mathbf{i} \in \{0,1\}^{\log k}} \text{EQ}(\mathbf{s}, \text{bin}(a_{\text{int}(\mathbf{i})})) \\ &= \text{EQ}(\mathbf{s}, \text{bin}(a_1)) + \dots + \text{EQ}(\mathbf{s}, \text{bin}(a_m)) \\ &= \sum_{a_i \in \text{uniq}(\mathbf{a})} \text{EQ}(\mathbf{s}, \text{bin}(a_i)) \\ &= \sum_{\mathbf{i} \in \{0,1\}^{\log M}} \text{EQ}(\mathbf{s}, \mathbf{i}) \cdot \left(\sum_{a_i \in \text{uniq}(\mathbf{a})} \text{EQ}(\mathbf{s}, \text{bin}(a_i)) \right) \\ &= \tilde{u}(\mathbf{s}) \\ &= u \end{aligned}$$

Soundness. Consider an instance tuple $\mathbf{x} = (\mathcal{R}, M, k)$ and witness tuple $\mathbf{w} = (V, V', \mathbf{a})$ such that $(\mathbf{x}, \mathbf{w})$ does not satisfy the relation R_{AidRAM} . Then, by multivariate Schwartz-Zippel, the prover manages to pass the first multilinear sumcheck identity test with probability at most $O(\frac{M}{|\mathcal{C}|})$ and the second multilinear sumcheck identity with probability at most $O(\frac{k}{|\mathcal{C}|})$. The remaining components of the soundness error come from the two invocations of the multilinear sumcheck sub-protocol and one invocation of the lookup sub-protocol. This concludes the analysis of soundness. $\square$

Plugging into the above protocol: (i) any one of our lattice lookup PIOPs, and (ii) the multilinear sumcheck protocol over rings from Appendix A.3, we obtain the following corollary.

Corollary 6.11. *For every ring $\mathcal{R} = \mathbb{Z}_q/(x^d + 1)$ as in Section 5 and positive integers $M, k < |\mathcal{C}|$, there exists a holographic PIOP for the relation R_{AidRAM} that has the following parameters:*

- *Soundness error:* $O(\frac{k+M}{|\mathcal{C}|}) + O(\frac{d+\log k+\log M}{|\mathcal{S}|})$
- *Round complexity:* $O(\log M + \log k)$
- *Proof length:* $O(M + \text{poly}(d)k)$
- *Prover cost:* $O(M + \text{poly}(d)k)\mathcal{R}$
- *Verifier cost:* $O(\log M + d \log k)\mathcal{R}$
- *Query complexity:* $O(d)$

where $\mathcal{C}$ denotes the set of all ring elements with binary coefficients and $\mathcal{S}$ denotes the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

6.3 Putting Everything Together

In this section, we describe how to put together (any one of) our lookup PIOPs with the PIOPs in Section 6.1 and Section 6.2 to achieve a PIOP for batch-verification of RAM over rings. We first formally define the batch-verification of RAM relation below.

We use vectors in $\mathcal{R}^M$ to denote “large” RAMs, where the index column is implicitly assumed to be the set of unique values $(1, \dots, M)$. Let $V, V' \in \mathcal{R}^M$ denote the initial and final RAM states. For $\mathcal{O} = \{0, 1\} \times [M] \times (\mathcal{R} \cup \{\perp\})$, let $o = (\mathbf{a}, \mathbf{v}) \in \mathcal{O}^k$ be a sequence of k RAM operations that update V to V' .

Definition 6.12 (Batch-verification of RAM). *The batch-verification of RAM updates relation $R_{\text{BatchVerRAM}}$ is the set of tuples*

$$(\mathbb{x}, \mathbb{w}) = ((\mathcal{R}, M, k), (V, o = (\mathbf{a}, \mathbf{v}), V'))$$

where

- $V, V' \in \mathcal{R}^M$ for a positive integer $M \in \mathbb{N}$;
- $o = (\mathbf{a}, \mathbf{v}) \in \mathcal{O}^k$ for a positive integer $k \in \mathbb{N}$; and
- if $V_0 := V$, and for all $i \in [k]$, V_i is obtained from V_{i-1} by updating according to o_i , then $V_k = V'$.

To design a PIOP for $R_{\text{BatchVerRAM}}$, we proceed via the following sub-protocols:

- **Sub-protocol-1: Indexed Lookup.** We isolate the k -sized sub-RAMs $W = (\mathbf{a}, \mathbf{v})$ and $W' = (\mathbf{a}, \mathbf{v}')$ of the large RAMs V and V' consisting of rows corresponding to indices in $\mathbf{a}$, and prove that $\mathbf{v} = V[\mathbf{a}]$ and $\mathbf{v}' = V'[\mathbf{a}]$. This requires two invocations of any one of our PIOPs (either from Section 5.1 or from Section 5.2) for the relation R_{ILU} with respect to the following set of tuples (index, statement, witness) tuples.

$$(\mathbf{i}_{\text{ILU},1}, \mathbb{x}_{\text{ILU},1}, \mathbb{w}_{\text{ILU},1}) = (g([M]), (\mathcal{R}, g, k, M), (V, \mathbf{v}, g(\mathbf{a})))$$

$$(\mathbf{i}_{\text{ILU},2}, \mathbb{x}_{\text{ILU},2}, \mathbb{w}_{\text{ILU},2}) = (g([M]), (\mathcal{R}, g, k, M), (V', \mathbf{v}', g(\mathbf{a})))$$

In particular, the same set of lookup addresses $\mathbf{a}$ is reused as the witness for both cases.

- **Sub-protocol-2: Memory Consistency.** On the isolated sub-RAMs W and W' of size k , we use memory checking arguments to prove that sequence o correctly and consistently updates W to W' . More formally, we invoke the memory-consistency PIOP for R_{MC} from Section 6.1 on the following tuple (statement, witness):

$$(\mathbb{x}_{\text{MC}}, \mathbb{w}_{\text{MC}}) = ((\mathcal{R}, k, k), (W, o, W'))$$

- **Sub-protocol-3: Almost Identical RAM States.** Finally, we show that the RAMs V and V' are *almost* identical, i.e., they are identical outside the indices $\mathbf{v}$ in the sequence of operations. More formally, we invoke the memory-consistency PIOP for R_{AidRAM} from Section 6.2 on the following tuple (statement, witness):

$$(\mathbb{x}_{\text{AidRAM}}, \mathbb{w}_{\text{AidRAM}}) = ((\mathcal{R}, M, k), (V, V', \mathbf{a}))$$

We summarize the overall PIOP in the following theorem.

Theorem 6.13 (PIOP for Batch-verification of RAM over Rings). *For every ring $\mathcal{R} = \mathbb{Z}_q/(x^d + 1)$ as in Section 5, and positive integers $M, k < |\mathcal{C}|$, there exists a holographic PIOP for the indexed relation $R_{\text{BatchVerRAM}}$ that has the following parameters:*

- *Soundness error:* $O(\frac{k+M}{|\mathcal{C}|} + \frac{d+\log k+\log M}{|\mathcal{S}|})$
- *Round complexity:* $O(\log M + \log k)$
- *Proof length:* $O(M + \text{poly}(d)k)$
- *Prover cost:* $O(M + \text{poly}(d)k)\mathcal{R}$
- *Verifier cost:* $O(\log M + d \log k)\mathcal{R}$
- *Query complexity:* $O(d)$

where $\mathcal{C}$ denotes the set of all ring elements with binary coefficients and $\mathcal{S}$ denotes the set of all $\mathbb{Z}_q$ -valued ring elements (i.e., those with degree 0).

The proof of the above theorem follows immediately from Lemma 5.6 (or, equivalently, Lemma 5.12), Corollary 6.7, and Corollary 6.11.

A Additional Preliminaries

A.1 Vector operations

Definition A.1 (Cyclic shift). A vector $\mathbf{b} \in \mathcal{R}^N$ is said to be the **cyclic shift** of $\mathbf{a} \in \mathcal{R}^N$, if $\mathbf{b}_N = \mathbf{a}_1$ and, for all $i \in [N - 1]$, it holds that $\mathbf{a}_{i+1} = \mathbf{b}_i$. In this case, we write $\mathbf{b} = \text{shift}(\mathbf{a})$.

Definition A.2 (Merge). Let $\mathbf{a} \in \Sigma^M$ and $\mathbf{b} \in \Sigma^N$ with $\mathbf{a} \subseteq \mathbf{b}$. We define $\text{merge}_{\mathbf{b}}(\mathbf{a})$ to be the set of all vectors $\mathbf{w} \in \Sigma^{N+M}$ obtained as follows: initialize a list $\mathbf{w}$ to be equal to $\mathbf{b}$; then, for each $j \in [M]$, insert $\mathbf{a}_j$ into $\mathbf{w}$ directly to the left of any entry $\mathbf{w}_i$ of $\mathbf{w}$ such that $\mathbf{a}_j = \mathbf{w}_i$. (At least one such entry exists because $\mathbf{a} \subseteq \mathbf{b}$.)

We recall a lemma from [LNP22] that gives an inner-product relation that holds if an integer vector $\mathbf{y} \in \mathbb{Z}^N$ contains binary values.

Lemma A.3 (Lemma 5.2, [LNP22]). Let $\mathbf{y} \in \mathbb{Z}^N$ for $N \in \mathbb{N}$. If $\langle \mathbf{y}, \mathbf{y} - \mathbf{1}^N \rangle = 0$ then $\mathbf{y} \in \{0, 1\}^N$.

Definition A.4 (Entry product). We say that $\tau \in \mathcal{R}$ is the entry product of a vector $\mathbf{a} \in \mathcal{R}^{\ell \cdot N}$, denoted $\tau = \text{prod}(\mathbf{a})$, if $\tau = \prod_{i \in [\ell \cdot N]} \mathbf{a}_i$.

Every vector $\mathbf{a} \in \mathcal{R}^N$ can be extended to a unique multilinear polynomial whose evaluations agree with the original entries of $\mathbf{a}$ over the $\log N$ -bit hypercube. A multivariate polynomial is said to be multilinear if the degree of each variable is at most one. We formalize this as follows.

Definition A.5 (Multilinear extension). Let $\mathbf{a} \in \mathcal{R}^N$ be a vector of ring elements. Then $\hat{\mathbf{a}} : \mathcal{R}^{\log N} \rightarrow \mathcal{R}$ is the unique $\log N$ -variate multilinear extension polynomial in which the entries of $\mathbf{a}$ are embedded as evaluations. In particular,

$$\hat{\mathbf{a}}(x_1, \dots, x_{\log N}) := \sum_{\mathbf{k} \in \{0, 1\}^{\log N}} \text{EQ}(\mathbf{k}, \mathbf{x}) \cdot \mathbf{a}_{\text{int}(\mathbf{k})}$$

where

$$\text{EQ}(\mathbf{k}, \mathbf{x}) := \prod_{i \in [\log N]} \mathbf{k}_i \mathbf{x}_i + (1 - \mathbf{k}_i)(1 - \mathbf{x}_i)$$

A.2 Holographic Polynomial IOPs

A public-coin holographic polynomial IOP over a ring family $\mathcal{R}$ for an indexed relation R is specified by a tuple

$$\text{IOP} = (\text{rc}, \text{s}, \text{d}, \mathbf{I}, \mathbf{P}, \mathbf{V}) .$$

Here, $\text{rc}, \text{s}, \text{d} : \{0, 1\}^* \rightarrow \mathbb{N}$ are polynomial-time computable functions and $\mathbf{I}, \mathbf{P}, \mathbf{V}$ are called the *indexer*, *prover*, and *verifier* algorithms. The parameter rc gives the number of rounds of interaction, s gives the number of polynomial oracles sent in each round, and d gives a degree bound on each polynomial.

In the 0-th round, or offline phase, the indexer $\mathbf{I}$ receives as input a ring $\mathcal{R} \in \mathcal{R}$ and an index $\mathbf{i}$ for R , and outputs $\text{s}(0)$ polynomials $p_1^{(0)}, \dots, p_{\text{s}(0)}^{(0)} \in \mathcal{R}[x]$ of degrees at most $\text{d}(|\mathbf{i}|, 0, 1), \dots, \text{d}(|\mathbf{i}|, 0, \text{s}(0))$.

In the online phase, given $(\mathbf{i}, \mathbf{x}, \mathbf{w}) \in R$, the prover $\mathbf{P}$ takes as input $(\mathcal{R}, \mathbf{i}, \mathbf{x}, \mathbf{w})$. The verifier $\mathbf{V}$ takes as input $(\mathcal{R}, \mathbf{x})$ and receives oracle access to the polynomials produced by $\mathbf{I}(\mathcal{R}, \mathbf{i})$. The prover $\mathbf{P}$ and the verifier $\mathbf{V}$ interact over $\text{rc} = \text{rc}(|\mathbf{i}|)$ rounds. For $j \in [\text{rc}]$, in the j -th round of

interaction, the verifier $\mathbf{V}$ sends the message $\rho_j \in \mathcal{R}$ to the prover $\mathbf{P}$. The prover $\mathbf{P}$ responds with $s(j)$ polynomial oracles $p_1^{(j)}, \dots, p_{s(j)}^{(j)} \in \mathcal{R}[x]$. The verifier may query any oracle polynomial oracle $p_i^{(j)}$ at any evaluation point $y \in \mathcal{R}$ any number of times, and receive $p_i^{(j)}(y) \in \mathcal{R}$. At the end of the interaction, the verifier accepts or rejects.

The degree function $\mathbf{d}$ defines the **admissible** prover algorithms considered for completeness and soundness. A possibly malicious prover $\tilde{\mathbf{P}}$ is called admissible for IOP if for every round $j \in [\text{rc}]$ and oracle index $i \in [s(j)]$ we have $\deg(p_i^{(j)}) \leq \mathbf{d}(|\mathbf{i}|, j, i)$. The honest prover $\mathbf{P}$ must be admissible.

The *proof length* l is the sum $l(|\mathbf{i}|) := \sum_{j=0}^{\text{rc}(|\mathbf{i}|)} \sum_{i=1}^{s(j)} \mathbf{d}(|\mathbf{i}|, j, i)$ of all degree bounds in the offline and online phases. The *query complexity* $\mathbf{q}$ is the total number of oracle queries made by the verifier, including those produced by the indexer⁶. Let $\epsilon: \{0, 1\}^* \rightarrow [0, 1]$. We say that IOP has perfect completeness and soundness error ϵ if:

Completeness. For every ring $\mathcal{R} \in \mathbf{R}$ and index-instance-witness tuple $(\mathbf{i}, \mathbf{x}, \mathbf{w}) \in R$, the probability that $\mathbf{P}(\mathcal{R}, \mathbf{i}, \mathbf{x}, \mathbf{w})$ convinces $\mathbf{V}^{\mathbf{I}(\mathcal{R}, \mathbf{i})}(\mathcal{R}, \mathbf{x})$ to accept in the interactive oracle protocol is 1.

Soundness. For every ring $\mathcal{R} \in \mathbf{R}$, index-instance pair $(\mathbf{i}, \mathbf{x}) \notin L(R)$, and admissible prover $\tilde{\mathbf{P}}$, the probability that $\tilde{\mathbf{P}}$ convinces $\mathbf{V}^{\mathbf{I}(\mathcal{R}, \mathbf{i})}(\mathcal{R}, \mathbf{x})$ to accept in the interactive oracle protocol is at most $\epsilon(\mathcal{R}, \mathbf{x})$.

IOP of Proximity. An IOP of proximity (IOPP) is a generalization of a PIOP in which the verifier additionally has query access to the witness. In this work, we primarily give constructions for IOPP.

A.3 Sum-check Protocol over Rings

The sum-check protocol [LFKN92] has been extended to polynomials over rings [CCKP19]. The prover $\mathbf{P}$ and verifier $\mathbf{V}$ receive a sum-check instance $\mathbf{x} = (\mathcal{R}, H, \ell, \tau, \mathcal{C})$, where $\mathcal{R}$ is a ring, H is a subset of $\mathcal{R}$, ℓ is a number of variables, $\tau \in \mathcal{R}$ is a claimed sum, and $\mathcal{C} \subseteq \mathcal{R}$ is a sampling set. The prover $\mathbf{P}$ additionally receives a polynomial $p \in \mathcal{R}[x_1, \dots, x_\ell]$ such that $\sum_{\omega \in H^\ell} p(\omega) = \tau$. The protocol has ℓ rounds and works as follows.

1. For $i = 1, \dots, \ell$:

(a) $\mathbf{P}$ sends to $\mathbf{V}$ the polynomial:

$$q_i(x) := \sum_{\omega_{i+1}, \dots, \omega_\ell \in H} p(r_1, \dots, r_{i-1}, x, \omega_{i+1}, \dots, \omega_\ell) \in \mathcal{R}[x];$$

(b) $\mathbf{V}$ sends to $\mathbf{P}$ a random challenge $r_i \leftarrow \mathcal{C}$.

2. $\mathbf{V}$ checks that $\sum_{\omega_1 \in H} q_1(\omega_1) = \tau$ and, for $i \in \{2, \dots, \ell\}$, that

$$\sum_{\omega_i \in H} q_i(\omega_i) = q_{i-1}(r_{i-1}).$$

⁶“Proof length” is the standard terminology used for interactive oracle proofs and refers to the total length of the oracles that get sent during the interaction. When an IOP is compiled into a succinct protocol with a commitment scheme, the prover will commit to these oracles, which then gets absorbed into the concrete prover cost as “commitment time”.

3. If the checks pass then $\mathbf{V}$ sets $v := q_\ell(r_\ell) \in \mathcal{R}$ and outputs the tuple $((r_1, \dots, r_\ell), v)$.

The security guarantee of the sum-check protocol requires $\mathcal{C}$ to be a sampling set and follows from a generalization of the Schwartz–Zippel lemma over rings (see Lemma 3.6). The proof of Lemma 3.6 follows the same approach as the usual inductive proof of the standard Schwartz–Zippel lemma. The properties of $\mathcal{C}$ are used to establish that a polynomial $f \in \mathcal{R}[x]$ of degree D has at most D roots in $\mathcal{C}$, which in turn is used in the base case and in the inductive step.

Lemma A.6. *Let $\mathbb{x}_{\text{SC}} = (\mathcal{R}, H, \ell, \tau, \mathcal{C})$ be a sum-check instance and a polynomial $p \in \mathcal{R}[x_1, \dots, x_\ell]$ of total degree d . If $\mathcal{C}$ is a sampling set for $\mathcal{R}$ then the following properties hold.*

Completeness. *If $\sum_{\omega \in H^\ell} p(\omega) = \tau$, then $\Pr_{\mathbf{r} \leftarrow \mathcal{C}^\ell} [\langle \mathbf{P}(\mathbb{x}_{\text{SC}}, p), \mathbf{V}(\mathbb{x}_{\text{SC}}; \mathbf{r}) \rangle = (\mathbf{r}, p(\mathbf{r}))] = 1$.*

Soundness. *If $\sum_{\omega \in H^\ell} p(\omega) \neq \tau$, then for every $\tilde{P}_{\text{SC}}$, $\Pr_{\mathbf{r} \leftarrow \mathcal{C}^\ell} [\langle \tilde{P}_{\text{SC}}, \mathbf{V}(\mathbb{x}_{\text{SC}}; \mathbf{r}) \rangle = (\mathbf{r}, p(\mathbf{r}))] < \frac{\ell d}{|\mathcal{C}|}$.*

Above $\langle A, \mathbf{V}(\mathbb{x}_{\text{SC}}; \mathbf{r}) \rangle$ is the output of $\mathbf{V}(\mathbb{x}_{\text{SC}})$ when interacting with algorithm A using randomness $\mathbf{r}$.

B Sub-protocols

We provide PIOPs for scalar product, checking that a ring element has binary coefficients, Hadamard product, cyclic-shift, and entry-product relations. The PIOPs for cyclic-shift, and entry-product are adapted from [BCG20].

B.1 Scalar Products

Definition B.1. *The scalar product relation R_{SP} is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, N, \tau), (\mathbf{a}, \mathbf{b}))$$

where $\mathbf{a}, \mathbf{b} \in \mathcal{R}^N$, $\tau \in \mathcal{R}$, and $\langle \mathbf{a}, \mathbf{b} \rangle = \tau$.

Construction B.2. Let $\mathcal{R}$ be a ring with sampling space $\mathcal{C}$. Let $N = 2^n$. We construct a PIOP for the indexed relation R_{SP} . The prover $\mathbf{P}$ takes as input an index $\mathbf{i} = \perp$, instance $\mathbf{x} = (\mathcal{R}, \tau)$, and witness $\mathbf{w} = (\mathbf{a}, \mathbf{b})$; the verifier $\mathbf{V}$ takes as input the index $\mathbf{i}$ and the instance $\mathbf{x}$ and has query access to the witness given by multilinear extension oracles $\hat{\mathbf{a}}, \hat{\mathbf{b}}$.

- Let $H := \{0, 1\}$. The prover $\mathbf{P}$ and verifier $\mathbf{V}$ run a sum-check protocol over $\mathcal{R}$ to show that

$$\sum_{\omega \in H^n} \hat{\mathbf{a}} \cdot \hat{\mathbf{b}} = \tau. \quad (23)$$

- The verifier $\mathbf{V}$ for the multivariate sum-check protocol outputs a claim that

$$(\hat{\mathbf{a}} \cdot \hat{\mathbf{b}})(\rho_0, \dots, \rho_{n-1}) = \nu$$

where $\rho_0, \dots, \rho_{n-1}$ is the verifier randomness used in the sum-check protocol.

- The verifier makes MLE evaluation queries to $\hat{\mathbf{a}}$ and $\hat{\mathbf{b}}$ to obtain the evaluations

$$A = \hat{\mathbf{a}}(\rho_0, \dots, \rho_{n-1})$$

and

$$B = \hat{\mathbf{b}}(\rho_0, \dots, \rho_{n-1})$$

- The verifier checks whether $A \cdot B = \nu$.

Lemma B.3. *For every ring $\mathcal{R}$ with sampling space $\mathcal{C}$, positive integer N , Construction B.2 is a PIOP for R_{SP} that has the following efficiency parameters*

- Soundness error: $O(\frac{\log N}{|\mathcal{C}|})$
- Round complexity: $O(\log N)$
- Proof length: $O(N)$
- Prover cost: $O(N)$
- Verifier cost: $O(\log N)$
- Query complexity: $O(1)$

Proof. The efficiency is dominated by a sum-check with $\log N$ rounds. We omit a formal proof as it follows from extending the strawman construction in [BCG20, Section 2.4] to rings. $\square$

B.2 Hadamard Product

Definition B.4. The *Hadamard product* relation R_{HP} is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, N), (\mathbf{a}, \mathbf{b}, \mathbf{c}))$$

where $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathcal{R}^N$ and $\mathbf{a} \circ \mathbf{b} = \mathbf{c}$.

Construction B.5. Let $\mathcal{R}$ be a suitable ring with sampling space $\mathcal{C}$ and let $H = \{0, 1\}$. We construct a PIOP for the indexed relation R_{HP} . The prover $\mathbf{P}$ takes as input an index $\mathbf{i} = \perp$, instance $\mathbf{x} = (\mathcal{R}, N)$, and witness $\mathbf{w} = (\mathbf{a}, \mathbf{b}, \mathbf{c})$; the verifier $\mathbf{V}$ takes as input the index $\mathbf{i}$ and the instance $\mathbf{x}$ and has query access to the witness given by multilinear extension oracles $\hat{\mathbf{a}}$, $\hat{\mathbf{b}}$, and $\hat{\mathbf{c}}$.

- The verifier $\mathbf{V}$ sends uniformly random challenge $\mathbf{y} \in \mathcal{C}^{\log N}$.
- The prover $\mathbf{P}$ and verifier $\mathbf{V}$ engage in a sum-check protocol over $\mathcal{R}$ for the following relation

$$\sum_{\omega \in H^{\log N}} \text{EQ}(\omega, \mathbf{y}) \cdot (\hat{\mathbf{a}}(\omega) \cdot \hat{\mathbf{b}}(\omega) - \hat{\mathbf{c}}(\omega)) = 0$$

- At the end of the sum-check protocol, the verifier outputs a claim ν and uses $\hat{\mathbf{a}}$, $\hat{\mathbf{b}}$, $\hat{\mathbf{c}}$, and EQ to make a query at the point $\boldsymbol{\rho} = (\rho_0, \dots, \rho_{n-1})$, where $\rho_0, \dots, \rho_{n-1}$ is the verifier's randomness used in the sum-check protocol. In particular, the verifier checks that

$$\text{EQ}(\boldsymbol{\rho}, \mathbf{y}) \cdot (\hat{\mathbf{a}}(\boldsymbol{\rho}) \cdot \hat{\mathbf{b}}(\boldsymbol{\rho}) - \hat{\mathbf{c}}(\boldsymbol{\rho})) = \nu$$

Lemma B.6. For every ring $\mathcal{R}$ with sampling space $\mathcal{C}$ and positive integer N , Construction B.5 is a PIOP for R_{HP} that has the following efficiency parameters

- Soundness error: $O(\frac{\log N}{|\mathcal{C}|})$
- Round complexity: $O(\log N)$
- Proof length: $O(N)$
- Prover cost: $O(N)$
- Verifier cost: $O(\log N)$
- Query complexity: $O(1)$

Proof. The asymptotics arise from a multilinear sum-check with $\log N$ rounds. We omit a formal proof. $\square$

B.3 Cyclic-shift test

Definition B.7. The *cyclic shift* relation R_{CS} is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, N), (\mathbf{a}, \mathbf{b}))$$

where $\mathbf{b} = \text{shift}(\mathbf{a})$.

Lemma B.8. Let $\mathcal{R}$ be a suitable ring with challenge space $\mathcal{C}$. Let $\mathbf{a} \in \mathcal{R}^N$ and $\mathbf{b} \in \mathcal{R}^N$. Let $A(x) := \sum_{i=1}^N \mathbf{a}_i x^{i-1}$ and $B(x) := \sum_{i=1}^N \mathbf{b}_i x^{i-1}$ be polynomials in $\mathcal{R}[x]$. Then $(\perp, (\mathcal{R}, N), (\mathbf{a}, \mathbf{b})) \in R_{\text{CS}}$ if and only if

$$A(x) - x \cdot B(x) = (1 - x^N) \cdot \mathbf{b}_N$$

over $\mathcal{R}[x]$.

Construction B.9. Let $\mathcal{R}$ be a ring with sampling space $\mathcal{C}$. Let $N \in \mathbb{N}$. We construct a PIOP for the relation R_{CS} . The prover $\mathbf{P}$ takes as input an index $\mathfrak{i} = \perp$, instance $\mathfrak{x} = (\mathcal{R}, N)$, and witness $\mathfrak{w} = (\mathbf{a}, \mathbf{b})$. The verifier $\mathbf{V}$ takes as input the index $\mathfrak{i}$ and the instance $\mathfrak{x}$ and has query access to the witness given by multilinear extension oracles $\hat{\mathbf{a}}, \hat{\mathbf{b}}$.

- The verifier $\mathbf{V}$ samples random $\gamma \leftarrow \mathcal{C}$ and sends γ to the prover.
- The prover $\mathbf{P}$ and $\mathbf{V}$ engage in scalar-product protocols for

$$\langle \mathbf{a}, \gamma \rangle = \nu_0$$

and

$$\langle \mathbf{b}, \gamma \rangle = \nu_1$$

where $\gamma := (1, \gamma, \dots, \gamma^{N-1})$ and $\nu_0, \nu_1 \in \mathcal{R}$ are the prover's claims for the scalar product.

- At the end of the scalar-product protocols, the verifier $\mathbf{V}$ directly evaluates the multi-linear extension of γ at a random point, where the randomness is chosen during the sum-check within the scalar-product protocol.
- The verifier makes a single oracle query to $\hat{\mathbf{b}}$ at the vector $\mathbf{1}^{\log N}$ to obtain the value $\mathbf{b}_N$.
- The verifier $\mathbf{V}$ checks that

$$\nu_0 - \gamma \cdot \nu_1 = (1 - \gamma^N) \cdot \mathbf{b}_N$$

Lemma B.10. For every ring $\mathcal{R}$ with sampling space $\mathcal{C}$, positive integer N , Construction B.9 is a PIOP for R_{CS} that has the following efficiency parameters

- Soundness error: $O(\frac{N}{|\mathcal{C}|})$
- Round complexity: $O(\log N)$
- Proof length: $O(N)$
- Prover cost: $O(N)$
- Verifier cost: $O(\log N)$
- Query complexity: $O(1)$

Proof. We omit a formal proof but remark that the proof length is dominated by the cost of the underlying scalar-product protocols. The verifier cost is $O(\log N)$ because the MLE $\hat{\gamma}$ can be evaluated in $\log N$ time since $\hat{\gamma}(x_1, \dots, x_{\log N}) = \prod_{i \in [\log N]} x_i \cdot \gamma^{2^{i-1}} + 1 - x_i$. The soundness is $O(\frac{N}{|\mathcal{C}|})$ from the univariate Schwartz-Zippel test in the last step. $\square$

B.4 Entry-product protocol

Definition B.11. The **entry-product** relation R_{EP} is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, \ell, N, \tau), \mathbf{a})$$

where $N \in \mathbb{N}$, $\mathbf{a} \in \mathcal{R}^N$ and $\tau = \text{prod}(\mathbf{a})$.

Construction B.12 (entry-product). Let $\mathcal{R}$ be a suitable ring with sampling space $\mathcal{C}$. Let $H = \{0, 1\}$. We construct a polynomial IOP of proximity $\text{IOPP} = (\mathbf{P}, \mathbf{V})$ for the relation R_{EP} . The prover $\mathbf{P}$ takes as input an instance $\mathbf{x} = (\mathcal{R}, \ell, N, \tau)$, and witness $\mathbf{w} = \mathbf{a}$; the verifier $\mathbf{V}$ takes as input the instance $\mathbf{x}$ and has query access to the witness given by multilinear extension oracle $\widehat{\mathbf{a}}$.

- The prover $\mathbf{P}$ computes the partial products $\{\prod_{i \leq \kappa} \mathbf{a}_i\}_{\kappa \in [N]}$. Let $\mathbf{c}, \mathbf{d}, \mathbf{e} \in \mathcal{R}^N$ be defined as follows:

$$\begin{aligned} \mathbf{c} &:= (1, \mathbf{a}_1, \mathbf{a}_1 \cdot \mathbf{a}_2, \dots, \prod_{i=1}^{N-1} \mathbf{a}_i) , \\ \mathbf{d} &:= (\mathbf{a}_1, \mathbf{a}_1 \cdot \mathbf{a}_2, \dots, \prod_{i=1}^N \mathbf{a}_i) , \\ \mathbf{e} &:= (\mathbf{a}_1, \mathbf{a}_1 \cdot \mathbf{a}_2, \dots, \prod_{i=1}^{N-1} \mathbf{a}_i, 1) . \end{aligned}$$

The prover computes oracles $(\widehat{\mathbf{c}}, \widehat{\mathbf{d}}, \widehat{\mathbf{e}})$ and sends them to the verifier.

- The prover $\mathbf{P}$ and verifier $\mathbf{V}$ engage in a Hadamard-product protocol with $\mathbf{x} = (\mathcal{R}, N)$ and $\mathbf{w} = (\mathbf{a}, \mathbf{c}, \mathbf{d})$ to show that $\mathbf{a} \circ \mathbf{c} = \mathbf{d}$.
- The prover $\mathbf{P}$ and verifier $\mathbf{V}$ engage in a cyclic-shift protocol for the claim $\mathbf{e} = \text{shift}(\mathbf{c})$. In the process, the verifier learns $\mathbf{e}_N$, and rejects if it is not equal to 1.
- The verifier $\mathbf{V}$ samples random $\boldsymbol{\eta} \in \mathcal{R}^{\log N}$ and checks

$$\widehat{\mathbf{d}}(\boldsymbol{\eta}) - \widehat{\mathbf{e}}(\boldsymbol{\eta}) = \text{EQ}(\mathbf{1}^{\log N}, \boldsymbol{\eta})(\tau - 1)$$

where EQ can be evaluated directly.

Lemma B.13. For every suitable ring $\mathcal{R}$ with sampling space $\mathcal{C}$ and positive integer N , Construction B.12 is a PIOP for the indexed relation R_{EP} that has the following parameters:

- Soundness error: $O(N/|\mathcal{C}|)$
- Round complexity: $O(\log(N))$
- Communication: $O(\log N)$
- Proof length: $O(N)$
- Prover cost: $O(N)$

- *Verifier cost:* $O(\log N)$
- *Query complexity:* $O(1)$

Proof. We give an informal analysis. The prover cost and proof length are $O(N)$ from the prover computing and sending $(\mathbf{c}, \mathbf{d}, \mathbf{e}) \in \mathcal{R}^{3N}$ to the verifier. The round complexity $O(\log N)$ is dominated by the sum-checks in the underlying Hadamard-product and cyclic-shift protocols. The soundness error is $O(N/|\mathcal{C}|)$ from the underlying cyclic-shift protocols. The verifier cost is $O(\log N)$ because the verifier engaged in sum-checks of size $O(\log N)$ and evaluates $\text{EQ}(\mathbf{1}^{\log N}, \boldsymbol{\eta})$ once. $\square$

B.5 Integer Check

Definition B.14. *The integer check relation R_{IC} is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, N), \mathbf{a})$$

where $N \in \mathbb{N}$, $\mathbf{a} \in \mathbb{Z}_q^N$.

Construction B.15. Let $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$. We present a PIOP for the relation R_{IC} . The prover $\mathbf{P}$ takes as input an instance $\mathbf{x} = (\mathcal{R}, N)$ and witness $\mathbf{w} = \mathbf{a}$. The verifier takes as input the instance $\mathbf{x}$ and has oracle access to the witness given by $\hat{\mathbf{a}}$.

- The verifier $\mathbf{V}$ samples a vector $\boldsymbol{\rho} \in \mathbb{Z}_q^{\log N}$ and makes a single oracle query to obtain $\tau = \hat{\mathbf{a}}(\boldsymbol{\rho})$.
- If $\tau \notin \mathbb{Z}_q$, then the verifier will reject.

We can amplify soundness and achieve negligible error by repeating the above protocol security-parameter many times.

Lemma B.16. *For every ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$, positive integer N , Construction B.15 is a PIOP for R_{IC} with the following efficiency parameters*

- *Soundness error:* $O(\frac{\log N}{q})$
- *Round complexity:* $O(1)$
- *Proof length:* $O(N)$
- *Prover cost:* $O(1)$
- *Verifier cost:* $O(1)$
- *Query complexity:* $O(1)$

Proof. We first prove completeness and soundness.

Let $H = \{0, 1\}$. Completeness follows by inspection, since the evaluation of the MLE takes a linear combination of the integer entries in $\mathbf{a}$ as follows

$$\begin{aligned} \tau &= \hat{\mathbf{a}}(\rho_0, \dots, \rho_{n-1}) = \sum_{\boldsymbol{\omega} \in H^{\log N}} \text{EQ}(\boldsymbol{\omega}, \boldsymbol{\rho}) \cdot \mathbf{a}_{\text{int}(\boldsymbol{\omega})} \\ &= \sum_{\boldsymbol{\omega} \in H^{\log N}} \left(\prod_{i \in \log N} \omega_i \rho_i + (1 - \omega_i)(1 - \rho_i) \right) \cdot \mathbf{a}_{\text{int}(\boldsymbol{\omega})+1} \end{aligned} \quad (24)$$

For $\mathbf{a}_i \in \mathbb{Z}_q^N$ and $\boldsymbol{\rho} \in \mathbb{Z}_q^{\log N}$, we have $\tau \in \mathbb{Z}_q$.

We now prove soundness. Fix a cheating prover $\tilde{\mathbf{P}}$ such that $\mathbf{a} \notin \mathbb{Z}_q^N$. Then there exists some index $i \in [N]$ such that $\mathbf{a}_i \in \mathcal{R}$ is a ring element represented by a non-constant polynomial. Namely

$$\mathbf{a}_i \leftrightarrow a_0 + a_1x + \dots + a_{d-1}x^{d-1}$$

where there exists $j \in [d-1]$ such that $a_j \neq 0$.

Observe that evaluating the MLE $\hat{\mathbf{a}}$ at an integer-vector $\boldsymbol{\rho}$ produces the linear combination

$$\sum_{i \in [N]} z_i \mathbf{a}_i$$

where $z_i = \text{EQ}(\text{bin}(i-1), \boldsymbol{\rho}) \in \mathbb{Z}_q$. Let us view this linear combination over the coefficient basis of entries in $\mathbf{a}$. Fix a monomial x^j (i.e., the monomial term which contains non-zero coefficients). By a Schwartz-Zippel argument over the coefficient basis on the monomial x^j , the probability that the coefficient on x^j is zero is at most $\frac{\log N}{q}$. If we repeat the protocol λ times, we achieve soundness error $(\frac{\log N}{q})^\lambda$. □

B.6 Binary Check

In this section, we adapt the range proof from LatticeFold+ [BC25] to a PIOP (without commitments) for the simpler setting when the range is just $\{0, 1\}$. The range proof can be used to show that a vector of ring elements $\mathbf{f} \in \mathcal{R}^N$ have $\mathbb{Z}_q$ -coefficients within a publicly known range. We refer to Sections 4.2 and 4.3 of LatticeFold+ for full details.

Definition B.17 (Tensor product). *For a vector $\mathbf{r} \in \mathcal{R}^{\log N}$ we define the tensor product*

$$\otimes \mathbf{r} := \otimes_{i \in [\log N]} (1 - \mathbf{r}_i, \mathbf{r}_i) \in \mathcal{R}^N$$

Definition B.18. *The **binary check** relation R_{BC} is the set of tuples*

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathcal{R}, N), \mathbf{f})$$

where $N \in \mathbb{N}$, $\mathbf{f} \in \mathcal{R}^N$, and each $\mathbf{f}_i \in \mathcal{R}$ has binary coefficients.

Construction B.19 (LatticeFold+ Range Check PIOP). Let $\mathcal{R}$ be a suitable ring with sampling space $\mathcal{C} = \mathbb{Z}_q$. Let $H = \{0, 1\}$ and $\mathcal{M} := \{0, 1, x, \dots, x^{d-1}\} \subseteq \mathcal{R}$ be the set of monomials of $\mathcal{R}$. Prover gets input witness $f \in \mathcal{R}^n$. We construct a polynomial IOP of proximity $\text{IOPP} = (\mathbf{P}, \mathbf{V})$ for the relation R_{BC} . The prover $\mathbf{P}$ takes as input an instance $\mathbf{x} = (\mathcal{R}, N)$, and witness $\mathbf{w} = \mathbf{f}$; the verifier $\mathbf{V}$ takes as input the instance $\mathbf{x}$ and has query access to the witness given by multilinear extension oracle $\hat{\mathbf{f}}$.

Batched monomial set check.

1. The prover $\mathbf{P}$ computes the monomial matrix $M_{\mathbf{f}} \in \mathcal{R}^{d \times N}$ where $M_{\mathbf{f}}[j, i] = x^{\mathbf{f}_i, j} \in \mathcal{R}$ (i.e., the monomial with exponent the j -th coefficient of $\mathbf{f}_i \in \mathcal{R}$) and sends oracles for the multilinear extensions, $\widehat{M_{\mathbf{f}, j}}$ for $j \in [d]$, of each row of $M_{\mathbf{f}}$.

2. The verifier $\mathbf{V}$ sends challenges $\mathbf{c} \leftarrow \mathcal{C}^{\log N}$ and $\beta \leftarrow \mathcal{C}$
3. The prover $\mathbf{P}$ treats the ring element of $M_{\mathbf{f},j}$ for each row $j \in [d]$ as polynomials and evaluates said polynomial at β and β^2 . This produces vectors $\mathbf{m}^{(j)}, \mathbf{m}'^{(j)} \in \mathbb{Z}_q^N$:

$$\begin{aligned}\mathbf{m}^{(j)} &= (\text{ev}_{0,j}(\beta), \dots, \text{ev}_{N-1,j}(\beta)) \\ \mathbf{m}'^{(j)} &= (\text{ev}_{0,j}(\beta^2), \dots, \text{ev}_{N-1,j}(\beta^2)).\end{aligned}$$

The prover $\mathbf{P}$ computes oracles for the multilinear extensions $\widehat{\mathbf{m}}^{(j)}$ and $\widehat{\mathbf{m}'}^{(j)}$ for all $j \in [d]$ rows where $\text{ev}_{e_j}(\beta)$ represents the evaluation of polynomial e_j at input β . Implicitly, these define the following sum-check claims for $j \in [d]$:

$$\sum_{\omega \in H^{\log N}} \text{EQ}(\mathbf{c}, \omega) (\widehat{\mathbf{m}}^{(j)}(\omega)^2 - \widehat{\mathbf{m}'}^{(j)}(\omega)) = 0.$$

The prover $\mathbf{P}$ sends oracles $\widehat{\mathbf{m}}^{(j)}$ and $\widehat{\mathbf{m}'}^{(j)}$ for all $j \in [d]$ rows.

4. The verifier $\mathbf{V}$ samples batching challenge $\alpha \leftarrow \mathcal{C}$ and sends to prover.
5. The prover $\mathbf{P}$ and verifier $\mathbf{V}$ engage in a batched sum-check claim for the relation

$$\sum_{\omega \in H^{\log N}} \text{EQ}(\mathbf{c}, \omega) \cdot \left[\sum_{j \in [d]} \alpha^j (\widehat{\mathbf{m}}^{(j)}(\omega)^2 - \widehat{\mathbf{m}'}^{(j)}(\omega)) \right] = 0.$$

At the end of this sum-check, the verifier has the sum-check claim $z \in \mathcal{R}$ and the sum-check randomness $\mathbf{r}$.

6. The verifier $\mathbf{V}$ makes $2d$ oracle calls to $\widehat{\mathbf{m}}^{(j)}(\mathbf{r})$ and $\widehat{\mathbf{m}'}^{(j)}(\mathbf{r})$ for $j \in [d]$ to check the sum-check claims

$$\text{EQ}(\mathbf{c}, \mathbf{r}) \cdot \left[\sum_{j \in [d]} \alpha^j (\widehat{\mathbf{m}}^{(j)}(\mathbf{r})^2 - \widehat{\mathbf{m}'}^{(j)}(\mathbf{r})) \right] = z.$$

7. The prover $\mathbf{P}$ sends evaluations of each $\widehat{M}_{\mathbf{f},j}$ for $j \in [d]$ at the randomness $\mathbf{r}$, given by

$$e_j = \widehat{M}_{\mathbf{f},j}(\mathbf{r}) \in \mathcal{R}$$

and sends all evaluations e_j to the verifier.

8. The verifier $\mathbf{V}$ views each $e_j \in \mathcal{R}$ as a polynomial and evaluates it at β and β^2 to check the following batched claim:

$$\text{EQ}(\mathbf{c}, \mathbf{r}) \cdot \left[\sum_{j \in [d]} \alpha^j (\text{ev}_{e_j}(\beta)^2 - \text{ev}_{e_j}(\beta^2)) \right] = z$$

9. The prover $\mathbf{P}$ and the verifier $\mathbf{V}$ run d scalar-products (Construction B.2) to confirm that $\langle M_{\mathbf{f},j}, \otimes \mathbf{r} \rangle = e_j$ for each row $j \in [d]$ ⁷.

Consistency check on $\mathbf{CF}(\mathbf{f})$ and $\mathbf{f}$.

10. The prover $\mathbf{P}$ computes $\mathbf{CF}(\mathbf{f}) \in \mathbb{Z}_q^{d \times N}$ such that each column contains the coefficients of each ring elements in $\mathbf{f}$. The prover $\mathbf{P}$ computes the matrix-vector multiplication

$$\mathbf{CF}(\mathbf{f}) \cdot (\otimes \mathbf{r}) = \mathbf{v} \in \mathbb{Z}_q^d$$

the scalar-product

$$w = \langle \mathbf{f}, \otimes \mathbf{r} \rangle \in \mathcal{R},$$

and the oracles of the multilinear extensions $\widehat{\mathbf{CF}_j(\mathbf{f})}$ for $j \in [d]$ where $\mathbf{CF}_j(\mathbf{f})$ is the j^{th} row of $\mathbf{CF}(\mathbf{f})$. The prover $\mathbf{P}$ sends $\widehat{\mathbf{CF}_j(\mathbf{f})}$, $\mathbf{v} \in \mathbb{Z}_q^d$, and $w \in \mathcal{R}$ to the verifier.

11. The prover $\mathbf{P}$ and the verifier $\mathbf{V}$ engage in $d+1$ scalar product protocols (Construction B.2) as follows.

- They run d scalar-product protocols for the claims

$$\langle \mathbf{CF}_j(\mathbf{f}), \otimes \mathbf{r} \rangle = v_j \in \mathbb{Z}_q$$

for all $j \in [d]$.

- They run a scalar-product protocol for the claim

$$\langle \mathbf{f}, \otimes \mathbf{r} \rangle = w \in \mathcal{R}$$

12. The verifier $\mathbf{V}$ treats v_j for $j \in [d]$ as coefficients of a single ring element $\tilde{v} := \sum_{j=0}^{d-1} v_j x^j$ and checks that $w = \tilde{v} \in \mathcal{R}$

Range proof on $\mathbf{f}$.

13. The verifier $\mathbf{V}$ also checks that for all $j \in [d]$

$$\text{ct}(x^{-1} \cdot e_j) = v_j,$$

where $\text{ct}(a)$ denotes the constant coefficient of the ring element $a \in \mathcal{R}$.

Lemma B.20. *For ring $\mathcal{R} = \mathbb{Z}_q[x]/(x^d + 1)$ with strong sampling space $\mathcal{C}$, positive integer N , Construction B.19 is a PIOP for R_{BC} with the following efficiency parameters*

- *Soundness error:* $O(\frac{d+\log N}{|\mathcal{C}|})$

⁷Note that due to the special structure of tensor products the verifier can compute efficiently the evaluation to the multilinear extension of $\otimes \mathbf{r}$ required by the scalar-product

- Round complexity: $O(\log N)$
- Proof length: $O(\text{poly}(d)N)$
- Prover cost: $O(\text{poly}(d)N)$
- Verifier cost: $O(d \log N)$
- Query complexity: $O(d)$

C Plookup Vector Relation Proof

Lemma C.1. *Let $\mathbf{a} \in \mathcal{R}^M$ and $\mathbf{b} \in \mathcal{R}^N$. Then $\mathbf{a} \subseteq \mathbf{b}$ if and only if there exists $\mathbf{w} \in \mathcal{R}^{M+N}$ such that*

$$\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\}_{k \in [M+N]} = \{\mathbf{a}_i, \mathbf{a}_i\}_{i \in [M]} \cup \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}_{j \in [N]} , \quad (25)$$

as multi-sets. In the case that $\mathbf{a} \subseteq \mathbf{b}$, we may take any $\mathbf{w} \in \text{merge}_{\mathbf{b}}(\mathbf{a})$ to satisfy Equation (25).

Proof.

($\implies$) Suppose that $\mathbf{a} \subseteq \mathbf{b}$. Define vector $\mathbf{w} \in \text{merge}_{\mathbf{b}}(\mathbf{a}) \in \mathcal{R}^{M+N}$ to be $\mathbf{a} \parallel \mathbf{b}$, sorted by $\mathbf{b}$ by inserting $\mathbf{a}_i$ to the left of $\mathbf{b}_j$ when $\mathbf{a}_i = \mathbf{b}_j$ for all $i \in [M]$. Then for all $i \in [M]$, by definition of $\mathbf{w}$ there exists a $k \in [M+N]$ such that $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{a}_i, \mathbf{b}_j\} = \{\mathbf{a}_i, \mathbf{a}_i\}$. It now suffices to show that for the remaining N entries of $\mathbf{w}$ which were not accounted for by $\mathbf{a}_i$ it holds that $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}$ for some $j \in [N]$. Let $\mathbf{w}_k$ be one of the remaining entries not yet accounted for. Since $\mathbf{w}$ is sorted by $\mathbf{b}$ and we already removed all repeated entries that arise from inserting $\mathbf{a}_i$ to the left, we know that $\mathbf{w}_k$ is the last “distinct” entry. Thus we must have $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}$ for all remaining N entries. We have exhaustively covered all $M+N$ entries of $\mathbf{w}$ and shown that $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{a}_i, \mathbf{a}_i\}$ or $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}$ which concludes this direction of the proof.

($\impliedby$) Suppose there exists $\mathbf{w} \in \mathcal{R}^{M+N}$ such that

$$\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\}_{k \in [M+N]} = \{\mathbf{a}_i, \mathbf{a}_i\}_{i \in [M]} \cup \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}_{j \in [N]} , \quad (26)$$

as multi-sets. Then for all pairs $\{\mathbf{w}_{k-1}, \text{shift}(\mathbf{w})_{k-1}\}$ and $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\}$ for $k \in \{1, \dots, M+N-1\}$, since $\text{shift}(\mathbf{w})_{k-1} = \mathbf{w}_k$ we have four cases:

1. $\{\mathbf{w}_{k-1}, \text{shift}(\mathbf{w})_{k-1}\} = \{\mathbf{a}_{i_1}, \mathbf{a}_{i_1}\}$ and $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{a}_{i_2}, \mathbf{a}_{i_2}\}$, where $\mathbf{a}_{i_1} = \mathbf{a}_{i_2}$;
2. $\{\mathbf{w}_{k-1}, \text{shift}(\mathbf{w})_{k-1}\} = \{\mathbf{a}_i, \mathbf{a}_i\}$ and $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}$, where $\mathbf{a}_i = \mathbf{b}_j$;
3. $\{\mathbf{w}_{k-1}, \text{shift}(\mathbf{w})_{k-1}\} = \{\mathbf{b}_j, \text{shift}(\mathbf{b})_j\}$ and $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{a}_i, \mathbf{a}_i\}$, where $\mathbf{a}_i = \text{shift}(\mathbf{b})_j = \mathbf{b}_{j+1}$;
4. $\{\mathbf{w}_{k-1}, \text{shift}(\mathbf{w})_{k-1}\} = \{\mathbf{b}_{j_1}, \text{shift}(\mathbf{b})_{j_1}\}$ and $\{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\} = \{\mathbf{b}_{j_2}, \text{shift}(\mathbf{b})_{j_2}\}$ where $\text{shift}(\mathbf{b})_{j_1} = \mathbf{b}_{j_1-1} = \mathbf{b}_{j_2}$.

It suffices to show that for each $i \in [M]$, there is an index $j \in [N]$ such that $\mathbf{a}_i = \mathbf{b}_j$. Since both the left-hand and the right-hand side of Equation (26) contain $M+N$ sets, each set appears on each side exactly once. If we are in Case (2) or Case (3), this directly holds. Also, Case (4) does not involve any element $\mathbf{a}_i$. It is left to show what happens in Case (1).

Case (1) shows that there is an $i' \neq i$ such that $\mathbf{a}_i = \mathbf{a}_{i'}$. This implies that if $(\{\mathbf{w}_{k-1}, \text{shift}(\mathbf{w})_{k-1}\}, \{\mathbf{w}_k, \text{shift}(\mathbf{w})_k\})$ lies in Case (1), then there is a chain of size $C \leq M$ such that for all $c \in [C]$, $(\{\mathbf{w}_{k-1+c}, \text{shift}(\mathbf{w})_{k-1+c}\}, \{\mathbf{w}_{k+c}, \text{shift}(\mathbf{w})_{k+c}\})$ lies in Case (1) (showing that $\mathbf{a}_{i_1} = \mathbf{a}_{i_2} = \dots = \mathbf{a}_{i_C}$). If the chain wraps around vector $\mathbf{w}$, then the subscripts should be computed modulo $[M+N]$. Thus, it must hold that $(\{\mathbf{w}_{k+C}, \text{shift}(\mathbf{w})_{k+C}\}, \{\mathbf{w}_{k+C+1}, \text{shift}(\mathbf{w})_{k+C+1}\})$ lies in Case (2) or Case (3), i.e., there must always exist some “inflection point” $j \in [N]$ such that $\mathbf{a}_{i_1} = \mathbf{a}_{i_2} = \dots = \mathbf{a}_{i_C} = \mathbf{b}_j$. Since for all $i \in [M]$, there is a $j \in [N]$ such that $\mathbf{a}_i = \mathbf{b}_j$, we conclude that $\mathbf{a} \subseteq \mathbf{b}$. $\square$

D Proof of Lemma 5.6

Proof. We first discuss the efficiency parameters claimed in Lemma 5.6. Construction 5.5 is an interactive reduction followed by various sub-protocols, including entry-product protocols invoked on vectors of length $M+N$ and a binary-check invoked on a vector of length M . The entry-product protocol dominates the cost of all of the other efficiency parameters. Thus, all of the other values in Lemma 5.6 apart from the soundness error come from Lemma B.13 for instance size $M+N$.

We now show that Construction 5.5 has soundness error $\epsilon_{\text{LU}} = O(M+N)/|\mathcal{C}| + \frac{d+\log M}{q}$. Suppose that $(i, \mathbf{x}, \mathbf{w}) = ((g([N]), (\mathcal{R}, M, N), (\mathbf{a}, \mathbf{b}, \mathbf{c}))) \notin R_{\text{ILU}}$. Then $\{(\mathbf{a}_i, \mathbf{c}_i)\}_{i \in [M]} \not\subseteq \{(\mathbf{b}_j, g(j))\}_{j \in [N]}$. In other words, there exists some $i \in [M]$ such that, for all $j \in [N]$, we have $(\mathbf{a}_i, \mathbf{c}_i) \neq (\mathbf{b}_j, g(j))$.

Fix a malicious prover, which determines certain next-message functions for oracle messages $\mathbf{b}_\circ, \mathbf{w}, \mathbf{w}_\circ, \mathbf{a}^*(\gamma, \beta), \mathbf{b}^*(\gamma, \beta), \mathbf{w}^*(\gamma, \beta)$; and non-oracle messages $\chi_{\mathbf{a}^*}(\gamma, \beta), \chi_{\mathbf{b}^*}(\gamma, \beta), \chi_{\mathbf{w}^*}(\gamma, \beta)$.

Evaluating the polynomial identity at random $\beta, \gamma \in \mathcal{R}$, by the Schwartz–Zippel lemma and Lemma 5.4, with probability at least $1 - (M+N)/|\mathcal{C}|$, we have

$$\prod_{k=1}^{M+N-1} (\mathbf{w}_k + \alpha \cdot \text{shift}(\mathbf{w})_k + \gamma \cdot \boldsymbol{\sigma}_k - \beta) \neq \prod_{i=1}^{M-1} (\mathbf{a}_i + \alpha \cdot \mathbf{a}_i + \gamma \cdot \mathbf{c}_i - \beta) \cdot \prod_{j=1}^{N-1} (\mathbf{b}_j + \alpha \cdot \text{shift}(\mathbf{b})_j + \gamma \cdot g(j) - \beta) .$$

This implies that one of the following conditions holds.

- $\mathbf{b}_\circ \neq \text{shift}(\mathbf{b})$. By Lemma B.10, the verifier’s cyclic-shift test rejects, except with probability at most $N/|\mathcal{C}|$.
- $\mathbf{w}_\circ \neq \text{shift}(\mathbf{w})$. Similar to the previous case, the verifier rejects except for a soundness error of $(N+M)/|\mathcal{C}|$.
- $\mathbf{a}^* \neq \mathbf{a} + \alpha \cdot \mathbf{a} + \gamma \cdot \mathbf{c} - \beta \cdot 1^M$. With $\eta_1 \in \mathcal{R}^{\log M}$, we apply the Schwartz–Zippel Lemma to the non-zero polynomial $P(\eta_1) := \widehat{\mathbf{a}^*}(\eta_1) - \widehat{\mathbf{a}}(\eta_1) - \alpha \cdot \widehat{\mathbf{a}}(\eta_1) - \gamma \cdot \widehat{\mathbf{c}}(\eta_1) + \beta \cdot \widehat{1^M}(\eta_1)$ of total degree $\log M$, concluding that $\Pr_{\eta_1}[P(\eta_1) = 0] \leq \log(M)/|\mathcal{C}|$. Whenever $P(\eta_1) \neq 0$, the verifier rejects.
- $\mathbf{b}^* \neq \mathbf{b} + \alpha \cdot \mathbf{b}_\circ + \gamma \cdot g([N]) - \beta \cdot 1^N$ or $\mathbf{w}^* \neq \mathbf{w} + \alpha \cdot \mathbf{w}_\circ + \gamma \cdot \boldsymbol{\sigma} - \beta \cdot 1^{M+N}$. Similar to the previous case, the verifier rejects except for a soundness error of $\log(N)/|\mathcal{C}|$ or $\log(M+N)/|\mathcal{C}|$ respectively.

- $\chi_{\mathbf{a}^*} \neq \text{prod}(\mathbf{a}^*)$. By the soundness of the entry-product protocol (Lemma B.13), the verifier accepts with probability at most $\epsilon_{\text{EP}}(M)$.
- $\chi_{\mathbf{b}^*} \neq \text{prod}(\mathbf{b}^*)$ or $\chi_{\mathbf{w}^*} \neq \text{prod}(\mathbf{w}^*)$. Similar to the previous case, the soundness error is $\epsilon_{\text{EP}}(N)$ or $\epsilon_{\text{EP}}(N + M)$.
- $\chi_{\mathbf{w}^*} \neq (1 + \beta)^M \cdot \chi_{\mathbf{a}^*} \cdot \chi_{\mathbf{b}^*}$. The verifier checks this directly, and will therefore reject.

Lastly, a malicious prover passes the well-formedness check on $\mathbf{c}$ with error at most $\text{Soundness}(\text{BC}, M) = \frac{d + \log M}{|\mathcal{C}'|}$ where $\mathcal{C}'$ is the challenge space used for the binary-check sub-protocol. A union bound shows that $\epsilon_{\text{LU}} = \frac{M+N}{|\mathcal{C}|} + \max\{\frac{M+N}{|\mathcal{C}|}, \text{Soundness}(\text{EP}, N), \text{Soundness}(\text{EP}, M), \text{Soundness}(\text{EP}, N + M)\} + \frac{d + \log M}{|\mathcal{C}'|}$, where, e.g. $\text{Soundness}(\text{EP}, M) = \frac{M}{|\mathcal{C}|}$ represents the soundness error of an entry-product protocol on a vector of length M . $\square$

E Ring-LogUp For Tables with Special Entries

In this section, we directly extend the techniques of LogUp [Hab22] to give a lookup argument which works when the lookup table contains only ring elements which are non-zero-divisors and their differences are invertible. As an example the set of short ring elements is a sampling set and contains invertible elements (see Definition 3.5 and Lemma 3.8); thus this construction directly implies a lookup for tables with short entries over these rings. In contrast to Construction 5.11, which requires rings of special form, this construction works for arbitrary rings as long as the lookup contains elements with the above properties. We also require that the size of the lookup table and query are much smaller than the size of the challenge space to ensure that the rational terms in a PIOP are well-defined with high-probability. We begin by presenting a helpful lemma towards proving the full lookup relation.

Lemma E.1. *Let $\mathcal{R}$ be a ring and $\mathcal{C} \subseteq \mathcal{R}$ be a sampling space containing non-zero-divisors. Let $f : \mathcal{C} \rightarrow \mathcal{R}$ be any function. Then for $L_f : \mathcal{C} \rightarrow \mathcal{R}$,*

$$L_f(x) = \sum_{\alpha \in \mathcal{C}} \frac{f(\alpha)}{x - \alpha} = 0$$

over the total quotient ring $\mathcal{R}(x)$, if and only if $f(\alpha) = 0$ for all $\alpha \in \mathcal{C}$.

Proof. Since $\alpha \in \mathcal{C}$, it must be that α is not a zero-divisor; hence $\frac{f(\alpha)}{x - \alpha}$ belongs to the total quotient ring $\mathcal{R}(x)$.

($\Leftarrow$) Suppose that $f(\alpha) = 0$ for all $\alpha \in \mathcal{C}$. Then clearly $L_f(x) = 0$.

($\Rightarrow$) Suppose that $L_f(x) = 0$. Then

$$L_f(x) = 0 \Rightarrow L_f(x) \prod_{\beta \in \mathcal{C}} (x - \beta) = 0 \Rightarrow \sum_{\alpha \in \mathcal{C}} f(\alpha) \cdot \prod_{\beta \in \mathcal{C}, \beta \neq \alpha} (x - \beta) = 0$$

We view $\sum_{\alpha \in \mathcal{C}} f(\alpha) \cdot \prod_{\beta \in \mathcal{C}, \beta \neq \alpha} (x - \beta)$ as a polynomial over $\mathcal{R}[x]$. Let $x = \alpha \in \mathcal{C}$. The product $\prod_{\beta \in \mathcal{C}, \beta \neq \alpha} (\alpha - \beta) \neq 0$ is non-zero since every pairwise difference $\alpha - \beta$ is not a zero-divisor. Indeed if not, then some term $(\alpha - \beta)$ is a zero-divisor in $\mathcal{R}$, which contradicts the fact that $\alpha, \beta \in \mathcal{C}$. Thus we must have $f(\alpha) = 0$ for all $\alpha \in \mathcal{C}$ in $\mathcal{R}[x]$. $\square$

The above lemma is used to show the LogUp relation for tables which contain only short ring elements. This relation can be combined with our check for integer values Construction B.15 to directly construct a simplified lookup PIOP similarly to Construction 5.11. Indeed, evaluating the PIOP is well-defined because the denominator in the rational terms is the difference between the verifier's random challenge and the lookup entry. Since both values are in the challenge space, then for $M, N \ll |\mathcal{C}|$, the probability that the verifier's challenge is equal to a table element is at most $\frac{N}{|\mathcal{C}|}$. Since $|\mathcal{C}|$ is exponentially sized and M, N are polynomial sized, we guarantee that this probability is negligible, and the pairwise differences in the denominators of the rational terms will be non-zero and invertible.

Lemma E.2 (Non-indexed LogUp for elements in a sampling set). *Let $\mathcal{R}$ be a ring and $\mathcal{C} \subseteq \mathcal{R}$ be a sampling set containing non-zero-divisors. Let $\ell := |\mathcal{C}|$ and let $\mathbf{a} \in \mathcal{C}^M$ and $\mathbf{b} \in \mathcal{C}^N$ for $M, N < \ell$. Then $(\perp, ((\mathcal{R}, \mathcal{C}), M, N), (\mathbf{a}, \mathbf{b})) \in R_{\text{LU}}$ if and only if there exists vector $\mathbf{m} \in \mathbb{Z}^N$ such that*

$$\sum_{i \in [M]} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j} \quad (27)$$

over the total quotient ring $\mathcal{R}(x)$.

Proof.

($\implies$) Suppose $\mathbf{a}$ is a valid lookup query into $\mathbf{b}$. This means that for every $i \in [M]$ there exists some $j \in [N]$ such that $\mathbf{a}_i = \mathbf{b}_j$. For $j \in [N]$, let $\mathbf{m}[\mathbf{b}_j]$ denote the number of times $\mathbf{b}_j$ appears in $\mathbf{a}$. Hence, by definition of $\mathbf{m}$ and $\mathbf{c}$, the term $\frac{1}{\mathbf{b}_j + g(j)x_1 + x_2}$ appears exactly $\mathbf{m}[\mathbf{b}_j]$ times on the LHS of Equation (27) which implies that

$$\sum_{i \in [M]} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j}.$$

($\impliedby$) Suppose there exists $\mathbf{m} \in \mathbb{Z}^N$ such that

$$\sum_{i \in [M]} \frac{1}{x - \mathbf{a}_i} = \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j}$$

Let $\mathbf{m}_{\mathbf{a}} \in \mathbb{Z}^{|\mathcal{C}|}$ be the vector such that $\mathbf{m}_{\mathbf{a}}[\alpha]$ is the number of times α appears in $\mathbf{a}$. For $\alpha \notin \mathbf{a}$, let $\mathbf{m}_{\mathbf{a}}[\alpha] = 0$. We also define $\mathbf{m}_{\mathbf{b}}$ such that $\mathbf{m}_{\mathbf{b}}[\mathbf{b}_j] = \mathbf{m}[\mathbf{b}_j]$ and $\mathbf{m}_{\mathbf{b}}[\alpha] = 0$ if $\alpha \notin \mathbf{b}$. Then

$$\begin{aligned} 0 &= \sum_{i \in [M]} \frac{1}{x - \mathbf{a}_i} - \sum_{j \in [N]} \frac{\mathbf{m}[\mathbf{b}_j]}{x - \mathbf{b}_j} \\ &= \sum_{\alpha \in \mathcal{C}} \frac{\mathbf{m}_{\mathbf{a}}[\alpha]}{x - \alpha} - \sum_{\alpha \in \mathcal{C}} \frac{\mathbf{m}_{\mathbf{b}}[\alpha]}{x - \alpha} \\ &= \sum_{\alpha \in \mathcal{C}} \frac{\mathbf{m}_{\mathbf{a}}[\alpha] - \mathbf{m}_{\mathbf{b}}[\alpha]}{x - \alpha} \end{aligned}$$

We define $f : \mathcal{C} \rightarrow \mathcal{R}$ such that $f(x) := \mathbf{m}_\mathbf{a}[x] - \mathbf{m}_\mathbf{b}[x]$. By Lemma E.1, $f(\alpha) = 0$ for all $\alpha \in \mathcal{C}$. Thus $\mathbf{m}_\mathbf{a}[x] = \mathbf{m}_\mathbf{b}[x]$ for all $\alpha \in \mathcal{C}$, which means that for $j \in [N]$, the term $\frac{1}{x-\mathbf{b}_j}$ appears $\mathbf{m}_\mathbf{a}[\mathbf{b}_j] = \mathbf{m}_\mathbf{b}[\mathbf{b}_j] = \mathbf{m}[\mathbf{b}_j]$ times on the LHS of Equation (27). Also, for any $\alpha \notin \mathbf{b}$, then term $\frac{1}{x-\alpha}$ appears $\mathbf{m}_\mathbf{a}[\alpha] = \mathbf{m}_\mathbf{b}[\alpha] = 0$ times on the LHS of Equation (27). Thus, for all $j \in [N]$ the term $\frac{1}{x-\mathbf{b}_j}$ appears $\mathbf{m}[\mathbf{b}_j]$ times on both the LHS and RHS of Equation (27) and we conclude that $\mathbf{a}$ is a multiset of $\mathbf{b}$, satisfying the lookup relation. $\square$

F Proof of Lemma 5.12

Proof. We first discuss the asymptotic efficiency (excluding soundness) and then prove completeness and soundness.

- **Asymptotic Efficiency.** The round complexity is given by the maximum number of rounds for running: parallel sum-checks to confirm $v = \sum_{\mathbf{x} \in \{0,1\}^{\log M}} \hat{A}(x_1, \dots, x_{\log M})$ and $v = \sum_{\mathbf{x} \in \{0,1\}^{\log N}} \hat{B}(x_1, \dots, x_{\log M})$, parallel zero-checks for the well-formedness of $\hat{A}(\mathbf{x})$ and $\hat{B}(\mathbf{x})$, and the binary-check (Construction B.19) on $\mathbf{c}$ at the end.

The proof length is asymptotically dominated by the size of the oracles for $\mathbf{a}$ and $\mathbf{b}$, as well as the overhead from the binary-check protocol, $O(\text{poly}(d)M)$. The proof length from the zero-checks is $O(N + M)$ since the maximum degree of the univariate sent in each round is $O(1)$. A multilinear polynomial with t variables can be described with $O(2^t)$ coefficients, so the final cost is $O(N + \text{poly}(d)M)$.

The prover cost is broken down as follows. First, the prover must compute the multiplicity vector $\mathbf{m}$ which costs $O(\max\{M, N\})$ (i.e., scanning the vector $\mathbf{a}$ and counting multiplicities, then filling in 0 for the remaining $N - M$ entries). Then the prover must write down $A(\mathbf{x})$ and $B(\mathbf{x})$ by computing $O(N + M)$ inverses (the sum-check claim v can simultaneously be computed here). The prover can compute the evaluations of the multilinear extensions $\hat{A}, \hat{P}$ as vectors of size M over the $\log M$ -dimensional hypercube in $O(M)$ time (since each evaluation is $O(1)$ ring arithmetic). Similarly, the evaluation vectors of $\hat{B}, \hat{Q}$ over the $\log N$ -dimensional hypercube can be computed in $O(N)$ time. The evaluations of $\hat{A}, \hat{P}$ at a random vector cost $O(M)$ and the evaluations of $\hat{B}, \hat{Q}$ cost $O(N)$ using known techniques, e.g. [Rot24]. The prover will also participate in $O(\max\{\log M, \log N\})$ rounds of sum-check, wherein each round the prover computes a univariate of degree 1 for a recursive claim, which costs $O(M + N)$ total. The prover performs parallel zero-checks with $O(\max\{\log M, \log N\})$ rounds and in each round compute a univariate of maximum degree 3, which is again $O(M + N)$ cost. Finally, the prover engages in Construction B.19 to perform the binary-check for the length- M vector $\mathbf{c}$ which costs $O(\text{poly}(d)M)$.

The verifier cost comes from $O(\max\{\log M, \log N\})$ rounds of sum-check, where in each round the verifier makes a constant number of univariate polynomial evaluations. The verifier overhead from the binary-check by is $O(d \log M)$.

The query complexity is $O(d)$ because the verifier must make a constant number of oracle calls in the final round of the sum-checks and engages in a binary-check for $\mathbf{c}$. Note that the sum-check evaluations are not oracle calls, but rather the verifier is directly evaluating the univariate polynomial that the prover sent.

- **Correctness.**

- **Completeness.** By inspection, completeness follows from the lattice log-derivatives relation in Lemma 5.8, the integer check on $\mathbf{m}$ (Construction B.15), and the binary check on $\mathbf{c}$ (Construction B.19).
- **Soundness.** Suppose that $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = ((\mathbf{c}, g([N])), (\mathcal{R}, M, N), (\mathbf{a}, \mathbf{b})) \notin R_{\text{LU}}$. Then there is some $\mathbf{a}_i \in \mathbf{a}$ such that $\mathbf{a}_i \neq \mathbf{b}_j$ for all $j \in [N]$. By Lemma 5.8, this means that

$$\sum_{\mathbf{x} \in \{0,1\}^{\log M}} \frac{1}{\mathbf{a}_{\text{int}(x)} + \mathbf{c}_{\text{int}(x)}x_1 - x_2} \neq \sum_{\mathbf{x} \in \{0,1\}^{\log N}} \frac{1}{\mathbf{b}_{\text{int}(x)} + g(\text{int}(x))x_1 - x_2}$$

Consider a verifier that sends challenges α, β and $\tilde{\mathbf{P}}$ that responds with oracles $\hat{A}(\mathbf{x})$ and $\hat{B}(\mathbf{x})$ and sum-check claim v . Then the equality

$$\begin{aligned} \sum_{\mathbf{x} \in \{0,1\}^{\log M}} \hat{A}(\mathbf{x}) &= \sum_{\mathbf{x} \in \{0,1\}^{\log N}} \hat{B}(\mathbf{x}) \\ \sum_{\mathbf{x} \in \{0,1\}^{\log M}} \frac{1}{\mathbf{a}_{\text{int}(x)} + \mathbf{c}_{\text{int}(x)}\alpha - \beta} &= \sum_{\mathbf{x} \in \{0,1\}^{\log N}} \frac{1}{\mathbf{b}_{\text{int}(x)} + g(\text{int}(x))\alpha - \beta} \end{aligned} \quad (28)$$

can be viewed as the evaluation of the bivariate polynomials in the denominators of Lemma 5.8 at the verifier's random challenges $x_1 = \alpha$ and $x_2 = \beta$. To analyze the probability that Equation (28) holds, we re-express as a polynomial identity

$$\begin{aligned} 0 &= \underbrace{\sum_{i \in \{0,1\}^{\log M}} \frac{1}{\hat{\mathbf{a}}(i) + \hat{\mathbf{c}}(i)x_1 - x_2} - \sum_{j \in \{0,1\}^{\log N}} \frac{\hat{\mathbf{m}}(j)}{\hat{\mathbf{b}}(j) + \hat{g}(j)x_1 - x_2}}_{(\star)} \\ 0 &= \left(\prod_{i \in \{0,1\}^{\log M}} (\hat{\mathbf{a}}(i) + \hat{\mathbf{c}}(i)x_1 - x_2) \prod_{j \in \{0,1\}^{\log N}} (\hat{\mathbf{b}}(j) + \hat{g}(j)x_1 - x_2) \right) (\star) \\ &= \sum_{i \in \{0,1\}^{\log M}} \left(\prod_{k \in \{0,1\}^{\log M}, k \neq i} (\hat{\mathbf{a}}(k) + \hat{\mathbf{c}}(k)x_1 - x_2) \prod_{j \in \{0,1\}^{\log N}} (\hat{\mathbf{b}}(j) + \hat{g}(j)x_1 - x_2) \right) - \\ &\quad \sum_{j \in \{0,1\}^{\log N}} \hat{\mathbf{m}}(j) \left(\prod_{i \in \{0,1\}^{\log M}} (\hat{\mathbf{a}}(i) + \hat{\mathbf{c}}(i)x_1 - x_2) \prod_{\ell \in \{0,1\}^{\log N}, \ell \neq j} (\hat{\mathbf{b}}(\ell) + \hat{g}(\ell)x_1 - x_2) \right) \end{aligned}$$

The expression above has total degree $O(M + N)$ from the highest order term $x_1^M x_2^N$.

⁸ By multivariate Schwartz-Zippel, for a random $\alpha \leftarrow \mathcal{C}$ chosen by the verifier, $\tilde{\mathbf{P}}$ will satisfy pass this identity test with probability at most $O(\frac{M+N}{|\mathcal{C}|})$.

Then the sum-check protocol for the claim

$$v = \sum_{\mathbf{x} \in \{0,1\}^{\log M}} \hat{A}(\mathbf{x})$$

⁸Note that the oracle expressions $\hat{a}(i)$, $\hat{b}(j)$, $\hat{m}(j)$, etc., are fixed evaluations points and not the extension polynomials themselves, and hence do not contribute to the degree.

has soundness error $O(\frac{\log M}{|\mathcal{C}|})$. Similarly, the sum-check for $\widehat{B}(x)$ has soundness error $O(\frac{\log N}{|\mathcal{C}|})$. The soundness error for the zero-checks on $\widehat{P}(x)$ and $\widehat{Q}(x)$ are $O(\frac{\log M}{|\mathcal{C}|})$ and $O(\frac{\log N}{|\mathcal{C}|})$. The soundness of the integer-check is $\frac{\log N}{q}$, which can be repeated several times to achieve desirable soundness. We leave this amplification as a tunable parameter when the **PIOP** is compiled to a succinct protocol. The soundness error of the binary-check is $O(\frac{d+\log M}{q})$. By a union bound, we conclude that the overall soundness error is $O(\frac{M+N}{|\mathcal{C}|} + \frac{d+\log M+\log N}{q})$.

□

Acknowledgements

We thank the anonymous reviewers for their helpful comments and suggestions. We also thank Zihan Yu for identifying an issue in an earlier version of our Ring-LogUp PIOP.

References

- [ACK21] Thomas Attema, Ronald Cramer, and Lisa Kohl. *A Compressed Σ -Protocol Theory for Lattices*. Cryptology ePrint Archive, Paper 2021/307. 2021. URL: <https://eprint.iacr.org/2021/307>.
- [ACLMT22] Martin R. Albrecht, Valerio Cini, Russell W. F. Lai, Giulio Malavolta, and Sri AravindaKrishnan Thyagarajan. *Lattice-Based SNARKs: Publicly Verifiable, Pre-processing, and Recursively Composable*. Cryptology ePrint Archive, Paper 2022/941. 2022. URL: <https://eprint.iacr.org/2022/941>.
- [AST23] Arasu Arun, Srinath Setty, and Justin Thaler. *Jolt: SNARKs for Virtual Machines via Lookups*. Cryptology ePrint Archive, Paper 2023/1217. 2023. URL: <https://eprint.iacr.org/2023/1217>.
- [BBCPGL18] Carsten Baum, Jonathan Bootle, Andrea Cerulli, Rafaël del Pino, Jens Groth, and Vadim Lyubashevsky. “Sub-linear Lattice-Based Zero-Knowledge Arguments for Arithmetic Circuits”. In: *Proceedings of the 38th Annual International Cryptology Conference*. CRYPTO ’18. 2018, pp. 669–699.
- [BBF19] Dan Boneh, Benedikt Bünz, and Ben Fisch. “Batching Techniques for Accumulators with Applications to IOPs and Stateless Blockchains”. In: *CRYPTO 2019, Part I*. Ed. by Alexandra Boldyreva and Daniele Micciancio. Vol. 11692. LNCS. Aug. 2019, pp. 561–586. DOI: 10.1007/978-3-030-26948-7_20.
- [BC24] Dan Boneh and Binyi Chen. *LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems*. Cryptology ePrint Archive, Paper 2024/257. 2024. URL: <https://eprint.iacr.org/2024/257>.
- [BC25] Dan Boneh and Binyi Chen. *LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems*. Cryptology ePrint Archive, Paper 2025/247. 2025. URL: <https://eprint.iacr.org/2025/247>.
- [BCG20] Jonathan Bootle, Alessandro Chiesa, and Jens Groth. *Linear-Time Arguments with Sublinear Verification from Tensor Codes*. Cryptology ePrint Archive, Paper 2020/1426. 2020. URL: <https://eprint.iacr.org/2020/1426>.
- [BCGJM18] Jonathan Bootle, Andrea Cerulli, Jens Groth, Sune K. Jakobsen, and Mary Maller. “Arya: Nearly Linear-Time Zero-Knowledge Proofs for Correct Program Execution”. In: *Proceedings of the 24th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’18. 2018, pp. 595–626.
- [BCS21] Jonathan Bootle, Alessandro Chiesa, and Katerina Sotiraki. *Sumcheck Arguments and their Applications*. Cryptology ePrint Archive, Paper 2021/333. 2021. URL: <https://eprint.iacr.org/2021/333>.

- [BCS23] Jonathan Bootle, Alessandro Chiesa, and Katerina Sotiraki. *Lattice-Based Succinct Arguments for NP with Polylogarithmic-Time Verification*. Cryptology ePrint Archive, Paper 2023/930. 2023. URL: <https://eprint.iacr.org/2023/930>.
- [BEGKN91] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. “Checking the correctness of memories”. In: *Proceedings of the 32nd Annual Symposium on Foundations of Computer Science*. FOCS ’91. 1991, pp. 90–99.
- [BGH20] Sean Bowe, Jack Grigg, and Daira Hopwood. *Halo2*. 2020. URL: <https://zcash.github.io/halo2/design/proving-system/lookup.html>.
- [BLMG25] Jonathan Bootle, Vadim Lyubashevsky, and Antonio Merino-Gallardo. “Efficient Verifiable Mixnets from Lattices, Revisited”. In: *IACR International Conference on Public-Key Cryptography*. 2025, pp. 237–270.
- [BLNS20] Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. *A non-PCP Approach to Succinct Quantum-Safe Zero-Knowledge*. Cryptology ePrint Archive, Paper 2020/737. 2020. URL: <https://eprint.iacr.org/2020/737>.
- [BS22] Ward Beullens and Gregor Seiler. *LaBRADOR: Compact Proofs for R1CS from Module-SIS*. Cryptology ePrint Archive, Paper 2022/1341. 2022. URL: <https://eprint.iacr.org/2022/1341>.
- [Bit+14] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Avi Rubin, and Eran Tromer. *The Hunting of the SNARK*. ePrint 2014/580. 2014.
- [CCCFFG25] Matteo Campanelli, Mario Carrillo, Ignacio Cascudo, Dario Fiore, Danilo Francati, and Rosario Gennaro. *On the Power of Polynomial Preprocessing: Proving Computations in Sublinear Time, and More*. Cryptology ePrint Archive, Paper 2025/238. 2025. URL: <https://eprint.iacr.org/2025/238>.
- [CCCFGSV25] Ignacio Cascudo, Anamaria Costache, Daniele Cozzo, Dario Fiore, Antonio Guimarães, and Eduardo Soria-Vazquez. *Verifiable Computation for Approximate Homomorphic Encryption Schemes*. Cryptology ePrint Archive, Paper 2025/286. 2025. URL: <https://eprint.iacr.org/2025/286>.
- [CCKP19] Shuo Chen, Jung Hee Cheon, Dongwoo Kim, and Daejun Park. “Verifiable Computing for Approximate Computation”. In: *IACR Cryptol. ePrint Arch.* (2019), p. 762.
- [CFLL24] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. “Lookup arguments: Improvements, extensions and applications to zero-knowledge decision trees”. In: *IACR International Conference on Public-Key Cryptography*. Springer. 2024, pp. 337–369.
- [CFG25] Matteo Campanelli, Dario Fiore, and Rosario Gennaro. “Natively compatible super-efficient lookup arguments and how to apply them”. In: *Journal of Cryptology* 38.1 (2025), p. 14.
- [CFHKKO22] Matteo Campanelli, Dario Fiore, Semin Han, Jihye Kim, Dimitris Kolonelos, and Hyunok Oh. “Succinct Zero-Knowledge Batch Proofs for Set Accumulators”. In: ACM Press, 2022, pp. 455–469. DOI: 10.1145/3548606.3560677.

- [CGSS23] Arka Rai Choudhuri, Sanjam Garg, Aarushi Goel, Sruthi Sekar, and Rohit Sinha. *SublonK: Sublinear Prover PlonK*. Cryptology ePrint Archive, Paper 2023/902. 2023. URL: <https://eprint.iacr.org/2023/902>.
- [CHMMVW20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS”. In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’20. 2020, pp. 738–768.
- [CL02] Jan Camenisch and Anna Lysyanskaya. “Dynamic Accumulators and Application to Efficient Revocation of Anonymous Credentials”. In: *CRYPTO 2002*. Ed. by Moti Yung. Vol. 2442. LNCS. Aug. 2002, pp. 61–76. DOI: 10.1007/3-540-45708-9_5.
- [CMM19] N ria Costa, Ramiro Mart nez, and Paz Morillo. *Lattice-based proof of a shuffle*. Cryptology ePrint Archive, Paper 2019/357. 2019. URL: <https://eprint.iacr.org/2019/357>.
- [CMNW24] Valerio Cini, Giulio Malavolta, Ngoc Khanh Nguyen, and Hoeteck Wee. “Polynomial Commitments from Lattices: Post-quantum Security, Fast Verification and Transparent Setup”. In: LNCS. 2024, pp. 207–242. DOI: 10.1007/978-3-031-68403-6_7.
- [DGPPS24] Moumita Dutta, Chaya Ganesh, Sikhar Patranabis, Shubh Prakash, and Nitin Singh. “Batching-efficient RAM using updatable lookup arguments”. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 4077–4091.
- [DXNT23] Zijing Di, Lucas Xia, Wilson Nguyen, and Nirvan Tyagi. *MuxProofs: Succinct Arguments for Machine Computation from Vector Lookups*. Cryptology ePrint Archive, Paper 2023/974. 2023. URL: <https://eprint.iacr.org/2023/974>.
- [Dor24] Tesseract Dore. *TaSSLE: Lasso for the commitment-phobic*. Cryptology ePrint Archive, Paper 2024/1075. 2024. URL: <https://eprint.iacr.org/2024/1075>.
- [EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. “cq: Cached quotients for fast lookups”. In: *Cryptology ePrint Archive* (2022).
- [ENS20] Muhammed F. Esgin, Ngoc Khanh Nguyen, and Gregor Seiler. *Practical Exact Proofs from Lattices: New Techniques to Exploit Fully-Splitting Rings*. Cryptology ePrint Archive, Paper 2020/518. 2020. URL: <https://eprint.iacr.org/2020/518>.
- [GK22] Ariel Gabizon and Dmitry Khovratovich. “flookup: Fractional decomposition-based lookups in quasi-linear time independent of table size”. In: *Cryptology ePrint Archive* (2022).
- [GNSV21] Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. *Rinocchio: SNARKs for Ring Arithmetic*. Cryptology ePrint Archive, Paper 2021/322. 2021. URL: <https://eprint.iacr.org/2021/322>.

- [GPS25] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. *Samaritan: Linear-time Prover SNARK from New Multilinear Polynomial Commitments*. Cryptology ePrint Archive, Paper 2025/419. 2025. URL: <https://eprint.iacr.org/2025/419>.
- [GW20] Ariel Gabizon and Zachary J. Williamson. *plookup: A simplified polynomial protocol for lookup tables*. Cryptology ePrint Archive, Paper 2020/315. 2020. URL: <https://eprint.iacr.org/2020/315>.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive, Report 2019/953. 2019.
- [GWHD25] Albert Garreta, Hendrik Waldner, Katerina Hristova, and Luca Dall’Ava. *Zinc: Succinct Arguments with Small Arithmetization Overheads from IOPs of Proximity to the Integers*. Cryptology ePrint Archive, Paper 2025/316. 2025. URL: <https://eprint.iacr.org/2025/316>.
- [Gro16] Jens Groth. “On the Size of Pairing-Based Non-interactive Arguments”. In: *Proceedings of the 35th Annual International Conference on Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’16. 2016, pp. 305–326.
- [HACF25] Hossein Hafezi, Gaspard Anthoine, Matteo Campanelli, and Dario Fiore. *SoK: Lookup Table Arguments*. Cryptology ePrint Archive, Paper 2025/1876. 2025. URL: <https://eprint.iacr.org/2025/1876>.
- [HSS24] Intak Hwang, Jinyeong Seo, and Yongsoo Song. “Concretely efficient lattice-based polynomial commitment from standard assumptions”. In: *Annual International Cryptology Conference*. Springer. 2024, pp. 414–448.
- [HYOK25] Semin Han, Geonho Yoon, Hyunok Oh, and Jihye Kim. “DUPLEX: Scalable Zero-Knowledge Lookup Arguments over RSA Group”. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 2025, pp. 72–86.
- [Hab22] Ulrich Haböck. *Multivariate lookups based on logarithmic derivatives*. Cryptology ePrint Archive, Paper 2022/1530. 2022. URL: <https://eprint.iacr.org/2022/1530>.
- [KU08] Kiran S Kedlaya and Christopher Umans. “Fast modular composition in any characteristic”. In: *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. IEEE. 2008, pp. 146–155.
- [KU11] Kiran S Kedlaya and Christopher Umans. “Fast polynomial factorization and modular composition”. In: *SIAM Journal on Computing* 40.6 (2011), pp. 1767–1802.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Advances in Cryptology - ASIACRYPT 2010*. Ed. by Masayuki Abe. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 177–194. ISBN: 978-3-642-17373-8.
- [LFKN92] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. “Algebraic Methods for Interactive Proof Systems”. In: *Journal of the ACM* 39.4 (1992), pp. 859–868.

- [LNP22] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plancon. *Lattice-Based Zero-Knowledge Proofs and Applications: Shorter, Simpler, and More General*. Cryptology ePrint Archive, Paper 2022/284. 2022. URL: <https://eprint.iacr.org/2022/284>.
- [LS17] Vadim Lyubashevsky and Gregor Seiler. *Short, Invertible Elements in Partially Splitting Cyclotomic Rings and Applications to Lattice-Based Zero-Knowledge Proofs*. Cryptology ePrint Archive, Paper 2017/523. 2017. URL: <https://eprint.iacr.org/2017/523>.
- [LZLHQZ25] Chongrong Li, Pengfei Zhu, Yun Li, Cheng Hong, Wenjie Qu, and Jiaheng Zhang. “HyperPianist: Pianist with Linear-Time Prover and Logarithmic Communication Cost”. In: *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2025, pp. 3383–3401.
- [Mar19] SCIPR Lab. *A Rust library for the Marlin preprocessing zkSNARK*. 2019. URL: <https://github.com/scipr-lab/marlin>.
- [Mer88] Ralph C. Merkle. “A Digital Signature Based on a Conventional Encryption Function”. In: *CRYPTO’87*. Ed. by Carl Pomerance. Vol. 293. LNCS. Aug. 1988, pp. 369–378. DOI: 10.1007/3-540-48184-2_32.
- [NOZ26] Ngoc Khanh Nguyen, George O’Rourke, and Jiapeng Zhang. *Hachi: Efficient Lattice-Based Multilinear Polynomial Commitments over Extension Fields*. Cryptology ePrint Archive, Paper 2026/156. 2026. URL: <https://eprint.iacr.org/2026/156>.
- [NS24] Ngoc Khanh Nguyen and Gregor Seiler. *Greyhound: Fast Polynomial Commitments from Lattices*. Cryptology ePrint Archive, Paper 2024/1293. 2024. DOI: 10.1007/978-3-031-68403-6_8. URL: <https://eprint.iacr.org/2024/1293>.
- [Ngu24] Hai H. Nguyen. “Towards Breaking the Half-Barrier of Local Leakage-Resilient Shamir’s Secret Sharing”. In: LNCS. 2024, pp. 257–285. DOI: 10.1007/978-3-031-68388-6_10.
- [OLB25] Alex Ozdemir, Evan Laufer, and Dan Boneh. “Volatile and Persistent Memory for zkSNARKs via Algebraic Interactive Proofs”. In: IEEE Computer Society Press, 2025, pp. 3309–3327. DOI: 10.1109/SP61157.2025.00054.
- [OWWB20] Alex Ozdemir, Riad S. Wahby, Barry Whitehat, and Dan Boneh. “Scaling Verifiable Computation Using Efficient Set Accumulators”. In: *USENIX Security 2020*. USENIX Association, 2020, pp. 2075–2092. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/ozdemir>.
- [PH23] Shahar Papini and Ulrich Haböck. *Improving logarithmic derivative lookups using GKR*. Cryptology ePrint Archive, Paper 2023/1284. 2023. URL: <https://eprint.iacr.org/2023/1284>.
- [PK22] Jim Posen and Assimakis A Kattis. “Caulk+: Table-independent lookup arguments”. In: *Cryptology ePrint Archive* (2022).
- [PLFT25] Kabir Peshawaria, Zeyu Liu, Ben Fisch, and Eran Tromer. *Laminate: Succinct SIMD-Friendly Verifiable FHE*. Cryptology ePrint Archive, Paper 2025/2285. 2025. URL: <https://eprint.iacr.org/2025/2285>.

- [RU19] Barry Whitehat. *Rollup*. https://github.com/barryWhiteHat/roll_up. 2018.
- [Rot24] Ron D. Rothblum. *A Note on Efficient Computation of the Multilinear Extension*. Cryptology ePrint Archive, Paper 2024/1103. 2024. URL: <https://eprint.iacr.org/2024/1103>.
- [SLZ24] Haochen Sun, Jason Li, and Hongyang Zhang. *zkLLM: Zero Knowledge Proofs for Large Language Models*. 2024. arXiv: 2404.16109 [cs.LG]. URL: <https://arxiv.org/abs/2404.16109>.
- [ST25] Srinath Setty and Justin Thaler. *Twist and Shout: Faster memory checking arguments via one-hot addressing and increments*. Cryptology ePrint Archive, Paper 2025/105. 2025. URL: <https://eprint.iacr.org/2025/105>.
- [STW23] Srinath Setty, Justin Thaler, and Riad Wahby. *Unlocking the lookup singularity with Lasso*. Cryptology ePrint Archive, Paper 2023/1216. 2023. URL: <https://eprint.iacr.org/2023/1216>.
- [Sch80] Jacob T. Schwartz. “Fast Probabilistic Algorithms for Verification of Polynomial Identities”. In: *Journal of the ACM* 27.4 (1980), pp. 701–717.
- [Set20] Srinath Setty. “Spartan: Efficient and general-purpose zkSNARKs without trusted setup”. In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO ’20. Referencing Cryptology ePrint Archive, Report 2019/550, revision from 2020.02.28. 2020.
- [Sou25] Lev Soukhanov. *Logup*: faster, cheaper logup argument for small-table indexed lookups*. Cryptology ePrint Archive, Paper 2025/946. 2025. URL: <https://eprint.iacr.org/2025/946>.
- [TNLKN25] Dien HA Tran, Tam NB Nguyen, Nhien-An Le-Khac, and Thuc D Nguyen. “LogaLookup: Efficient Multivariate Lookup Argument for Accelerated Proof Generation”. In: *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 2025, pp. 215–230.
- [WW24] Hoeteck Wee and David J. Wu. *Lattice-Based Functional Commitments: Fast Verification and Cryptanalysis*. Cryptology ePrint Archive, Paper 2024/028. 2024. URL: <https://eprint.iacr.org/2024/028>.
- [XHTP24] Jiajun Xin, Arman Haghighi, Xiangnan Tian, and Dimitrios Papadopoulos. *Notus: Dynamic Proofs of Liabilities from Zero-knowledge RSA Accumulators*. Cryptology ePrint Archive, Paper 2024/395. 2024. URL: <https://eprint.iacr.org/2024/395>.
- [ZBKMNS22] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. “Caulk: Lookup arguments in sublinear time”. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2022, pp. 3121–3134.
- [ZGKMR22] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Ràfols. “Baloo: Nearly optimal lookup arguments”. In: *Cryptology ePrint Archive* (2022).

- [ZSG24] Yuncong Zhang, Shi-Feng Sun, and Dawu Gu. “Efficient KZG-based univariate sum-check and lookup argument”. In: *IACR International Conference on Public-Key Cryptography*. Springer. 2024, pp. 400–425.
- [Zip79] Richard Zippel. “Probabilistic algorithms for sparse polynomials”. In: *Proceedings of the 1979 International Symposium on Symbolic and Algebraic Computation*. EUROSAM '79. 1979, pp. 216–226.